



University of Glasgow | School of
Computing Science

Honours Individual Project Dissertation

TRUST-AWARE STOCK TREND PREDICTION AND INVESTMENT STRATEGY OPTIMISATION USING DEEP LEARNING

Dylan ShengHong Huang
March 26, 2026

Abstract

Deep learning methods are increasingly being applied to stock market prediction, with growing evidence that they can extract useful structure from financial time-series and related textual signals. However, prediction alone is not sufficient. In practice, model outputs must also be translated into sensible capital allocation decisions if they are to generate meaningful returns on investments.

The framework integrates historical price data with text-derived features and evaluates multiple deep learning architectures across binary, regression and ordinal multi-class forecasting tasks. It further incorporates prior model predictions and their realised errors as meta-features, allowing the system to learn when its own outputs are more or less reliable. In parallel, a variable-horizon design is introduced to compare short-term and longer-horizon forecasting behaviour. Beyond prediction, the dissertation develops and evaluates a downstream staking and allocation layer using Kelly Criterion, Modern Portfolio Theory (MPT) and a hybrid strategy.

Results show that this approach improves decision stability and supports stronger economic performance than accuracy only evaluation would suggest. In particular, longer-horizon classification models produced the most useful trading signals, while reliability-aware meta-features improved performance and risk adjusted return. The strongest investment result was achieved by the binary meta-feature system using MPT allocation, which returned 12.47% over 100 trading days. Overall, the findings show that financially useful stock prediction systems should be assessed not only by forecast accuracy but by how well calibrated and reliability-aware predictions translate into profitable and risk-conscious capital allocation.

Acknowledgements

I would like to express my sincere gratitude to Enrique Alejandro González Amador, a former MSc student at the University of Glasgow School of Computing Science, whose dissertation, *Adapting Natural Language Processing and Deep Learning Tools to an End-to-End Day Trading System (2025)*, provided an important foundation and inspiration for this work.

I would also like to thank my supervisor, Prof. Jose Joemon, who supervised both Enrique's dissertation and my own. His guidance, experience and continued support were invaluable throughout this project. Without his insight and encouragement, this dissertation would not have reached its current standard.

Education Use Consent

I hereby grant my permission for this project to be stored, distributed and shown to other University of Glasgow students and staff for educational purposes. **Please note that you are under no obligation to sign this declaration, but doing so would help future students.**

Signature: Dylan ShengHong Huang Date: 26 March 2026

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Purpose	2
1.2.1	Contributions	2
1.2.2	Novelty	3
1.3	Report Structure	3
2	Background & Related Work	4
2.1	Stock Movement Prediction	4
2.1.1	From Statistical Models to Deep Learning	4
2.1.2	Multimodal Extensions	4
2.2	Deep Learning for Time-Series Modelling	5
2.2.1	Long Short-Term Memory (LSTM)	6
2.2.2	Gated Recurrent Units (GRU)	6
2.2.3	Bidirectional Variants	6
2.2.4	Implications for Stock Movement Prediction	7
2.2.5	Ordinal Regression for Financial Returns	7
2.3	BERT for Natural Language Processing	7
2.4	Probability Calibration	8
2.4.1	Isotonic Regression	9
2.4.2	Temperature Scaling	9
2.5	Meta-Features and Model Self-Assessment	10
2.6	Capital Allocation Strategies	11
2.6.1	The Kelly Criterion	11
2.6.2	Mean-Variance Portfolio Theory (MPT)	11
3	Analysis and Research Questions	12
3.1	Problem Definition	12
3.1.1	Prediction Task	12
3.1.2	Decision Layer	13
3.2	Constraints and Assumptions	13
3.2.1	Temporal Integrity and Information Structure	13
3.2.2	Market Non-Stationarity	13
3.2.3	Capital, Risk and Leverage Constraints	13
3.3	Research Questions	14
4	System Design	15
4.1	Overall Architecture	15

4.2	Data Flow and Temporal Integrity	17
4.3	Model Families and Outputs	17
4.4	Calibration and Meta-Learning Design	17
4.5	Benchmarking and Investment Simulation Design	18
5	Implementation	19
5.1	Data Acquisition and Pre-processing	19
5.2	Feature Engineering	19
5.2.1	Technical Indicators	19
5.2.2	Textual Features	20
5.2.3	Meta-Features	20
5.3	Target Construction and Horizon Design	21
5.3.1	Regression and Binary Targets	21
5.3.2	Ordinal Multi-Class Targets	22
5.3.3	Sequence Horizon Design	22
5.4	Model Selection, Training and Calibration	22
5.4.1	Hyperparameter Tuning and Feature Selection	22
5.4.2	Model Training and Calibration	23
5.5	Investment Simulation Framework	23
5.5.1	Per-Asset Training and Walk-forward Retraining	23
5.5.2	Portfolio Construction	24
5.5.3	Capital Allocation	25
5.5.4	Early Exit Mechanism	26
5.5.5	Portfolio Tracking and Constraints	26
5.6	Reproducibility and Robustness Measures	27
5.6.1	Random Seed Control and Deterministic Execution	27
5.6.2	Data Handling	27
5.6.3	Chronological Splits and No-Shuffle Training	27
6	Experiments & Results	28
6.1	Evaluation Metrics	28
6.1.1	Economic Metrics	28
6.2	Experimental Setup	29
6.2.1	Evaluation Splits	29
6.2.2	Compared Models and Feature Sets	29
6.3	Predictive Performance	29
6.3.1	Binary Classification Results	29
6.3.2	Regression Results	30
6.3.3	Ordinal Multi-Class Results	30
6.4	Meta-Feature and Ablation Results	31
6.4.1	Effect of Meta-Features	31
6.4.2	Effect of Variable Horizon Design	32
6.5	Investment Simulation Results	33
6.5.1	Binary Simulation Results	33
6.5.2	Ordinal Multi-Class Simulation Results	34
6.5.3	Risk and Early Exit	35

6.5.4	Allocation Strategy Comparison	36
7	Discussion	37
7.1	Overview	37
7.2	Interpretation by Research Question	37
7.2.1	RQ1: Predictive Performance	37
7.2.2	RQ2: Meta-Features and Reliability Modelling	37
7.2.3	RQ3: Allocation Strategy Comparison	38
7.2.4	RQ4: Interaction Effects	38
7.3	Limitations	38
7.4	Concluding Discussion and Practical Implications	39
8	Conclusion	40
8.1	Future Work	40
	Appendices	41
A	Appendices	41
A.1	Dataset	41
A.1.1	Core Market Data	41
A.1.2	Technical Indicators	41
A.1.3	Return Features	41
A.1.4	Sentiment, Emotion, and Stance Features	42
A.1.5	Sector-Level Features	42
A.1.6	Meta-Target Columns	42
A.1.7	Regression Meta-Features	43
A.1.8	Classification Meta-Features	43
A.1.9	Complete Column List	43
A.2	Implementation Details	44
A.2.1	Feature Engineering Code	44
A.2.2	Target Construction and Horizon Design	46
A.2.3	Model Hyperparameter Tuning and Selection	47
A.2.4	Early Exit Mechanism	47
A.2.5	Early-Exit Example	48
A.2.6	Portfolio Construction	48
A.2.7	Portfolio Value Calculation	49
A.2.8	Robustness Controls	49
A.2.9	Investment Simulation Parameters	50
A.2.10	Compared Models and Feature Groups	50
A.3	Training Diagnostics	54
A.3.1	Binary Classification Loss Curves	54
A.3.2	Ordinal Multi-Class Loss Curves	56
A.3.3	Regression Loss Curves	57
A.4	Full Predictive Benchmark Results	59
A.4.1	1-Day Binary Classification	59
A.4.2	1-Day Regression	59
A.4.3	1-Day Ordinal Multi-Class	60

A.5	Meta-Feature Experimental Results	60
A.5.1	Binary Classification with Meta-Features	60
A.5.2	Regression with Meta-Features	61
A.5.3	Ordinal Multi-Class with Meta-Features	61
A.6	Variable-Horizon Results	62
A.6.1	Binary Classification: No Meta-Features	62
A.6.2	Binary Classification: With Meta-Features	63
A.6.3	Regression: No Meta-Features	63
A.6.4	Regression: With Meta-Features	64
A.6.5	Ordinal Multi-Class: No Meta-Features	64
A.6.6	Ordinal Multi-Class: With Meta-Features	64
A.7	Investment Simulation Supporting Results	65
A.7.1	Selected Binary Models for Investment Simulation	65
A.7.2	Selected Ordinal Models for Investment Simulation	65
A.7.3	Full Binary Investment Simulation Results	66
A.7.4	Full Ordinal Multi-Class Investment Simulation Results	66
A.7.5	Cumulative Return Figures	67
A.8	Risk and Early-Exit Supplementary Results	68
A.8.1	Binary System Without Meta-Features	69
A.8.2	Binary System With Meta-Features	71
A.8.3	Ordinal Multi-Class System Without Meta-Features	73
A.8.4	Ordinal Multi-Class System With Meta-Features	75
B	Source Code	78
	Bibliography	79

1 | Introduction

1.1 Motivation

Financial markets are inherently noisy, non-stationary systems in which stock prices are influenced by complex interactions between historical price dynamics and rapidly evolving external information (Giantsidi and Tarantola 2025). Despite decades of research, consistently forecasting market behaviour remains challenging, not only due to stochastic price movements but also due to the relationship between predictive signals and future returns varying across time and market regimes (Hammerschmid and Lohre 2018). As a result, even small modelling assumptions can significantly impact downstream decision-making.

Recent advances in deep learning have led to measurable improvements in stock movement prediction, particularly through the use of sequence models such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks, as well as the incorporation of unstructured textual data such as news and social media (Bustarviejo and Bousoño-Calzón 2026). However, much of the existing literature evaluates predictive success primarily through accuracy-based metrics, often without sufficient consideration of how model outputs are subsequently used in financial decision-making. In practice, predictive accuracy alone is insufficient (Gneiting et al. 2007; Bequé et al. 2017), investment decisions depend critically on the reliability of predicted probabilities and the ability to translate forecasts into appropriately sized positions under risk (Ziemba and MacLean 2011).

In practical investment settings, model outputs are rarely consumed as hard class labels, instead, they are interpreted as probabilities that inform position sizing and risk management. This places importance on both the directional accuracy and the reliability of the prediction. A well-calibrated model produces probability estimates whose empirical frequencies align with observed outcomes in practice, whilst poorly calibrated models generate unreliable probabilities that are treated as meaningful inputs to allocation rules leading to over- or under-sized positions, unstable trading behaviour and amplified drawdowns, even when headline accuracy metrics appear strong as established by Wright (2026).

Beyond calibration, a further limitation of many existing approaches is the implicit assumption that a model's predictive reliability is static. In reality, model performance fluctuates over time due to regime shifts, changes in market microstructure and evolving information flows. Models rarely account for their own historical errors when making new predictions and thus may continue to express high confidence even during periods of deteriorating performance. Incorporating information about prior predictions and their realised errors offers a principled way for a system to assess its own reliability and adapt its behaviour accordingly (Huang et al. 2026).

Finally, prediction and capital allocation are often treated as separate problems, with models evaluated independently of the investment strategies they enable. This separation obscures the true economic value of predictive improvements and limits the interpretability of reported results. Investment performance depends jointly on the quality of predictive signals, the reliability of associated probabilities and the manner in which capital is allocated under uncertainty (Yi and Hasuike 2026).

This project is thereby motivated by the need for an integrated framework that unifies multimodal prediction, model self-assessment, probability calibration and principled capital allocation within a single coherent system. By combining heterogeneous data sources with information derived from prior predictions and their realised errors, the framework seeks to improve both the reliability of forecasts and their practical utility under non-stationary market conditions. Through the joint evaluation of statistical performance and economic outcomes, this work aims to bridge the gap between predictive modelling and real-world investment decision-making, demonstrating how certainty-aware learning and risk-sensitive allocation can support more stable and economically meaningful trading strategies in dynamic environments.

1.2 Purpose

The purpose of this project is to design and evaluate an end-to-end, multi-horizon trading system that integrates multimodal predictive signals, probability calibration and risk-aware capital allocation within a single coherent pipeline. Rather than optimising predictive accuracy in isolation, the system is designed to assess the reliability and economic viability of model-driven investment decisions under realistic market conditions.

This work builds upon *Stock Movement Prediction from Tweets and Historical Prices* by Xu and Cohen (2018), which is used as a baseline for comparison. However, the focus of this project extends beyond next-day prediction and single-model evaluation by exploring variable forecasting horizons, multimodal feature integration and multiple predictive model formulations. Crucially, the value of these predictive signals is assessed through their interaction with capital allocation strategies in a simulated trading environment, rather than through benchmarking metrics alone. To account for non-stationarity and regime shifts, the proposed system incorporates prior predictions and their realised errors as meta-features, enabling the model to capture time-varying predictive reliability.

To this end, the dissertation evaluates multiple deep-learning architectures and systematically examines their integration with several capital allocation strategies, including fractional Kelly, mean-variance portfolio theory (MPT) and a hybrid fractional Kelly-MPT approach. By jointly analysing predictive performance, calibration quality and financial outcomes, this project aims to narrow the difference between statistical forecasting and practical investment decision-making.

1.2.1 Contributions

The following key contributions are made:

- **Stock direction and magnitude prediction formulations**, including binary classification for directional movement, regression for return magnitude and an ordinal classification approach that discretises returns into buckets to estimate expected return.
- **An end-to-end multimodal prediction and investment framework** that integrates historical price data, textual signals and auxiliary features within a single automated pipeline, designed for realistic walk-forward evaluation.
- **A variable-horizon trading system with an early-exit mechanism**, enabling positions to be held for multiple days while allowing premature exit when predicted downside risk increases, thereby improving flexibility relative to fixed-horizon strategies.
- **The explicit incorporation of model self-assessment through meta-features**, using prior predictions and their realised errors to capture time-varying predictive reliability in non-stationary financial markets.
- **A systematic application of probability calibration to deep-learning models**, enabling model confidence scores to be interpreted as meaningful probabilities suitable for downstream risk-sensitive decision-making.

- **A comparative evaluation of capital allocation strategies**, including fractional Kelly, mean-variance portfolio theory (MPT) and a hybrid fractional Kelly-MPT approach, assessed using economic performance measures.
- **An evaluation methodology grounded in economic outcomes**, assessing return and risk-adjusted performance alongside traditional accuracy-based predictive metrics.

1.2.2 Novelty

The novelty of this work lies not in proposing new model architectures, but in the integrated treatment of multimodal prediction, uncertainty estimation, model reliability and capital allocation. By embedding these components within a unified framework and evaluating them through economic outcomes, this dissertation advances existing stock prediction research towards a more trustworthy and practically deployable system.

1.3 Report Structure

This dissertation is organised into eight chapters.

Chapter 2, *Background and Related Work*, reviews prior research in stock movement prediction and outlines the key theoretical foundations of the project, including LSTM and GRU models, bidirectional variants, ordinal regression, BERT, probability calibration methods, the Kelly Criterion, MPT and the use of meta-features.

Chapter 3, *Analysis and Research Questions*, formalises the problem setting, defines the prediction tasks, discusses constraints such as temporal integrity and non-stationarity as well as presenting the research questions guiding the evaluation.

Chapter 4, *System Design*, describes the architecture of the end-to-end trading framework, including multimodal data integration, model formulations for direction and magnitude prediction, meta-feature incorporation, calibration strategy and the investment simulation design.

Chapter 5, *Implementation*, outlines the practical realisation of the system, covering data preprocessing, feature engineering, model training, calibration and capital allocation within a walk-forward simulation environment.

Chapter 6, *Results*, reports training & predictive performance and economic outcomes such as return and risk-adjusted measures, alongside comparative analyses of model, feature sets and allocation strategies.

Chapter 7, *Discussion*, interprets the findings in relation to the research questions and reflects on the interaction between multimodal prediction, model reliability, calibration and capital allocation.

Chapter 8, *Conclusion and Future Work*, summarises the main contributions, discusses limitations, and outlines directions for future research.

Additional material, including extended results and implementation details, is provided in the Appendices.

2 | Background & Related Work

2.1 Stock Movement Prediction

Stock movement prediction (SMP) aims to forecast future price direction or return magnitude using historical market data and auxiliary information. A fundamental challenge in this domain is the time-varying and regime-dependent nature of financial markets. Early empirical evidence by Schwert (1989) demonstrated that stock market volatility is persistent, clustered and strongly associated with macroeconomic uncertainty, recessions and financial crises. Crucially, volatility dynamics are not constant over time and no single macroeconomic factor fully explains their variation, establishing that financial markets are inherently non-stationary, with structural shifts in risk conditions occurring over time.

The implication for prediction models is clear in that the assumptions of stable statistical relationships are often violated in practice. Forecasting systems must therefore operate under changing volatility regimes and evolving market conditions, motivating adaptive and sequence-aware modelling approaches.

2.1.1 From Statistical Models to Deep Learning

Early approaches to SMP relied on classical statistical models such as ARIMA, which assume linear relationships and weak stationarity. While these models can capture short-term autocorrelation patterns, their performance is limited in the presence of non-linearity and structural breaks (Ho et al. 2021).

With the emergence of machine learning and deep learning, recurrent architectures such as LSTM and GRU networks became widely adopted for financial time-series modelling (Xiao et al. 2024; Touzani and Douzi 2021). These models are capable of capturing long-term dependencies and non-linear interactions within sequential data. Large-scale empirical evidence by Fischer and Krauss (2018) shows that LSTM-based models trained on S&P 500 constituents (1992–2015) outperform classical baselines such as ARIMA, Random Forests, and shallow neural networks when evaluation respects temporal ordering. However, performance was shown to decay post-2010, highlighting regime sensitivity and the fragility of learned patterns.

These findings suggest that while deep sequence models can extract predictive structure, their effectiveness depends heavily on market conditions and proper evaluation protocols.

2.1.2 Multimodal Extensions

Beyond price-only models, researchers have incorporated textual information such as news headlines and social media posts to capture sentiment and narrative signals not immediately reflected in historical prices. Xu and Cohen (2018) introduced *StockNet*, modelling market uncertainty using a recurrent variational autoencoder combined with an auxiliary temporal prediction head. Evaluated on Twitter and price data across 88 U.S. stocks (2014–2016), the model outperformed ARIMA, Random Forest, and text-only baselines. However, the use of dead-zone filtering, which excludes small-movement days, may have inflated reported accuracy by simplifying the classification task.

Building upon this direction, He and Gu (2022) proposed a Multimodal Attention Network (MMAN) that integrates credibility-aware textual features with price encoders through intra- and inter-modal attention mechanisms. Ablation studies demonstrate that both credibility modelling and cross-modal attention contribute positively to performance, suggesting that structured multimodal fusion can enhance predictive capability. In parallel, robustness to market noise has been explored through adversarial training. Feng et al. (2019) introduced an adversarially trained LSTM (Adv-LSTM), incorporating worst-case perturbations during training to improve generalisation under stochastic conditions. Reported improvements over prior baselines indicate enhanced resilience to noisy inputs.

More recent work has investigated multi-scale modelling and extended temporal windows, demonstrating that incorporating longer historical context can further improve statistical performance (Hiruta and Senoguchi 2025). Nevertheless, across these multimodal approaches, evaluation remains predominantly focused on classification metrics such as accuracy and MCC. Label filtering and the absence of cost-aware economic assessment are recurring limitations, leaving open the question of whether statistical improvements translate into meaningful financial performance.

2.2 Deep Learning for Time-Series Modelling

Early recurrent neural networks (RNNs) were theoretically well-suited for sequence modelling but struggled in practice due to the vanishing and exploding gradient problem. As demonstrated by Hochreiter and Schmidhuber (1997), gradients propagated through time either decay or grow exponentially as sequence length increases, preventing effective learning of long-term dependencies. This limitation makes standard RNNs unreliable for tasks requiring memory over extended time horizons.

To address this, Hochreiter and Schmidhuber (1997) introduced the Long Short-Term Memory (LSTM) architecture. LSTMs incorporate gated memory cells designed to preserve stable gradient flow across long temporal spans. By regulating information flow through input, forget and output gates, LSTMs enable controlled memory retention and selective forgetting. This architecture allows the network to learn dependencies spanning hundreds or even thousands of time steps, overcoming the instability inherent in traditional RNN training.

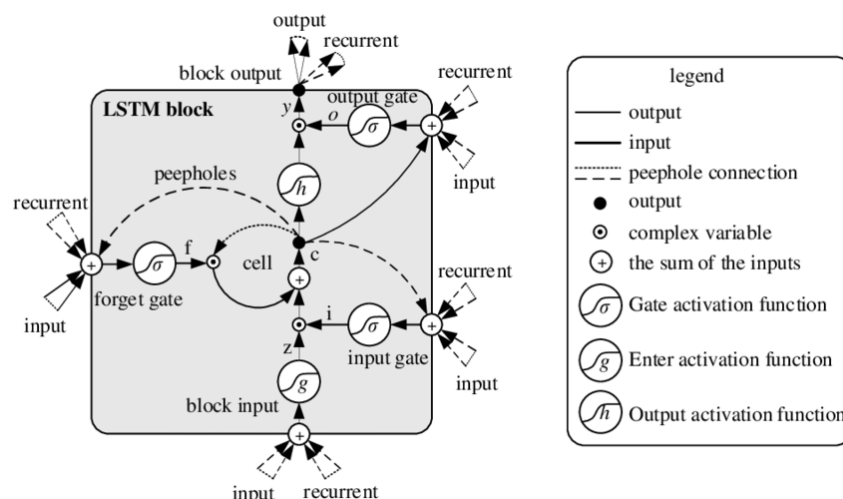


Figure 2.1: Schematic diagram of LSTM model (Shuangao et al. 2020).

2.2.1 Long Short-Term Memory (LSTM)

Subsequent analyses, including Staudemeyer and Morris (2019), clarify that the strength of LSTMs lies in their gated structure and explicit memory cell. The separation between hidden state and cell state permits stable error propagation while protecting memory from irrelevant noise. This design enables LSTMs to model long-term temporal dependencies more reliably than earlier recurrent architectures.

In time-series domains such as finance, where regime persistence and volatility clustering occur, the ability to retain information over extended periods is particularly relevant. However, LSTMs are not without limitations. Model capacity is finite, architectural design choices significantly influence performance, and increasing network size does not automatically guarantee improved long-term reasoning.

2.2.2 Gated Recurrent Units (GRU)

The Gated Recurrent Unit (GRU) was introduced by Cho et al. (2014) and described by Chung et al. (2016) as a simplified variant of the LSTM, merging the input and forget gates into a single update gate and removing the explicit memory cell. This reduces architectural complexity and parameter count while retaining gating mechanisms that regulate temporal information flow.

Empirical comparisons, such as those conducted by Cahuantzi et al. (2023), suggest that GRUs often achieve performance comparable to LSTMs on moderately complex sequence tasks while offering improved computational efficiency. However, for highly complex or long-range dependency structures, LSTMs may demonstrate superior stability and representational capacity.

In financial time-series modelling, this trade-off implies that GRUs may suffice for short-term pattern extraction, whereas LSTMs may better capture longer-term regime persistence and volatility dynamics. Consequently, both architectures remain widely adopted in stock movement prediction research.

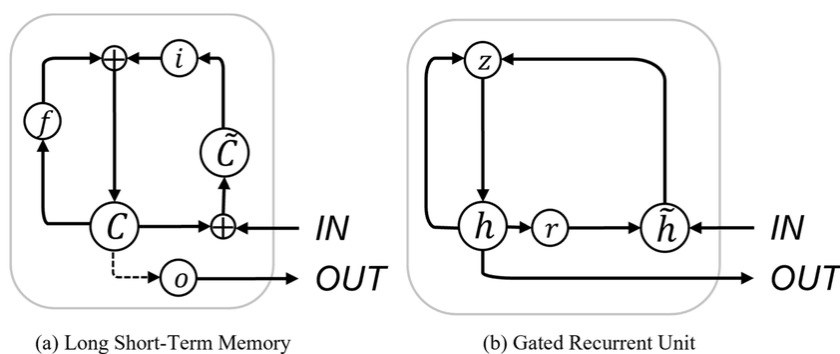


Figure 2.2: Diagram of LSTM and GRU structure. (a) The structure diagram of a vanilla LSTM architecture. “ C ” represents the memory cell, “ i ” denotes the input gate, “ f ” represents the forget gate, and “ o ” denotes the output gate; (b) The simple architecture of GRU. “ h ” is the activation function, “ z ” refers to the update gate, and “ r ” stands for the reset gate. (Liu et al. 2022).

2.2.3 Bidirectional Variants

Bidirectional recurrent neural networks (BRNNs) extend standard RNN architectures by processing sequences in both forward and backward directions (Schuster and Paliwal 1997). By combining hidden representations from past-to-future and future-to-past passes, bidirectional models leverage full-sequence context for each time step.

Bidirectional architectures have demonstrated improved performance in tasks such as speech recognition and sequence classification (Graves and Schmidhuber (2005), Huang et al. (2015)), and they form the basis of widely used models such as BiLSTM and BiGRU.

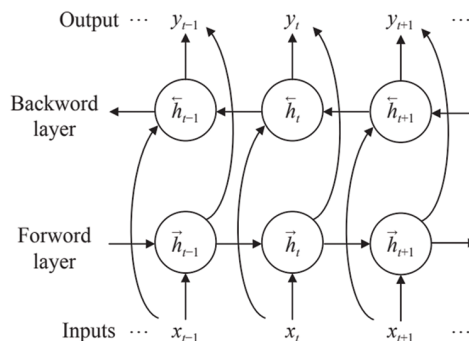


Figure 2.3: Bidirectional RNN model diagram (Wenqiu et al. 2023).

2.2.4 Implications for Stock Movement Prediction

The progression from classical RNNs to gated recurrent architectures and their bidirectional extensions reflects a broader shift toward models capable of handling long-term dependencies, non-linearity and temporal complexity. In financial markets, characterised by regime shifts, volatility clustering and evolving dynamics, such properties are particularly valuable.

2.2.5 Ordinal Regression for Financial Returns

Financial returns are inherently ordered quantities rather than purely categorical variables. Binary classification (up/down) ignores magnitude information, while regression directly models continuous returns but may be sensitive to noise. Ordinal regression provides an intermediate formulation by discretising returns into ordered buckets, preserving directional and magnitude structure while improving robustness (Wang et al. 2025). This formulation enables the estimation of expected return ranges rather than simple direction, offering more informative signals for downstream capital allocation according to the Bloomberg white paper by Wu (2020), *Weather transformer for stock return classification with model uncertainty estimation*. Formulated in the following way:

$$P(Y \leq c \mid x) = F(\theta_c - \eta(x)), \quad c = 1, \dots, K - 1,$$

where θ_c are ordered thresholds and $\eta(x)$ is the model score induced by the input features. This allows the model to exploit the natural ordering of return ranges and, in turn, supports the estimation of expected return ranges rather than simple direction alone (Kook et al. 2021).

2.3 BERT for Natural Language Processing

The integration of textual information into stock prediction models has attracted increasing attention, as market prices often reflect narratives and sentiment not immediately captured by historical returns alone. Xu and Cohen (2018) demonstrated that incorporating Twitter data alongside price time-series improves predictive performance relative to price-only baselines. Early approaches typically relied on static word embeddings (e.g., Word2Vec or GloVe) combined with recurrent or convolutional encoders to extract textual features. For example, He and Gu (2022) encoded textual posts using neural attention mechanisms alongside price features, while Li et al. (2020) model textual relationships through graph-based structures to capture inter-stock dependencies.

These methods, however, depend on context-independent embeddings, where each word is represented by a fixed vector regardless of surrounding context. Transformer-based language models, particularly BERT (Devlin et al. 2019), address this limitation by generating contextualised embeddings in which a word's representation depends on its sentence-level context. BERT employs a bidirectional self-attention mechanism, enabling it to capture long-range linguistic dependencies and nuanced semantic relationships more effectively than earlier recurrent encoders.

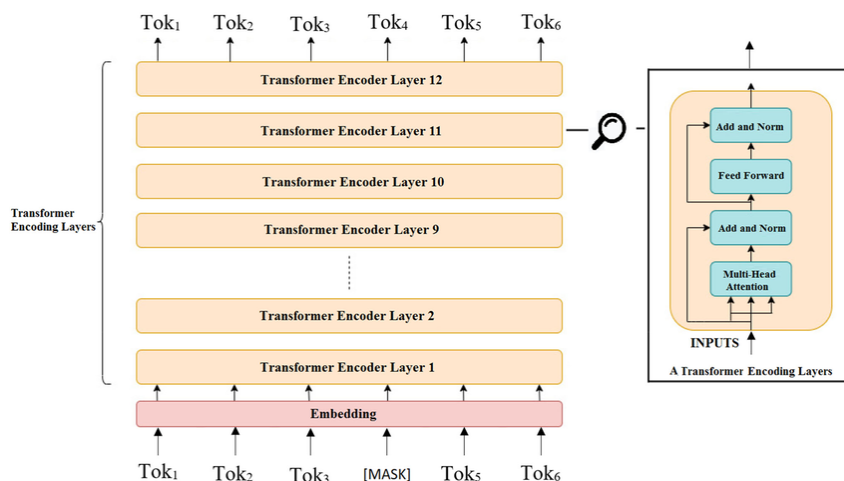


Figure 2.4: BERT model architecture (Noorian et al. 2023).

Domain-specific adaptations such as FinBERT by Araci (2019), further improve performance in financial applications fine-tuning the model on financial corpora, enhancing its ability to interpret domain-specific terminology, sentiment, and stance.

Despite these advances, Stan et al. (2026) finds that textual signals in financial markets remain noisy, sparse and temporally irregular. Careful alignment between text timestamps and market data is therefore essential to prevent information leakage. Furthermore, improvements in textual representation quality do not automatically translate into economic gains, reinforcing the need for integrated evaluation within realistic trading frameworks.

2.4 Probability Calibration

In classification models, outputs are often constrained to lie between zero and one, typically through sigmoid or softmax activations, and are therefore interpreted as probabilities. However, these values represent model confidence rather than true probabilistic quantities in the formal sense of Kolmogorov's axioms (von Plato 2005), as demonstrated by Guo et al. (2017). A neural network does not estimate the objective probability of a stock moving up or down, rather, it produces a score reflecting patterns learned from the training data and encoded in its parameters.

Even under ideal conditions—clean data, large sample sizes and well-optimised architectures—it is unrealistic to expect model outputs to coincide exactly with empirical outcome frequencies. Deep neural networks are known to exhibit systematic overconfidence, particularly when trained to minimise cross-entropy loss (Wei et al. 2022). Consistent with this, Guo et al. (2017) show that modern neural networks are often poorly calibrated, so a predicted probability of 0.7 does not necessarily imply that the event occurs approximately 70% of the time.

This distinction is particularly consequential in financial applications. Capital allocation strategies such as the Kelly Criterion and MPT explicitly require probability estimates to determine optimal position sizing. If predicted probabilities are miscalibrated, allocation decisions can systematically

overestimate or underestimate risk, leading to excessive leverage and unstable trading behaviour (Wright 2026).

A model is therefore said to be well-calibrated when its predicted probabilities align with observed empirical frequencies. Probability calibration methods aim to correct systematic distortions in model confidence, enabling outputs to be interpreted more reliably for downstream risk-sensitive decision-making.

2.4.1 Isotonic Regression

Isotonic regression is a non-parametric post-hoc calibration technique that fits a monotonic transformation to model outputs in order to align predicted confidence scores with empirical outcome frequencies. Originally proposed for probability estimation in supervised learning by Zadrozny and Elkan (2002), isotonic regression relaxes parametric assumptions and instead learns a piecewise-constant monotonic mapping using the pool-adjacent-violators (PAV) algorithm.

This flexibility allows isotonic regression to correct systematic distortions in model confidence while preserving ranking. Empirical studies demonstrate that isotonic calibration can significantly improve probability estimates for binary and multiclass classification tasks (Zadrozny and Elkan 2001; Niculescu-Mizil and Caruana 2005). However, because it is non-parametric and highly flexible, isotonic regression may overfit when calibration datasets are small, leading to unstable probability estimates in limited-data regimes (Guo et al. 2017).

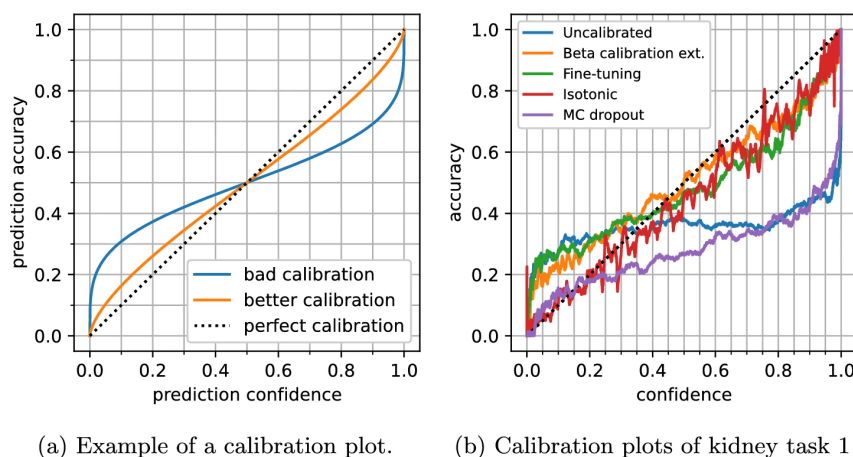


Figure 2.5: Calibration plots of an example (a) and of the results on the kidney task (b). As the plot moves closer to the diagonal, the predicted confidences become a better reflection of the accuracy, and the model is better calibrated (Rousseau et al. 2025).

2.4.2 Temperature Scaling

Temperature scaling is a parametric post-hoc calibration method introduced for modern deep neural networks by Guo et al. (2017). The method rescales the logits of a trained classifier using a single scalar temperature parameter $T > 0$, learned on a held-out validation set. The calibrated probabilities are obtained by applying the softmax function to the rescaled logits (Figure 2.6).

Unlike non-parametric methods such as isotonic regression, temperature scaling preserves the relative ranking of predictions while correcting systematic overconfidence. Empirical evidence demonstrates that deep neural networks trained with cross-entropy loss are often miscalibrated, typically exhibiting overconfident predictions and that temperature scaling substantially reduces calibration error without affecting classification accuracy Guo et al. (2017).

Because it introduces only a single additional parameter, temperature scaling is less prone to overfitting than flexible non-parametric approaches and performs reliably even with limited calibration data. As a result, it has become a standard baseline for post-hoc probability calibration in deep learning applications.

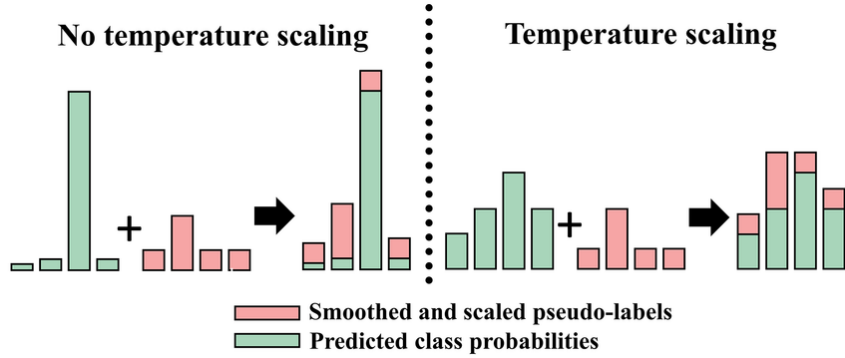


Figure 2.6: Effect of temperature scaling on ordinal pseudo-label smoothing. Without temperature scaling, predicted class probabilities remain sharp and poorly aligned with the smoothed pseudo-label distribution. Applying temperature scaling softens the predicted probabilities, improving their alignment with the target distribution and yielding better-calibrated outputs (Maracani et al. 2024).

2.5 Meta-Features and Model Self-Assessment

Pesaran and Timmermann (2002)’s survey identifies that financial time-series exhibits high volatility, nonlinear dependencies and regime changes associated with macroeconomic shocks, crises and policy transitions. These characteristics imply that predictive relationships are unstable over time and that model performance may deteriorate as market conditions change.

Despite this, many predictive systems implicitly assume static reliability, treating model outputs as equally trustworthy across all market regimes. Prior forecasting errors are rarely incorporated into subsequent predictions, despite containing information about recent model performance and potential regime shifts.

Pesaran and Timmermann (2002) go on to discuss the value of hybrid approaches in financial forecasting, combining complementary techniques such as evolutionary algorithms and neural networks to enhance predictive robustness in complex and potentially non-stationary market environments. Similarly, Varshney and Sharma (2024) show that stacked deep learning architectures can achieve consistently lower out-of-sample forecasting errors, reporting reductions in MSE and MAE across heterogeneous time-series datasets, suggesting a potential pathway towards improved predictive stability under evolving data conditions.

Incorporating prior predictions and realised errors as meta-features extends this adaptive perspective by modelling time-varying predictive reliability directly within the forecasting pipeline. Rather than assuming constant model confidence, this allows the system to adjust implicitly to recent performance degradation and changing market conditions. The idea is closely related to recent work on residual-guided forecasting by Huang et al. (2026) and adaptive forecast combination, where historical errors are treated as informative signals for updating future predictions or blending model outputs over time by Luo et al. (2025). However, despite growing interest in these ideas in the broader forecasting literature, their integration into deep learning-based financial trading systems remains limited.

2.6 Capital Allocation Strategies

Prediction alone does not determine profitability. Position sizing plays a crucial role in investment performance

2.6.1 The Kelly Criterion

Introduced by Kelly (1956), the Kelly Criterion derives the optimal fraction of capital to allocate to a favourable bet in order to maximise long-term logarithmic wealth growth (Kelly Jr. 1956). The original formulation arose from information theory and the analysis of signal transmission over noisy communication channels. It was later adapted to gambling and financial markets, where it provides a principled framework for capital allocation under probabilistic uncertainty (Breiman 1962; Thorp 2008; MacLean et al. 2011).

For a binary outcome with success probability p and net odds b , the optimal Kelly fraction is given by

$$f^* = \frac{bp - (1 - p)}{b},$$

where:

- f^* denotes the optimal fraction of available capital to allocate to the risky position;
- p represents the estimated probability of a favourable outcome;
- $1 - p$ is the probability of an unfavourable outcome;
- b denotes the net payoff ratio (i.e., the profit per unit stake if the outcome is favourable).

obtained by maximising expected logarithmic utility. In financial applications, this formulation links predicted probabilities directly to position sizing, making the reliability of probability estimates critical for stable portfolio growth.

2.6.2 Mean-Variance Portfolio Theory (MPT)

Introduced by Markowitz (1952), MPT frames construction as an optimisation problem balancing expected return against risk (Markowitz 1952).

Formally, the mean-variance optimisation problem can be written as:

$$\max_{\mathbf{w}} \quad \mathbf{w}^\top \boldsymbol{\mu} - \lambda \mathbf{w}^\top \Sigma \mathbf{w},$$

where:

- $\mathbf{w}$ denotes portfolio weights;
- $\boldsymbol{\mu}$ expected returns;
- Σ the covariance matrix;
- λ a risk-aversion parameter.

MPT emphasises diversification and covariance structure across assets, stated by Markowitz (1959), contrasting with single-asset Kelly Sizing. While Kelly maximises growth under independent bets, MPT explicitly accounts for cross-asset risk interactions.

The integration of probability-based sizing with diversification principles motivates hybrid allocation approaches (Peterson 2018).

3 | Analysis and Research Questions

The literature on stock movement prediction reveals several consistent findings:

- Deep sequence models frequently outperform classical statistical baselines when evaluated using temporally ordered validation (Section 2.1.1).
- Incorporating textual information alongside price-based features can provide incremental predictive gains (Section 2.3).
- Architectural and training enhancements, such as adversarial learning and multi-scale modelling, improve statistical performance (Sections 2.1.2 & 2.5).

Despite these advances, important limitations remain:

- Evaluation is typically restricted to accuracy, correlation or classification-based metrics, with limited consideration of economic outcomes.
- Filtering strategies, such as threshold-based label exclusion used in StockNet by Xu and Cohen (2018), are often applied, simplifying the prediction task and potentially inflating reported performance.
- Model reliability is generally assumed to be static, despite well-documented regime shifts and volatility clustering in financial markets.
- Methodological heterogeneity across studies makes cross-paper comparisons and ablation analysis difficult.

These observations motivate a more integrated formulation of the stock prediction problem that jointly considers predictive accuracy, probabilistic reliability and capital allocation under realistic market constraints.

3.1 Problem Definition

The objective of this dissertation is to develop and evaluate an end-to-end probabilistic stock movement prediction and capital allocation framework, assessed under realistic walk-forward market conditions.

3.1.1 Prediction Task

The predictive component consists of two primary objectives.

1. **Directional Probability Estimation:** Estimate the probability that a given asset will experience a positive return over a fixed forecast horizon H .
2. **Directional Magnitude Estimation:** Model return magnitude using ordinal multi-class classification rather than continuous regression. Returns are discretised into ordered buckets, enabling estimation of expected return categories while preserving directional structure.

Formally, for asset i at time t , the model estimates:

$$\hat{p}_{i,t} = \mathbb{P}(R_{i,t+H} > 0 \mid \mathcal{F}_t),$$

where $R_{i,t+H}$ denotes the return over horizon H and $\mathcal{F}_t$ represents the information set available at time t .

3.1.2 Decision Layer

Predicted probabilities are not the final objective. Instead, they are transformed into investment decisions through two sequential components:

1. **Probability Calibration:** Post-hoc calibration methods, including isotonic regression and temperature scaling, are applied to improve probabilistic reliability.
2. **Capital Allocation:** Calibrated probabilities are converted into position sizes using alternative allocation strategies:
 - Fractional Kelly
 - Mean–Variance Portfolio Theory (MPT)
 - Hybrid fractional Kelly–MPT allocation

Early Exit Mechanism. For positions held over multi-day horizons ($H > 1$), the framework includes a dynamic early-exit rule: if the next-day predicted probability indicates a sufficiently high likelihood of negative return, the position is liquidated prior to maturity. This mechanism allows adaptive horizon adjustment and mitigates adverse drift under changing market conditions.

This structure enables the joint evaluation of predictive accuracy, probabilistic reliability and economic performance.

3.2 Constraints and Assumptions

The proposed framework operates under realistic financial and statistical constraints, reflecting both empirical asset pricing practice and practical portfolio management considerations.

3.2.1 Temporal Integrity and Information Structure

Strict temporal integrity is enforced throughout the pipeline to eliminate lookahead bias. Targets are defined using forward returns, while all features are constructed using information available at or prior to time t . Model training, validation and testing follow a walk-forward (expanding window) procedure to ensure that parameter estimation relies exclusively on historical data (Micallef et al. 2025).

This design mirrors the predictive regression framework of Fama and French (1988), where returns over $(t, t + T)$ are regressed on state variables observed at time t , thereby preserving causal ordering and economic validity.

3.2.2 Market Non-Stationarity

Financial markets are inherently non-stationary. The data-generating process for returns, volatility and cross-asset correlations evolves across macroeconomic regimes. Consequently:

- Model reliability is time-varying,
- Calibration may degrade across structural breaks,
- Estimated probabilities may drift out of distribution.

The allocation layer must therefore remain robust to prediction instability and regime shifts rather than assuming parameter constancy.

3.2.3 Capital, Risk and Leverage Constraints

Capital is finite and position sizes must satisfy feasibility and risk constraints. In particular:

- Leverage limits restrict exposure,
- Portfolio weights must satisfy budget constraints,

- Estimation error can induce over-allocation.

Both Kelly sizing and Mean-Variance (MPT) optimisation are highly sensitive to input estimation error. Overconfident probability estimates can lead to excessive leverage and poor risk-adjusted returns. As demonstrated in the literature on backtest overfitting and estimation error amplification (Bailey et al. 2013; de Prado 2018), robust capital allocation requires shrinkage, fractional Kelly scaling or risk caps to mitigate tail risk.

3.3 Research Questions

This project aims to answer the following research questions:

RQ1. Predictive Performance

Does incorporating textual information alongside price-based features improve directional and magnitude prediction performance under strict temporal validation across binary classification, ordinal multi-class classification and regression-based return estimation?

RQ2. Meta-Features and Reliability Modelling

Does incorporating previous prediction outcomes and realised error statistics as meta-features improve model stability, robustness and resistance to performance degradation under non-stationary market conditions?

RQ3. Allocation Strategy Comparison

How do different capital allocation strategies, including fractional Kelly, Mean-Variance Portfolio Theory (MPT) and hybrid fractional Kelly-MPT allocation, compare in terms of economic performance when applied to identical predictive signals?

RQ4. Interaction Effects

How do probability calibration and capital allocation method interact to influence long-term capital growth, volatility and overall system-level performance?

4 | System Design

This chapter describes the conceptual architecture of the proposed framework. The system is designed as an end-to-end probabilistic decision pipeline, transforming raw market and textual information into calibrated investment allocations evaluated under walk-forward simulation. The focus of this chapter is the structural design of the framework rather than implementation-specific details.

4.1 Overall Architecture

The proposed system follows a structured, layered architecture that transforms raw financial and textual data into capital allocation decisions through a strictly sequential pipeline.

The architecture consists of the following high-level components:

1. **Data Ingestion and Preprocessing.** Stock price data and associated textual data (e.g., social media posts) are collected and cleaned and joined.
2. **Feature Engineering.** Tweets are passed through transformer-based language models to extract contextual representations and are then aggregated for each ticker. In parallel, technical indicators are computed from historical price series. Sector-level indicators are also incorporated. These feature sets are merged to form a temporally aligned initial master dataset.
3. **Initial Predictive Benchmarking.** The constructed dataset is used to benchmark predictive performance across multiple model families. At this stage, models are primarily evaluated using their raw outputs in order to assess statistical predictive capability independently of downstream decision mechanisms. Probability calibration is therefore not applied to the binary classification, regression or ordinal multi-class benchmarks.
4. **Meta-Feature Construction.** A second-stage dataset is constructed by augmenting the master dataset with meta-features derived from historical predictions and realised errors. These features capture time-varying predictive reliability in a leak-safe manner.
5. **Enhanced Predictive Benchmarking.** Models are re-evaluated using the augmented dataset to assess whether reliability-aware signals improve predictive stability and accuracy.
6. **Investment Simulation and Allocation.** Within the simulation layer, models are trained on a per-asset basis under a walk-forward regime. Predicted probabilities are calibrated before being mapped to position sizes using selected capital allocation strategies. Portfolio performance is then evaluated under realistic trading assumptions.

The framework supports multiple predictive formulations in parallel, including binary classification, ordinal multi-class classification and regression-based return estimation. These model outputs are integrated through blending mechanisms, whereby higher-level predictors learn from base-model outputs and their historical performance characteristics.

The overall architecture therefore constitutes a probabilistic decision system rather than a standalone predictive model, shown in Figure 4.1.

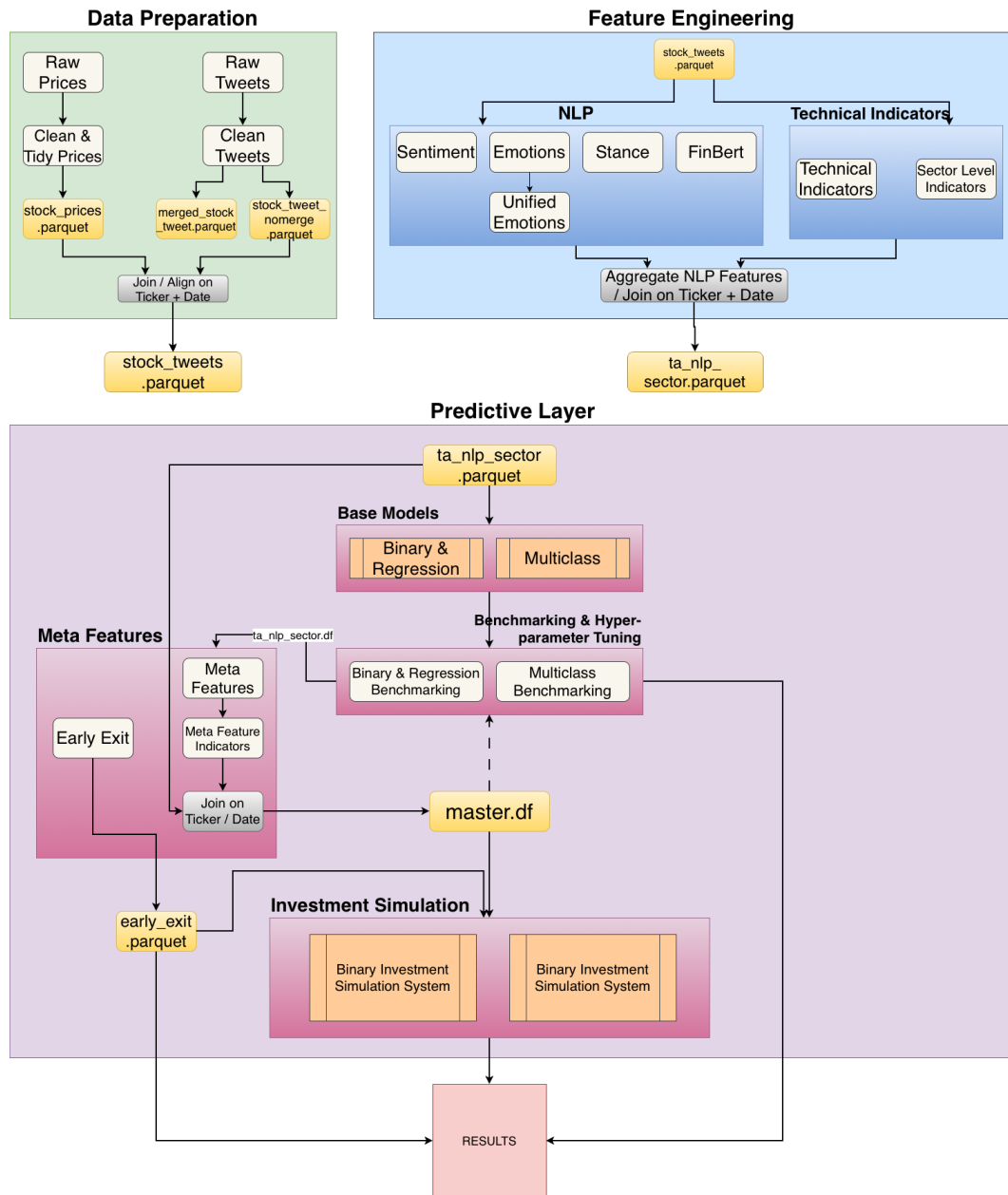


Figure 4.1: End-to-end architecture of the proposed prediction and investment framework.

4.2 Data Flow and Temporal Integrity

Maintaining temporal integrity is central to the system design. All components operate under strict chronological constraints to prevent look-ahead bias.

At time t , the information set $\mathcal{F}_t$ includes:

- Historical price data up to time t ,
- Derived technical indicators,
- Textual features extracted from documents published no later than time t ,
- Historical model predictions and lagged error-based reliability signals computed strictly from data available before time t .

Model training and evaluation follow a walk-forward structure (Micallef et al. 2025). Parameters are estimated on historical windows and applied to future unseen data without reusing future information. Meta-features are constructed in a leak-safe manner using lagged reliability statistics.

This design ensures that all reported predictive and economic results reflect realistic deployment conditions.

4.3 Model Families and Outputs

The framework supports three predictive formulations.

Binary models estimate the probability of a positive return over a variable forecast horizon H . The output is a probabilistic estimate of directional movement.

Ordinal classification models discretises returns into ordered magnitude buckets. This design preserves directional information while allowing estimation of expected return categories. The ordered structure enables modelling of both direction and magnitude without assuming linear return dynamics.

Regression models directly estimate expected return values. For evaluation consistency, regression outputs can be transformed into directional signals by considering the sign of the predicted return.

These three model families allow systematic comparison between probabilistic, ordinal, and continuous return formulations under identical evaluation conditions.

4.4 Calibration and Meta-Learning Design

Raw predictive probabilities are not assumed to represent well-calibrated estimates of true event likelihoods. Calibration is introduced within the investment simulation layer, where predicted probabilities directly influence capital allocation decisions.

Post-Hoc Calibration in Simulation

During simulation, model outputs are transformed using either isotonic regression or temperature scaling prior to capital allocation. This ensures that the probability magnitudes used for staking reflect more reliable confidence estimates.

Importantly, calibration is applied only after walk-forward prediction generation. Post-hoc calibration is fitted only on validation data drawn from the same historical training window. The window is then advanced forward through time and the process is repeated. This preserves temporal integrity and prevents lookahead bias.

Meta-Learning and Reliability Modelling

The framework incorporates model self-assessment through the construction of reliability-aware meta-features. These features are derived from historical model predictions and realised outcomes under a walk-forward regime. Here, walk-forward refers to a chronological training procedure in which, at each simulation step, the model is fitted using only historical data available before the prediction date, at decision time t , models predict using information available up to $t - 1$ (Zhang and Zhang 2023).

Blending and Reliability Integration

These reliability signals are incorporated into extended datasets that augment base predictive features. In this design, blending mechanisms allow secondary models to learn how base-model outputs behave over time, effectively modelling time-varying predictive quality.

Rather than assuming static model reliability, the system explicitly accounts for performance drift and regime-dependent volatility. This layered design separates predictive modelling and reliability modelling, enabling controlled evaluation of their economic effects.

4.5 Benchmarking and Investment Simulation Design

The framework separates predictive benchmarking from full investment simulation.

Predictive Benchmarking Layer

The benchmarking layer is designed for efficient model comparison and hyperparameter tuning. Models are assessed on statistical predictive metrics without probability calibration or capital allocation. This stage serves two primary purposes:

- Rapid comparison of model families (binary, regression, ordinal multi-class),
- Systematic hyperparameter tuning and feature selection.

Raw model outputs are evaluated directly to measure predictive quality in isolation. By excluding calibration and allocation mechanisms at this stage, the framework ensures that statistical performance is not conflated with downstream economic effects.

Investment Simulation Layer

The investment simulation layer extends the predictive outputs into capital allocation decisions under realistic trading constraints.

At each decision point, model predictions have post-hoc calibration applied to improve probabilistic reliability.

Calibrated probabilities are then mapped to capital allocations according to a selected staking rule. The system supports: growth-optimal allocation (Fractional Kelly sizing), mean-variance portfolio optimisation, hybrid allocation combining growth and diversification principles.

For multi-day positions, an adaptive early-exit mechanism allows dynamic liquidation if subsequent predictions indicate deteriorating conditions.

Portfolio value evolves multiplicatively through time, incorporating transaction costs and rebalancing effects. Performance is continuously tracked using growth and risk metrics.

By holding predictive inputs constant while varying calibration and allocation logic, the simulation layer isolates the economic contribution of probability reliability and capital allocation design.

5 | Implementation

This chapter describes the practical implementation of the proposed framework. While the previous chapter outlined the conceptual architecture of the system, this chapter focuses on the technical realisation of each component, including data preparation, feature construction, model training procedures and the investment simulation environment.

5.1 Data Acquisition and Pre-processing

The dataset used in this project is comparative to the StockNet benchmark introduced by Xu and Cohen (2018). It spans the period from 1 January 2014 to 1 January 2016 and contains 88 U.S. stocks across nine sectors.¹ The selection focuses on highly capitalised firms, providing both sectoral diversity and broad market coverage.

Price data consists of daily OHLCV variables obtained from Yahoo Finance ², while textual data consists of time-stamped Twitter posts associated with individual tickers. Pre-processing includes tweet cleaning and deduplication, ticker/date alignment, chronological sorting and forward target construction. The resulting data sources are merged into a unified time-indexed master dataset.

5.2 Feature Engineering

Feature engineering combines structured market signals, tweet-derived signals and reliability-aware meta-features. Appendix A.1 contains the full details on the master dataset columns.

5.2.1 Technical Indicators

Technical indicators are generated at both the stock and sector levels in order to capture trend, momentum, volatility, and volume dynamics. This allows the model to learn from both asset-specific behaviour and broader sector structure (Lin and Lobo Marques 2024; Sinha et al. 2022).

At the stock level, the implementation computes exponential moving averages (EMA), MACD, RSI, Stochastic RSI, ATR, Bollinger Bands, OBV, and rolling returns (Appendix A.2.1).

To incorporate sector context, average OHLCV values are first computed for each sector-date pair, and the same technical analysis pipeline is then applied to these aggregated sector series before merging them back to ticker-level rows.

```
for col in ['open', 'high', 'low', 'close', 'volume']:
    ta_nlp[f'sector_{col}_mean'] = (ta_nlp.groupby(['date',
                                                    'sector']))[col].transform('mean')
    )
```

¹<https://github.com/yumoxu/stocknet-dataset.git>

²<https://uk.finance.yahoo.com>

Listing 5.1: Construction of sector-average features.

5.2.2 Textual Features

Textual information is converted into structured daily signals through a staged NLP pipeline (Appendix A.2.1). Signals are first computed at the tweet level and then aggregated by ticker and date before merging with stock and technical features.

Four main classes of textual features are extracted:

- **General sentiment**, using a BERT sentiment model (Devlin et al. 2019);
- **Emotion scores**, using an emotion classification transformer (Hartmann 2022);
- **Financial stance**, using a finance-tuned RoBERTa model³ (Saha et al. 2024);
- **FinBERT sentiment**, producing positive, negative, and neutral financial sentiment probabilities (Araci 2019).

After tweet-level inference, the resulting signals are aggregated at the daily ticker level using summary statistics such as mean scores and class proportions. Missing tweet-derived features on non-tweet days are filled with neutral defaults so that all trading days remain represented.

5.2.3 Meta-Features

The framework then incorporates meta-features derived from prior model predictions. These are intended to capture time-varying predictive reliability under non-stationary market conditions.

Meta-features are generated using a walk-forward out-of-sample procedure with periodic refitting. For each ticker, a model is trained on an initial historical window, used to predict a future block, then refit on an expanded window before the next block is predicted. This ensures that all meta-features are based on genuine historical forecasts rather than in-sample fits (Wright 2026; Souza 2026).

```
k = 0
while k < len(valid_idx):
    train_end = valid_idx[k]
    pred_start = train_end + 1
    pred_end = min(pred_start + refit_interval - 1, valid_idx[-1])

    train_df = df_tkr.iloc[:train_end + 1].copy()
    pred_df = df_tkr.iloc[pred_start:pred_end + 1].copy()

    scaler = StandardScaler()
    X_train = scaler.fit_transform(train_df[feature_cols])
    X_pred = scaler.transform(pred_df[feature_cols])

    model.fit(X_train, y_train)
    preds = model.predict(X_pred)

    k += refit_interval
```

Listing 5.2: Walk-forward out-of-sample meta-feature generation.

³<https://huggingface.co/zhayunduo/roberta-base-stocktwits-finetuned>

Two types of meta-features are constructed from our two base models (LSTM & GRU) and their bi-directional variants (Appendix A.2.1). As discussed in Section 4.4, these features are intended to expose recent model behaviour directly to the downstream predictor so that it can learn when prior forecasts are becoming more or less reliable.

- **Regression-based**, including predicted returns, absolute errors, squared errors, and directional correctness (Blaskowitz and Herwartz 2014; Huang et al. 2026). These features summarise recent point-forecast behaviour and error magnitude, allowing the model to detect persistent bias, unstable magnitude prediction, or deterioration in directional performance.
- **Classification-based**, including predicted up-probabilities, Brier scores, log-loss, and correctness indicators (Gneiting et al. 2007; Dimitriadis et al. 2024). These features capture recent probabilistic behaviour, including both forecast confidence and realised classification quality, and therefore provide a compact summary of recent calibration and predictive reliability.

To prevent leakage, all rolling reliability statistics are computed only after applying a one-step shift. Consequently, meta-features available at time t depend only on prediction outcomes observed up to $t - 1$.

```
df["regression_mae_20"] = (
    df["regression_abs_err_ret_1d"].shift(1).rolling(20).mean()
)
df["brier_20"] = df["brier"].shift(1).rolling(20).mean()
df["logloss_20"] = df["logloss"].shift(1).rolling(20).mean()
df["acc_20"] = df["correct"].shift(1).rolling(20).mean()
```

Listing 5.3: Leak-safe rolling reliability features.

5.3 Target Construction and Horizon Design

Targets are constructed for the three predictive formulations. In addition, variable-horizon targets are generated so that positions can be evaluated over different holding periods H .

5.3.1 Regression and Binary Targets

For a given asset at time t , the H -day forward return is defined as

$$r_t^{(H)} = \frac{P_{t+H}}{P_t} - 1,$$

where P_t denotes the closing price at time t . The corresponding binary direction target is

$$y_t^{(H)} = \begin{cases} 1 & \text{if } r_t^{(H)} > 0 \\ 0 & \text{otherwise.} \end{cases}$$

Function for the construction of targets found at Appendix A.2.2

5.3.2 Ordinal Multi-Class Targets

For the ordinal formulation, returns are discretised into 6 ordered buckets.

```
BUCKET_BINS = [-np.inf, -0.02, -0.005, 0.0, 0.005, 0.02, np.inf]

def bucketize_with_edges(x, edges):
    return np.clip(
        np.digitize(x, edges[1:-1], right=True),
        0,
        len(edges)-2
    )
```

Listing 5.4: Ordinal return bucket construction.

5.3.3 Sequence Horizon Design

Models operate on rolling feature windows of length L :

$$X_t = [x_{t-L+1}, \dots, x_t],$$

paired with the future target $r_t^{(H)}$. Training sequences are created by sliding this window across contiguous historical observations.

```
for i in range(seq_len-1, usable_end+1):
    X.append(data[i-seq_len+1:i+1])
    y.append(targets[i+H])
```

Listing 5.5: Sequence construction for horizon prediction.

5.4 Model Selection, Training and Calibration

5.4.1 Hyperparameter Tuning and Feature Selection

Hyperparameter tuning and feature selection are performed using Optuna (Akiba et al. 2019). This is particularly suitable for the project because it supports efficient black-box optimisation while allowing search spaces to be defined dynamically.

Feature sets are organised hierarchically into three groups:

- **Base:** price-based and technical features together with meta-features and their associated error statistics;
- **NLP:** all Base features augmented with textual signals such as sentiment, emotion, stance and FinBERT sentiment;
- **Sector:** all NLP features further augmented with sector-level indicators.

For binary and ordinal classification, the tuning objective is to maximise MCC. For regression, the objective is to minimise MAE. The search space includes model family, feature set, sequence length, hidden dimensions, dropout, batch size, learning rate, weight decay and early stopping settings (see Appendix A.2.3).

The resulting Optuna configurations are stored in CSV files and reused during benchmarking and simulation, allowing expensive search procedures to be decoupled from final model evaluation.

5.4.2 Model Training and Calibration

The best-performing architectures and hyperparameter configurations identified during benchmarking are loaded from the stored results (Appendix A.2.3).

For each selected horizon H , a separate model is trained independently for each asset using the best configuration associated with that horizon. This allows the framework to specialise predictions to individual tickers while remaining consistent with benchmark-validated settings.

Before capital allocation, predicted probabilities are optionally calibrated to improve probabilistic reliability.

Temperature scaling. For ordinal multi-class prediction, temperature scaling is used to rescale logits while preserving class ranking.

```
def temperature_scale(logits, T):
    return torch.softmax(logits / T, dim=1)
```

Listing 5.6: Temperature scaling.

Isotonic regression. For binary simulation, isotonic regression is used as a non-parametric calibration method fitted on validation probabilities.

```
iso = IsotonicRegression(out_of_bounds="clip")
iso.fit(val_probs, val_labels)
calibrated_probs = iso.predict(test_probs)
```

Listing 5.7: Isotonic regression calibration.

These calibration methods ensure that probabilities passed to the capital allocation layer more closely reflect empirical outcome frequencies.

5.5 Investment Simulation Framework

The investment simulation layer converts predictive model outputs into sequential trading decisions. Predictions are generated using the model configurations identified during the benchmarking stage and applied within a walk-forward framework that mimics realistic deployment conditions.

5.5.1 Per-Asset Training and Walk-forward Retraining

Models are trained independently for each asset to capture asset-specific dynamics for a prediction horizon H .

Following an expanding walk-forward procedure, at each decision point, the model is trained using historical observations up to time t , then used to generate predictions for future observations. The model is periodically refit as new data becomes available.

```
for t in range(train_start, len(data)-H):
    train_df = data.iloc[:t]
    model.fit(X_train, y_train)
```

```
pred = model.predict(X_t)
```

This design ensures that all predictions are produced strictly out-of-sample.

5.5.2 Portfolio Construction

Before capital allocation, the simulation first determines which assets are eligible for investment on a given day. This portfolio construction stage acts as a filtering and diversification layer by removing weak signals, reducing concentration and ensuring that subsequent capital allocation is performed only over a smaller set of higher-conviction candidates. The binary and ordinal multi-class simulations apply this filtering differently, but both ultimately use a softmax-based action selection rule within each sector before proceeding to position sizing.

Binary System In the binary simulation, each ticker receives a calibrated probability of an upward move, denoted p_i . The first portfolio gate is applied at the *sector level*. For each sector s , the simulation computes the mean calibrated probability across all stocks in that sector on that day, a sector is retained only if it clears the set sector threshold. This means the binary simulation does *not* filter stocks individually at the first stage. Instead, it asks whether the sector as a whole is sufficiently strong on average. If the answer is no, all stocks in that sector are discarded for that day.

```
df = daily_predictions_df.copy()
df = df.groupby('sector').filter(
    lambda g: g['calibrated_prediction'].mean() >= sector_threshold
)
```

Listing 5.8: Sector-level filtering in the binary simulation.

After the surviving sectors are identified, the simulation evaluates individual stocks within those sectors. For ticker i , the Kelly edge is computed. Only stocks with positive edge are retained as candidates for selection.

Multi-Classification System In the ordinal multi-class simulation, each ticker receives a probability distribution over ordered return buckets. Let $P_i(c)$ denote the probability of bucket c and let μ_c denote the historical mean return associated with that bucket. The expected return for ticker i is then computed as

$$\hat{r}_i = \sum_{c=0}^{K-1} P_i(c) \mu_c.$$

The probability of an upward move is obtained by summing the probabilities of all buckets corresponding to non-negative returns.

```
Pc = ordinal_to_class_probs(P_cum)
expected_rise = float((Pc[0] * mu_c).sum())

positive_buckets = [i for i in range(K_used) if BUCKET_BINS[i] >= 0]
prob_up = float(Pc[0, positive_buckets].sum())
```

Listing 5.9: Expected return and up-probability in the ordinal simulation.

Unlike the binary simulation, the ordinal multi-class simulation does *not* apply a sector-average confidence filter. Instead *ticker_threshold* is used at the *ticker level*. A ticker survives only if the expected return is above the threshold and it is positive.

After this filtering stage only valid candidates are retained.

Softmax action selection

For both simulation systems, the final stock choice is made using the same softmax-style action selection mechanism. Within each surviving sector, candidates are ranked, the top- K names are retained and a softmax distribution is applied over their scores. In the binary case, the ranking score is the calibrated probability; in the ordinal case, it is the expected return. One stock is then sampled from this sector-specific top- K set (the formula can be found at Appendix A.2.6).

```
top_group = group.sort_values(score_col, ascending=False).head(max(1, top_k))

logits = top_group[score_col].to_numpy() * 100.0 * alpha
logits = logits - logits.max()
weights = np.exp(logits)
probs = weights / weights.sum()

choice_idx = np.random.choice(len(top_group), p=probs)
choice = top_group.iloc[choice_idx]
```

Listing 5.10: Softmax-based action selection within each sector.

This means portfolio construction is not a simple ranking step. Instead, it combines filtering, diversification and probabilistic action selection before capital allocation is applied.

5.5.3 Capital Allocation

Predicted probabilities or expected returns are converted into position sizes using risk-aware allocation rules. The primary allocation strategy is based on the Kelly criterion, which determines the optimal fraction of capital to wager given an edge in probabilistic forecasts. In this project we use quarter-Kelly sizing (see Appendix A.2.9 for more details on *kelly_multiplier* and other investment simulation parameters).

```
kelly_fraction = (p * payoff - (1 - p)) / payoff
stake = capital * kelly_fraction * kelly_multiplier
```

Listing 5.11: Fractional Kelly position sizing.

While Kelly sizing determines the amount of capital allocated to individual positions, cross-asset portfolio allocation is further refined using Mean-Variance Portfolio Theory (MPT). Given expected returns μ and covariance matrix Σ .

```
def compute_mpt_weights(mu, Sigma, risk_lambda=1.0):
    import scipy.optimize as opt

    def obj(w):
        return -(mu @ w - risk_lambda * (w @ Sigma @ w))
```

```

cons = ({'type': 'eq', 'fun': lambda w: np.sum(w) - 1.0},)
bounds = [(0.0, 1.0)] * len(mu)

x0 = np.ones(len(mu)) / len(mu)
res = opt.minimize(obj, x0, method='SLSQP',
                  bounds=bounds, constraints=cons)

return res.x

```

Listing 5.12: Mean–variance portfolio weight optimisation.

A hybrid Kelly–MPT strategy is also implemented. In this approach, Kelly sizing first determines the candidate trade budget based on predicted probabilities, after which MPT redistributes this capital across selected assets using their expected returns and covariance structure.

```

kelly_budget = capital * kelly_fraction

weights = compute_mpt_weights(expected_returns, covariance_matrix)

investment_amounts = kelly_budget * weights

```

Listing 5.13: Hybrid Kelly–MPT allocation.

This hybrid approach allows the system to incorporate both probabilistic forecast confidence (via Kelly sizing) and cross-asset diversification effects (via mean–variance optimisation).

5.5.4 Early Exit Mechanism

Positions may be closed prior to the full horizon if subsequent predictions indicate deteriorating expected returns. At each new time step $t + 1$, the model re-evaluates the position using updated inputs.

If the predicted return falls below a defined threshold, the position is liquidated early. Appendix A.1 to visualises the frequency of model agreements that trigger an early exit.

```

if predicted_next_day_return < early_exit_threshold and regression_direction ==
    -1 and regression_confidence > 0.5:
    close_position()

```

Listing 5.14: Early exit trigger.

Auxiliary models trained using historical data up to the 100-day mark are stored in a parquet file (an example of the signal produced can be found at Appendix A.2.5). The models then generates next-day directional predictions for each asset using both binary classification, regression-based return direction and it's confidence in its prediction are then used to trigger an early exit. This mechanism reduces exposure during adverse market movements and improves risk control.

5.5.5 Portfolio Tracking and Constraints

Portfolio value evolves multiplicatively as positions are opened and closed over time.

```

portfolio_value *= (1 + np.sum(weights * realised_returns))

```

Listing 5.15: Portfolio value calculation (Appendix A.2.7).

Transaction costs, capital limits, drawdown controls and position size constraints are applied to ensure realistic trading behaviour. The specific parameter values used in the simulation (including transaction fee assumptions, capital caps, and risk limits) are provided in Appendix A.2.9.

5.6 Reproducibility and Robustness Measures

Several safeguards are implemented to ensure that results are reproducible and free from methodological bias.

5.6.1 Random Seed Control and Deterministic Execution

To improve reproducibility and ensure fair comparison across experiments, a fixed random seed of 42 is used throughout the notebooks. This seed is applied consistently to Python’s random module, NumPy, and PyTorch. For GPU execution, deterministic cuDNN behaviour is enabled and benchmark mode is disabled to reduce nondeterministic variation between runs.

This does not guarantee bitwise-identical results in every environment, but it substantially improves experimental consistency and comparability across model configurations.

5.6.2 Data Handling

Due to the non-continuous nature of real-world financial data, the system partitions each ticker’s history into contiguous trading periods before sequence generation. This ensures that recurrent models are trained only on temporally coherent windows and do not implicitly treat large gaps in observation as ordinary day-to-day transitions.

Within each contiguous period, rolling input sequences of length L are constructed and paired with their corresponding H -step-ahead targets. Periods that are too short to support both the input window and the forward horizon are discarded. Appendix A.2.8 exhibits this function.

5.6.3 Chronological Splits and No-Shuffle Training

Financial time-series exhibit strong temporal dependence, therefore, datasets are split chronologically rather than randomly. Training always uses earlier observations, while validation and test sets contain later observations.

This approach follows established recommendations for time-series forecasting (Arratia et al. 2025) (Appendix A.2.8). Shuffling is explicitly disabled to preserve temporal ordering.

6 | Experiments & Results

6.1 Evaluation Metrics

Evaluation is conducted across three complementary layers: predictive performance and economic outcomes. This multi-layered framework ensures that statistical accuracy, probability validity and financial viability are jointly assessed.

Classification and Ordinal Tasks. For directional and ordinal multi-class prediction, the primary metric is the Matthews Correlation Coefficient (MCC), which is most appropriate for imbalanced classification problems and was used by Xu and Cohen (2018) allowing for direct comparison.

Regression Tasks. For return magnitude estimation, the primary metric is Mean Absolute Error (MAE). This is preferred over Mean Squared Error (MSE) in financial return forecasting because financial returns exhibit heavy tails and outliers, with MAE being less sensitive to extreme deviations it therefore provides a more robust estimate of average prediction error (Brassington 2017).

To enable direct comparison with classification models, regression outputs are converted into directional signals based on the sign of the predicted return and the MCC is computed on the resulting binary outcomes.

6.1.1 Economic Metrics

The ultimate objective of the framework is economic viability rather than statistical accuracy alone. Allocation strategies are therefore evaluated using established portfolio performance metrics that capture both growth and risk characteristics.

- **Cumulative Return (Return on Investment (ROI)):** measures total capital growth over the evaluation period and reflects the compounding effect of sequential returns. It is fundamental in assessing whether a strategy generates positive long-term wealth (Thorp 1997; Elton et al. 2007).
- **Volatility:** represents the dispersion of portfolio returns and serves as a proxy for total risk. In mean-variance theory, variance (or standard deviation) is the primary measure of portfolio risk (Markowitz 1952).
- **Sharpe Ratio:** originally termed the “reward-to-variability ratio,” the Sharpe Ratio was introduced by Sharpe (1994) as a measure of risk-adjusted performance. It evaluates excess return relative to return volatility, allowing comparison of portfolios on a risk-normalised basis.

All economic metrics are computed with a transaction cost incorporated, ensuring realistic assessment of capital growth, risk exposure, and trading frictions.

6.2 Experimental Setup

The evaluation is divided into two stages. First, models are benchmarked to assess their predictive performance under fixed train, validation, and test splits. Second, the best-performing configurations are transferred into the investment simulation, where they are retrained in a walk-forward manner and evaluated in terms of economic performance.

6.2.1 Evaluation Splits

For the benchmarking stage, each ticker is divided into fixed date ranges for training, validation, testing and simulation:

- Train: [2014-01-01, 2015-08-01),
- Validation: [2015-08-01, 2015-10-01),
- Test: [2015-10-01, 2016-01-01),
- Simulation: [2017-04-11 2017-08-31).

In addition to comparing model families and feature groups, the experiments also vary the prediction horizon H .

For the investment simulation, the evaluation protocol is more realistic. An initial training window is used to fit the first per-asset models, after which predictions are generated sequentially through time over the last **100 days** of the dataset. Models are retrained periodically using only historical data available prior to the decision date. In this way, the simulation mimics live deployment rather than a single static train-test experiment.

6.2.2 Compared Models and Feature Sets

The experiments evaluate four recurrent architectures: LSTM, GRU, BiLSTM, and BiGRU. These models were chosen because LSTM and GRU are standard sequence learners for temporal forecasting, while their bidirectional variants test whether richer contextual encoding improves predictive performance (Section 2.2).

The full list of compared model families, feature subsets, horizon values, and tuning ranges is provided in Appendix A.2.10.

6.3 Predictive Performance

This section evaluates predictive performance before introducing meta-features, calibration, or portfolio allocation. The aim is to identify the strongest model configuration and forecast horizons to carry forward into the investment simulation stage, while also comparing the results against prior work such as Xu and Cohen (2018).

6.3.1 Binary Classification Results

The experiments evaluate four recurrent model families (Section 6.2.2) chosen to compare standard recurrent neural networks with their bidirectional variants. Table 6.1 reports the strongest 1-day binary configuration for each model family, ranked by MCC, with full results provided in Appendix A.2.

The best overall configuration is BiGRU with sentiment features, achieving a directional accuracy of 0.525 and an MCC of 0.086. This is the highest MCC among the tested binary models and slightly exceeds the 0.080 MCC reported for the HFA model of Xu and Cohen (2018). Although the gain is small, it is notable given the difficulty of next-day stock prediction and the absence of trade filtering.

Table 6.1: Top binary classification results by model for the 1-day prediction horizon.

Model	Feature Set	Dir Acc	MCC	F1	Precision	Recall
BiGRU	sentiment	0.525	0.086	0.331	0.476	0.360
BiLSTM	sentiment	0.525	0.077	0.363	0.488	0.404
GRU	sentiment	0.521	0.069	0.379	0.528	0.423
LSTM	base	0.516	0.069	0.405	0.508	0.486

Table 6.2 compares the best model in this study with representative baselines. While HFA reports higher average accuracy, the BiGRU sentiment model achieves the strongest MCC, indicating better balanced directional prediction.

Table 6.2: Comparison against reported binary prediction baselines.

Model	Avg. Accuracy	MCC
ARIMA (Brown 1963)	0.514	-0.020
HAN (Hu et al. 2019)	0.576	0.051
HFA (Xu and Cohen 2018)	0.582	0.080
BiGRU + Sentiment (ours)	0.525	0.086

Overall, sentiment-enhanced bidirectional recurrent models are among the strongest 1-day binary configurations, though gains from text are modest rather than universal. The BiGRU sentiment model is therefore selected as the strongest binary candidate for downstream simulation.

6.3.2 Regression Results

Table 6.3 presents the strongest 1-day regression configuration for each model family, ranked by MAE, with full results in Appendix A.3. The best configuration is BiLSTM with emotion features, achieving the lowest MAE of 0.011571. However, differences across models are small, suggesting broadly similar short-horizon regression performance.

Table 6.3: Top regression results by model for the 1-day prediction horizon, ordered by MAE (lower is better).

Model	Feature Set	Dir Acc	MAE	MSE	MCC	F1	Recall
BiLSTM	emotion	0.510	0.011571	0.000270	0.002	0.314	0.433
LSTM	sector_sentiment	0.496	0.011589	0.000272	-0.002	0.344	0.488
BiGRU	base	0.494	0.011598	0.000272	-0.005	0.380	0.520
GRU	sector_sentiment	0.501	0.011609	0.000272	0.002	0.425	0.566

Regression results also show that lower magnitude error does not necessarily imply stronger directional discrimination, as MCC values remain close to zero. For this reason, the regression benchmark is selected on MAE, but appears less decision-useful than the classification tasks.

6.3.3 Ordinal Multi-Class Results

Table 6.4 summarises the strongest 1-day ordinal multi-class configuration for each model family, ranked by MCC, with full results in Appendix A.4. The best MCC is achieved by BiGRU with base features, reaching 0.078 with a directional accuracy of 0.516. The highest micro-accuracy

and macro-F1, however, are achieved by sector-emotion variants, indicating that the models best at recovering exact return buckets are not always those best at capturing direction.

Table 6.4: Ordinal multi-class results for the 1-day prediction horizon.

Model	Feature Set	Micro Acc	Macro F1	MCC	Dir Acc	F1	Recall
BiGRU	base	0.152	0.104	0.078	0.516	0.362	0.400
LSTM	sector_emotion	0.186	0.120	0.071	0.522	0.441	0.496
GRU	sector_emotion	0.191	0.124	0.070	0.522	0.424	0.462
BiLSTM	sector_unified_emotion	0.167	0.116	0.065	0.513	0.397	0.426

As expected, exact bucket prediction is harder than binary direction prediction, so micro-accuracy remains low. Nevertheless, directional metrics remain competitive, suggesting that ordinal models preserve useful directional information while also modelling return magnitude structure. Overall, the BiGRU base model is selected as the strongest ordinal benchmark.

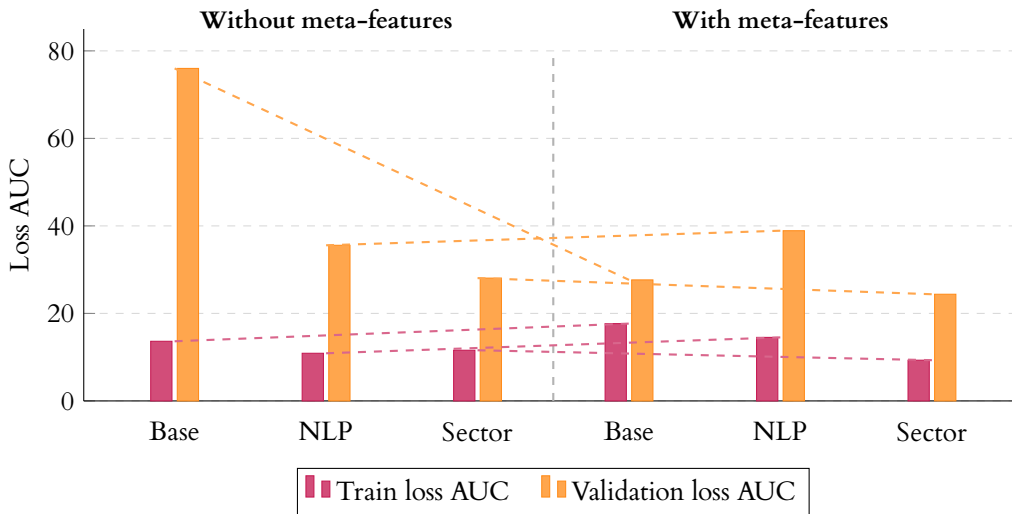
6.4 Meta-Feature and Ablation Results

6.4.1 Effect of Meta-Features

This section evaluates whether reliability-aware meta-features improve performance across binary classification, regression, and ordinal multi-class prediction. Full meta-feature results are provided in Appendix A.5.

Effects on Training To assess whether meta-features improve training stability and generalisation, we compare the area under the training and validation loss curves for the BiGRU models at the 1-day horizon. Figure 6.1 shows that the main effect is on validation behaviour rather than training loss: validation-loss AUC falls sharply for the base feature set (75.99 to 27.66) and modestly for the sector feature set (28.07 to 24.38), while the NLP setting worsens slightly (35.57 to 38.90). Meta-features therefore appear most helpful for the base and sector configurations, where they improve generalisation across epochs.

Figure 6.1: Effect of meta-features on training dynamics of the BiGRU models (1-day horizon). Loss behaviour summarised using area under the training and validation loss curves.



Effects on Results Next, we compare the strongest baselines against the strongest model for the task with meta-features.

Table 6.5: Best binary classification results with and without meta-features (1-day horizon).

Setting	Model	Feature Set	Dir Acc	MCC	F1	Recall
No Meta	BiGRU	sentiment	0.525	0.086	0.331	0.360
Meta	GRU	meta_sector_finbert	0.516	0.081	0.358	0.400

Binary Classification For binary classification, the best meta-feature model remains competitive but does not surpass the strongest baseline on MCC. It does, however, improve F1 and recall.

Regression In regression, meta-features provide a modest improvement in both MAE and MSE while leaving directional accuracy largely unchanged.

Table 6.6: Best regression results with and without meta-features (1-day horizon).

Setting	Model	Feature Set	Dir Acc	MAE	MSE
No Meta	BiLSTM	base	0.507	0.01155	0.000270
Meta	BiGRU	meta_sector	0.509	0.01135	0.000242

Ordinal Multi-Class Classification The clearest benefit appears in the ordinal task, where meta-features improve micro-accuracy, macro-F1, and MCC. Overall, meta-features produce mixed but encouraging results, with the strongest gains appearing in regression and ordinal prediction rather than binary classification.

Table 6.7: Best ordinal multi-class results with and without meta-features (1-day horizon).

Setting	Model	Feature Set	Micro Acc	Macro F1	MCC
No Meta	BiGRU	base	0.152	0.104	0.078
Meta	BiLSTM	meta_sector_sentiment	0.163	0.111	0.083

6.4.2 Effect of Variable Horizon Design

Prediction horizon has a substantial effect on performance. In general, extending the forecast window from 1 day to 5 days improves binary and ordinal classification, but worsens regression MAE. This suggests that longer horizons reduce short-term noise for direction-based tasks, while making magnitude prediction harder.

Table 6.8 compares the best 1-day and 5-day configurations, with full results in Appendix A.6. For binary classification, MCC increases from 0.086 to 0.152 without meta-features and from 0.081 to 0.145 with meta-features. Ordinal results show a similar pattern, with MCC rising from 0.078 to 0.145 without meta-features and from 0.083 to 0.126 with meta-features. By contrast, regression MAE worsens substantially at 5 days, rising from around 0.011 to around 0.025 in both settings.

Overall, horizon choice is a major modelling decision rather than a minor implementation detail. The 5-day horizon is more effective for classification-based tasks, while regression becomes less attractive as the prediction window increases.

Table 6.8: Comparison of the best-performing 1-day and 5-day models across tasks.

Task	Setting	Horizon	Model / Feature Set	Dir Acc	MCC	MAE / Micro Acc
Binary	No Meta	1-day	BiGRU / sentiment	0.525	0.086	–
Binary	No Meta	5-day	GRU / sector_sentiment	0.529	0.152	–
Binary	Meta	1-day	GRU / meta_sector_finbert	0.516	0.081	–
Binary	Meta	5-day	BiGRU / meta	0.531	0.145	–
Regression	No Meta	1-day	LSTM / base	0.505	0.022	0.0117
Regression	No Meta	5-day	LSTM / sector_finbert	0.533	0.055	0.0260
Regression	Meta	1-day	BiGRU / meta_sector	0.509	0.028	0.0114
Regression	Meta	5-day	BiLSTM / meta_sector_emotion	0.526	0.042	0.0253
Ordinal	No Meta	1-day	BiGRU / base	0.516	0.078	0.152
Ordinal	No Meta	5-day	BiGRU / sector	0.533	0.145	0.208
Ordinal	Meta	1-day	BiLSTM / meta_sector_sentiment	0.525	0.083	0.163
Ordinal	Meta	5-day	GRU / meta_emotion	0.530	0.126	0.213

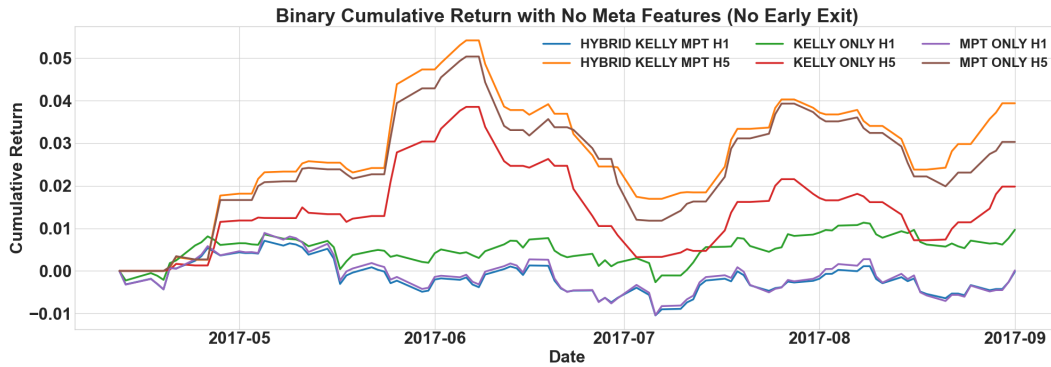
6.5 Investment Simulation Results

For the investment simulation experiments, the best performing models from both the 1-day and 5-day horizons were selected for the binary and ordinal multi-class prediction tasks. The corresponding feature configurations were then used to generate trading signals over the last 100 days within the investment simulation framework.

6.5.1 Binary Simulation Results

The binary investment simulation compares the strongest 1-day and 5-day configurations from the benchmarking stage, both without and with meta-features. These selected benchmark models are listed in Appendix A.7.1.

Figures 6.2 and 6.3 show the corresponding cumulative return curves, while Table 6.9 summarises the best-performing 1-day and 5-day strategies under each setting.

*Figure 6.2: Cumulative returns for binary classification strategies without meta features.*

Two patterns are clear. First, the 5-day horizon consistently outperforms the 1-day horizon. Without meta-features, the best 5-day strategy achieves 3.94% ROI, compared with 0.97% for the best 1-day strategy. The same pattern holds with meta-features, where the best 5-day result rises to 12.47% while the best 1-day result reaches only 1.05%. This is also visible in Figures 6.2 and 6.3, where the 5-day strategies show sustained growth while the 1-day strategies remain close to the initial capital.

Second, meta-features substantially improve the 5-day binary strategies. The best non-meta

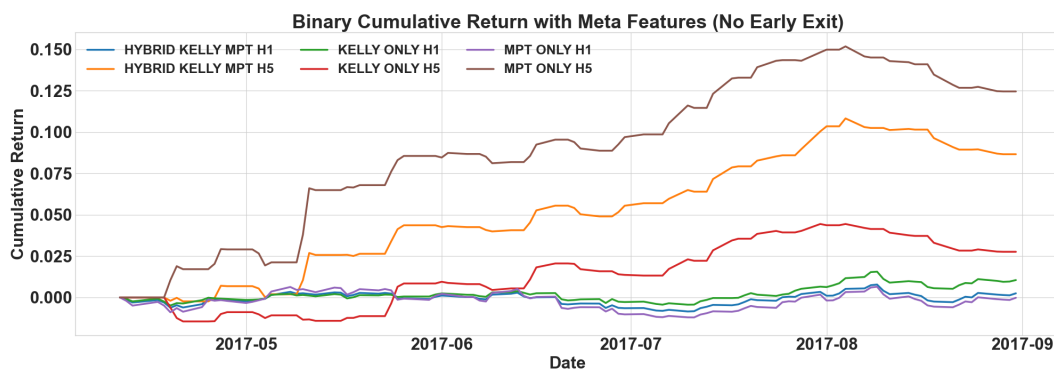


Figure 6.3: Cumulative returns for binary classification strategies incorporating meta features.

Table 6.9: Best binary investment simulation results by horizon, with and without meta-features. Full results are reported in Appendix A.16.

Setting	Best Strategy	Horizon	ROI (%)	Sharpe	Volatility
No Meta	Kelly	1-day	0.97	1.04	0.024
No Meta	Hybrid Kelly–MPT	5-day	3.94	2.21	0.044
Meta	Kelly	1-day	1.05	1.26	0.021
Meta	MPT	5-day	12.47	4.34	0.069

result is the hybrid Kelly–MPT strategy at 3.94% ROI, whereas the best meta-feature result is the MPT-only strategy at 12.47% ROI with a Sharpe ratio of 4.34. By contrast, meta-features have only a limited effect at the 1-day horizon.

Overall, the binary simulation results suggest that longer-horizon signals are more economically useful, and that meta-features are most beneficial when combined with the 5-day horizon.

6.5.2 Ordinal Multi-Class Simulation Results

The ordinal investment simulation compares the strongest 1-day and 5-day configurations from the benchmarking stage, both without and with meta-features. These selected benchmark models are listed in Appendix A.7.2. The corresponding cumulative return curves are provided in Appendix A.11 and A.12, while Table 6.10 summarises the best-performing 1-day and 5-day strategies under each setting.

Table 6.10: Best ordinal multi-class investment simulation results by horizon, with and without meta-features. Full results are reported in Appendix A.17.

Setting	Best Strategy	Horizon	ROI (%)	Sharpe	Volatility
No Meta	MPT	1-day	-0.27	-0.19	0.033
No Meta	MPT	5-day	10.56	4.59	0.055
Meta	Kelly	1-day	1.72	1.22	0.036
Meta	Kelly	5-day	3.61	2.22	0.041

The ordinal results show two clear patterns. First, the 5-day horizon again outperforms the 1-day horizon: without meta-features, the best 5-day strategy returns 10.56%, whereas the best 1-day result is still slightly negative at -0.27%. Second, meta-features have a mixed effect. They improve the 1-day setting, raising the best result to 1.72%, but weaken the 5-day setting, where

the best meta-feature strategy returns 3.61%. Overall, longer-horizon ordinal signals remain the most economically useful, while meta-features mainly improve short-horizon stability.

6.5.3 Risk and Early Exit

This section considers early exit and rolling risk together, since both are concerned with how the system manages uncertainty after a signal has been generated. Early exit is only applicable to the 5-day horizon, where positions can be closed before the full holding period if the signal becomes unfavourable. Risk is assessed using rolling 20-day volatility and Sharpe ratio, with full rolling figures and strategy-level results reported in the appendix.

Without Meta-Features Table 6.11 summarises the main 5-day results without meta-features. In the binary system, early exit is clearly beneficial: the best configuration improves from 3.94% ROI and a mean Sharpe of 1.51 without early exit to 6.57% ROI and 4.51 mean Sharpe with early exit, while volatility also falls. This suggests that explicit liquidation rules are valuable when the model relies only on raw predictive signals.

The ordinal system shows the opposite pattern. Its strongest 5-day configurations occur without early exit, with Kelly and MPT achieving mean Sharpe ratios above 4.8 and MPT reaching the highest ROI of 10.56%. Introducing early exit lowers both return and mean Sharpe, indicating that profitable longer-horizon ordinal trades are being closed too aggressively.

Table 6.11: Summary of binary early-exit and rolling-risk behaviour without meta-features. Full results are reported in Appendix A.8.1 and Appendix A.18.

Task	Configuration	ROI (%)	Mean Volatility	Mean Sharpe	Interpretation
Binary	Hybrid/5-day/OFF	3.94	0.0434	1.51	Best binary case without EE
Binary	Hybrid/5-day/ON	6.57	0.0374	4.51	Early exit improves binary case
Ordinal	MPT/5-day/OFF	10.56	0.0529	4.84	Best ordinal return without EE
Ordinal	Kelly/5-day/OFF	3.35	0.0194	5.03	Best ordinal Sharpe without EE
Ordinal	MPT/5-day/ON	2.62	0.0430	1.68	Early exit weakens ordinal case

Overall, without meta-features, early exit acts as a useful risk-control mechanism for the binary system but weakens the stronger ordinal 5-day configurations.

With Meta-Features Table 6.12 reports the corresponding results with meta-features. In the binary system, meta-features substantially improve the 5-day strategies even without early exit: the strongest case is MPT with 12.47% ROI and a mean Sharpe of 4.78. However, once meta-features are present, early exit becomes harmful, reducing both return and Sharpe. This suggests that reliability-aware inputs already provide part of the adaptive control that early exit previously supplied.

In the ordinal system, the effect is more selective. Meta-features do not match the strongest 5-day non-meta results, but they make early exit more useful: the best ordinal meta configuration is MPT with early exit, which reaches 4.86% ROI and a mean Sharpe of 2.58. Meta-features also improve weaker 1-day ordinal cases, as shown in the appendix, indicating that they are most helpful when the base signal is less stable.

Taken together, these results show that early exit and meta-features interact rather than contributing independently. In the binary system, meta-features improve risk-adjusted performance and largely replace the need for early exit. In the ordinal system, meta-features are more useful in weaker settings and appear to work better alongside early exit, but they do not improve on the strongest long-horizon non-meta baseline. The practical value of both mechanisms therefore depends on task formulation, horizon, and the underlying strength of the predictive signal.

Table 6.12: Summary of ordinal multi-class early-exit and rolling-risk behaviour with meta-features. Full results are reported in Appendix A.8.2 and again in Appendix A.18.

Task	Configuration	ROI (%)	Mean Volatility	Mean Sharpe	Interpretation
Binary	MPT/5-day/OFF	12.47	0.0630	4.78	Best binary meta case
Binary	MPT/5-day/ON	3.57	0.0490	1.81	Early exit now weakens binary case
Ordinal	Kelly/5-day/OFF	3.61	0.0410	1.87	Best ordinal case without EE
Ordinal	MPT/5-day/ON	4.86	0.0455	2.58	Early exit improves ordinal meta case
Ordinal	Hybrid/1-day/OFF	1.47	0.0319	0.70	Meta-features stabilise weaker 1-day case

6.5.4 Allocation Strategy Comparison

Early exit is excluded here to keep the comparison focused on allocation strategy rather than trade-management effects. Table 6.13 summarises how the relative strength of KELLY, MPT and HYBRID changes across horizons and settings, while Tables 6.9 and 6.10 report the corresponding headline simulation results. A clear pattern emerges: KELLY is generally strongest in weaker short-horizon settings, MPT performs best when signals are stronger and more persistent, and HYBRID is most attractive when return and risk need to be balanced.

Table 6.13: Allocation strategy comparison across the strongest simulation settings without Early-Exit.

Setting	H	Best ROI	Best Sharpe	Key Pattern
Binary/No Meta	1	Kelly	Kelly	Kelly best in weak short horizon
Binary/Meta	1	Kelly	Kelly	Kelly remains strongest
Ordinal/No Meta	1	MPT	MPT	All strategies unprofitable
Ordinal/Meta	1	Kelly	Kelly	Meta stabilises short horizon
Binary/No Meta	5	Hybrid	Hybrid	Hybrid strongest overall
Binary/Meta	5	MPT	MPT	MPT best with strong signal
Ordinal/No Meta	5	MPT	Hybrid	Return-risk trade-off appears
Ordinal/Meta	5	Kelly	Kelly	Meta ordinal setting weaker overall

For the **1-day horizon**, KELLY is usually the strongest strategy. As shown in Table 6.13, it achieves both the best ROI and best Sharpe in the binary no-meta, binary meta and ordinal meta settings. Tables 6.9 and 6.10 show the corresponding best 1-day returns: 0.97%, 1.05%, and 1.72%, respectively. The only exception is the ordinal no-meta setting, where all 1-day strategies are unprofitable and MPT is merely the least negative.

For the **5-day horizon**, the strongest strategy depends more on whether the objective is maximum return or stronger risk-adjusted performance. Table 6.13 shows that MPT delivers the best ROI in the binary meta and ordinal no-meta settings, corresponding to the strongest-return cases in Tables 6.9 and 6.10. In contrast, HYBRID is more competitive when risk-adjusted performance is emphasised. In the binary no-meta setting, HYBRID is the strongest overall 5-day strategy, while in the ordinal no-meta setting it achieves the best Sharpe even though MPT attains the higher absolute return. This highlights an important trade-off: the strategy with the highest return is not always the one with the best return per unit of risk.

Overall, the allocation results support a consistent interpretation. KELLY is best suited to noisier, short-horizon environments where concentrated sizing is preferable to broad diversification. MPT performs best when the predictive signal is stronger and more persistent, particularly at the 5-day horizon where diversification across opportunities yields the highest absolute returns. HYBRID, as shown in Table 6.13, is most useful when a balance between return and risk is required, combining Kelly-style selectivity with some of the stabilising benefits of diversification.

7 | Discussion

7.1 Overview

This dissertation examined whether stock prediction becomes more practically useful when embedded in an end-to-end decision framework combining multimodal forecasting, reliability-aware meta-features and capital allocation. Overall, the results support this framing. The main finding is that practical performance depended less on any single model component than on the interaction between horizon, task formulation, reliability modelling and allocation. In particular, 5-day classification-based signals were more useful than 1-day signals, meta-features improved performance in selected settings and allocation strategy strongly influenced whether predictive gains translated into realised returns.

7.2 Interpretation by Research Question

Research questions can be found at Section 3.3.

7.2.1 RQ1: Predictive Performance

RQ1 asked *whether combining textual and price-based information improves prediction* under temporally valid evaluation. The answer is **partly yes**. Text-enhanced feature sets appeared repeatedly among the strongest binary and ordinal models, but their benefits were not universal across all architectures or tasks.

A clearer result concerns horizon choice. The 5-day binary and ordinal models outperformed their 1-day counterparts both in benchmark metrics and in downstream simulation, suggesting that slightly longer horizons contain more learnable directional structure than next-day movement. By contrast, regression was less useful overall: although some models achieved competitive MAE, they did not translate as effectively into directional or economic value. This suggests that classification-based formulations were better aligned with the decision layer used in the simulation framework.

7.2.2 RQ2: Meta-Features and Reliability Modelling

RQ2 asked *whether prior predictions and realised errors, used as meta-features, improve performance* under non-stationary conditions. The answer is **yes, but selectively**. Meta-features produced their clearest gains in the binary 5-day investment setting, where they substantially improved results, particularly under MPT allocation.

Their effect was weaker in the ordinal setting. Meta-features improved the weaker 1-day ordinal configurations more clearly than the strongest 5-day ordinal ones, which suggests that reliability-aware inputs are most useful when the base signal is noisy or unstable. Meta-features should therefore be understood **less as a universal performance booster and more as an adaptive mechanism** whose value depends on the strength of the underlying predictive task.

7.2.3 RQ3: Allocation Strategy Comparison

RQ3 asked *how Kelly, MPT and Hybrid allocation compare*. The answer is that **no single allocator dominates across all settings**. Kelly performed best in most 1-day cases, where signals were weaker and concentrated sizing was more effective than diversification.

At the 5-day horizon, MPT produced the highest absolute returns in the strongest settings, particularly binary-meta and ordinal-no-meta. The Hybrid strategy was often the most balanced option, achieving competitive Sharpe ratios without relying entirely on concentrated Kelly-style sizing. This suggests that **allocation strategy should be matched to signal structure** rather than treated as a secondary design choice.

7.2.4 RQ4: Interaction Effects

RQ4 asked *whether calibration, reliability modelling and allocation interact in meaningful ways*. The answer is **clearly yes**. The effect of early exit, for example, depended strongly on whether meta-features were present: it improved some binary non-meta settings, but weakened stronger meta-feature configurations.

More broadly, the best allocation rule changed across horizons and task formulations, showing that system components cannot be evaluated independently. The practical value of a forecast depends not only on predictive quality, but also on how reliability is modelled and how the resulting signal is converted into capital allocation. This supports the central claim of the dissertation that financial prediction should be treated as an integrated decision problem rather than a stand-alone forecasting task.

7.3 Limitations

Several limitations should be acknowledged.

First, the underlying dataset imposes important constraints. The project is built on the StockNet-style benchmark setting, which covers a finite time period, a fixed set of U.S. stocks, and sparse tweet activity. As noted in the implementation chapter, tweet-derived features are not available uniformly across all days, requiring neutral filling and aggregation decisions that may weaken the textual signal. This limits external validity and may partly explain why some textual and meta-feature gains are inconsistent across tasks.

Second, although the simulation uses a realistic walk-forward design, the final investment evaluation still spans only the last 100 trading days. This is enough to compare strategies under controlled conditions, but it is not sufficient to claim robustness across many market regimes. A longer out-of-sample period, or evaluation across multiple market episodes, would strengthen the conclusions about stability and deployability.

Third, the decision layer remains sensitive to estimation error. This is especially true for Kelly and MPT, both of which rely on estimated probabilities, returns, or covariance structures. Even when calibration and meta-features help, errors in these inputs can still produce over-allocation, unstable exposure, or drawdown concentration. This is a known issue in portfolio construction and is already recognised in the dissertation's discussion of capital, leverage, and non-stationarity constraints.

Fourth, the meta-feature search space was substantially larger than the baseline feature space. Although the tuning process still used 100 Optuna trials, the inclusion of many additional feature-set combinations means that the effective search coverage per configuration was lower. In other words, the meta-feature experiments were conducted under the same trial budget despite a significantly expanded design space. This means that some meta-feature configurations may not

have been tuned as thoroughly as the baseline ones, and stronger results may have been possible with a larger search budget.

Fifth, transaction costs, capital caps, and portfolio controls were included, but the simulation does not capture all practical frictions of live trading. There is no explicit modelling of market impact, slippage beyond chosen assumptions, execution latency, or changing liquidity. As a result, the reported results should be interpreted as realistic simulations rather than evidence of immediate deployability.

Sixth, the study does not include live deployment or paper trading. This means the framework is still being evaluated in a historically replayed environment, not in a live adaptive market setting where behaviour could change in response to execution.

A further limitation is that the Kelly, MPT and hybrid allocation parameters were not exhaustively fine-tuned. Settings such as the fractional Kelly multiplier, MPT budget share and hybrid weighting were chosen to be reasonable rather than analyst-optimised, so the reported results should be interpreted as comparative evidence of framework behaviour rather than fully optimised trading performance.

Finally, computational limitations constrained the breadth of experimentation. The project already evaluated many architectures, feature groups, horizons and allocation rules, but further exploration of larger hyperparameter budgets, broader transformer comparisons, or alternative calibration and reliability formulations would likely reveal additional structure.

7.4 Concluding Discussion and Practical Implications

The main implication of this dissertation is that financial prediction is most useful when treated as an integrated decision problem rather than a stand-alone forecasting task. The strongest systems were those in which the predictive layer, reliability layer and allocation layer were well aligned: longer horizons improved extractable signal, meta-features improved adaptation in several settings and allocation strategy determined whether those signals produced return, stability, or both.

Several practical lessons follow. First, slightly longer-horizon signals appear more useful than next-day predictions in this dataset and framework, suggesting that daily trading systems may benefit more from modestly extended holding periods than from attempting to exploit highly noisy next-day movement. Second, reliability-aware features are most useful in unstable or noisy environments, implying that financial models can benefit from tracking their own recent predictive quality rather than assuming static confidence. Third, allocation strategy should be matched to signal characteristics: Kelly-style sizing is most effective in weaker short-horizon settings, MPT performs best when signals are stronger and more persistent and hybrid approaches offer a practical compromise when both growth and risk control matter.

Overall, the results reinforce that predictive accuracy alone is not sufficient. A model becomes practically valuable only when its outputs are reliable enough, informative enough and well matched to downstream decision rules to generate stable economic value under non-stationary conditions.

8 Conclusion

This dissertation examined how stock trend prediction can be made more practically useful by evaluating models within an end-to-end investment decision framework rather than through accuracy-focused benchmarking alone. Building on Xu and Cohen (2018), the project extended the task from fixed next-day prediction to a broader framework combining multimodal forecasting, reliability-aware meta-features, and downstream capital allocation.

To achieve this, the dissertation developed and evaluated a multimodal pipeline using historical price data, textual features, meta-features derived from prior predictive behaviour and multiple allocation strategies across binary classification, ordinal multi-class classification and investment simulation under 1-day and 5-day horizons. This made it possible to assess not only predictive quality, but also whether improvements translated into economically meaningful outcomes.

Three main findings emerged. Firstly, the 5-day horizon was consistently more useful than the 1-day horizon in both predictive and economic terms, suggesting that slightly longer-horizon signals contain more learnable directional structure than next-day movement. Secondly, meta-features improved performance selectively rather than uniformly: they were especially helpful in the stronger binary 5-day setting, but provided more mixed benefits in the ordinal system. Thirdly, allocation strategy had a decisive effect on realised performance. Kelly was generally strongest in weaker short-horizon settings, MPT produced the highest returns in the strongest 5-day settings and Hybrid often provided the best balance between return and risk.

The strongest empirical results support the dissertation's central claim that financial prediction should be treated as a decision problem rather than a stand-alone forecasting task. The framework exceeded the Xu and Cohen (2018) benchmark on MCC at the 1-day horizon without filtering and, more importantly, produced substantially stronger downstream economic outcomes, including a best binary return of 12.47% under the 5-day meta-feature MPT configuration.

Overall, the contribution of this dissertation lies not in proposing a single new network architecture, but in showing that useful financial prediction depends on the alignment of predictive modelling, reliability awareness and capital allocation. The results demonstrate that models become more practically valuable when judged by how effectively they support risk-aware investment decisions under non-stationary conditions.

8.1 Future Work

There are three main directions for future work. The first is broader evaluation: extending the dataset, increasing the out-of-sample period and testing across a wider range of market regimes would strengthen claims about robustness. The second is improved modelling: richer text sources, more explicit reliability and uncertainty estimation and deeper hyperparameter optimisation could improve predictive quality and adaptation. The third is a stronger decision layer: future work could explore alternative allocation schemes, more realistic execution assumptions, and continuously adaptive approaches such as online calibration or reinforcement learning in a live deployment setting.

A | Appendices

A.1 Dataset

This appendix summarises the columns contained in the final master DataFrame used throughout the benchmarking and simulation pipelines. The dataset combines raw price variables, technical indicators, NLP-derived features, sector-level aggregates, and reliability-aware meta-features.

A.1.1 Core Market Data

- `date`: Trading date.
- `open`: Opening price.
- `high`: Highest price during the session.
- `low`: Lowest price during the session.
- `close`: Closing price.
- `adj_close`: Adjusted closing price.
- `volume`: Trading volume.
- `ticker`: Stock ticker symbol.
- `company_name`: Company name.
- `sector`: Sector classification.

A.1.2 Technical Indicators

- `ema_12`, `ema_26`, `ema_50`: Exponential moving averages over 12, 26, and 50 days.
- `macd_12_26_9`: MACD line.
- `macdh_12_26_9`: MACD histogram.
- `macds_12_26_9`: MACD signal line.
- `rsi_14`: 14-day Relative Strength Index.
- `stochrsik_14_14_3_3`, `stochrsid_14_14_3_3`: Stochastic RSI components.
- `attr_14`: 14-day Average True Range ratio.
- `bb_upper`, `bb_middle`, `bb_lower`: Bollinger Band upper, middle, and lower levels.
- `obv`: On-Balance Volume.

A.1.3 Return Features

- `ret_1d`: One-day return.
- `roll_ret_1d`: Rolling short-horizon return feature.
- `roll_ret_5d`: Rolling 5-day return feature.
- `roll_ret_20d`: Rolling 20-day return feature.

A.1.4 Sentiment, Emotion, and Stance Features

- `sentiment`: Aggregate daily sentiment score from tweets.
- `emotion_anger`, `emotion_disgust`, `emotion_fear`, `emotion_joy`, `emotion_neutral`, `emotion_sadness`, `emotion_surprise`: Daily summed emotion scores.
- `emotion_anger_pct`, `emotion_disgust_pct`, `emotion_fear_pct`, `emotion_joy_pct`, `emotion_neutral_pct`, `emotion_sadness_pct`, `emotion_surprise_pct`: Proportional emotion features.
- `positive_emotion`, `negative_emotion`, `uncertainty_emotion`: Aggregated grouped emotion features.
- `positive_emotion_pct`, `negative_emotion_pct`, `uncertainty_emotion_pct`: Proportional grouped emotion features.
- `stance_label`: Daily stance classification label.
- `stance_score`: Confidence score from the stance classifier.
- `finbert_label`: FinBERT sentiment label.
- `finbert_score`: FinBERT confidence score.
- `finbert_up`, `finbert_down`, `finbert_neutral`: FinBERT class probabilities or aggregated class indicators.

A.1.5 Sector-Level Features

- `sector_open_mean`, `sector_high_mean`, `sector_low_mean`, `sector_close_mean`: Sector-level mean OHLC prices.
- `sector_volume_mean`: Mean sector volume.
- `sector_ret_1d`, `sector_ret_5d`, `sector_ret_20d`: Sector-level returns over different horizons.
- `sector_range`: Sector-level trading range.
- `sector_vol_20d`: Rolling 20-day sector volatility.
- `ema_12_sector`, `ema_26_sector`, `ema_50_sector`: Sector exponential moving averages.
- `macd_12_26_9_sector`, `macdh_12_26_9_sector`, `macds_12_26_9_sector`: Sector MACD components.
- `rsi_14_sector`: Sector RSI.
- `sector_bb_upper`, `sector_bb_middle`, `sector_bb_lower`: Sector Bollinger Bands.
- `market_close`: Market-level closing benchmark.
- `sector_rel_strength`: Sector relative strength.
- `sector_dispersion_1d`: Cross-sectional daily sector dispersion.

A.1.6 Meta-Target Columns

- `ret_1d_meta`: One-day return used in meta-feature construction.
- `up_1d_meta`: Binary one-day direction target used for meta-classification features.
- `has_meta_target`: Indicator showing whether a valid meta-target is available.

A.1.7 Regression Meta-Features

- `reg_pred_ret_1d_LSTM`, `reg_pred_ret_1d_BiLSTM`, `reg_pred_ret_1d_GRU`, `reg_pred_ret_1d_BiGRU`: Prior regression model predictions for 1-day return.
- `reg_abs_err_lag1_*`: Lag-1 absolute error of the corresponding regression model.
- `reg_mae_20_*`: Rolling 20-day mean absolute error of the corresponding regression model.
- `reg_rmse_20_*`: Rolling 20-day root mean squared error of the corresponding regression model.
- `reg_dir_acc_20_*`: Rolling 20-day directional accuracy of the corresponding regression model.

A.1.8 Classification Meta-Features

- `cls_prob_up_1d_LSTM`, `cls_prob_up_1d_BiLSTM`, `cls_prob_up_1d_GRU`, `cls_prob_up_1d_BiGRU`: Prior classification model probabilities of upward movement.
- `cls_brier_20_*`: Rolling 20-day Brier score of the corresponding classifier.
- `cls_logloss_20_*`: Rolling 20-day log-loss of the corresponding classifier.
- `cls_acc_20_*`: Rolling 20-day classification accuracy of the corresponding classifier.

A.1.9 Complete Column List

`date`, `open`, `high`, `low`, `close`, `adj_close`, `volume`, `ticker`,
`ema_12`, `ema_26`, `ema_50`,
`macd_12_26_9`, `macdh_12_26_9`, `macds_12_26_9`,
`rsi_14`, `stochrsik_14_14_3_3`, `stochrsid_14_14_3_3`, `attr_14`,
`bb_upper`, `bb_middle`, `bb_lower`, `obv`,
`ret_1d`, `roll_ret_1d`, `roll_ret_5d`, `roll_ret_20d`,
`sentiment`,
`emotion_anger`, `emotion_disgust`, `emotion_fear`, `emotion_joy`,
`emotion_neutral`, `emotion_sadness`, `emotion_surprise`,
`emotion_anger_pct`, `emotion_disgust_pct`, `emotion_fear_pct`,
`emotion_joy_pct`, `emotion_neutral_pct`, `emotion_sadness_pct`,
`emotion_surprise_pct`,
`positive_emotion`, `negative_emotion`, `uncertainty_emotion`,
`positive_emotion_pct`, `negative_emotion_pct`, `uncertainty_emotion_pct`,
`stance_label`, `finbert_label`, `stance_score`, `finbert_score`,
`finbert_up`, `finbert_down`, `finbert_neutral`,
`sector`, `company_name`,
`sector_open_mean`, `sector_high_mean`, `sector_low_mean`, `sector_close_mean`,
`sector_volume_mean`, `sector_ret_1d`, `sector_ret_5d`, `sector_ret_20d`,
`sector_range`, `sector_vol_20d`,
`ema_12_sector`, `ema_26_sector`, `ema_50_sector`,
`macd_12_26_9_sector`, `macdh_12_26_9_sector`, `macds_12_26_9_sector`,
`rsi_14_sector`, `sector_bb_upper`, `sector_bb_middle`, `sector_bb_lower`,
`market_close`, `sector_rel_strength`, `sector_dispersion_1d`,
`ret_1d_meta`, `up_1d_meta`, `has_meta_target`,
`reg_pred_ret_1d_LSTM`, `reg_pred_ret_1d_BiLSTM`,
`reg_pred_ret_1d_GRU`, `reg_pred_ret_1d_BiGRU`,
`reg_abs_err_lag1_LSTM`, `reg_mae_20_LSTM`,
`reg_rmse_20_LSTM`, `reg_dir_acc_20_LSTM`,

```

reg_abs_err_lag1_BiLSTM, reg_mae_20_BiLSTM,
reg_rmse_20_BiLSTM, reg_dir_acc_20_BiLSTM,
reg_abs_err_lag1_GRU, reg_mae_20_GRU, reg_rmse_20_GRU,
reg_dir_acc_20_GRU,
reg_abs_err_lag1_BiGRU, reg_mae_20_BiGRU,
reg_rmse_20_BiGRU, reg_dir_acc_20_BiGRU,
cls_prob_up_1d_LSTM, cls_prob_up_1d_BiLSTM,
cls_prob_up_1d_GRU, cls_prob_up_1d_BiGRU,
cls_brier_20_LSTM, cls_logloss_20_LSTM, cls_acc_20_LSTM,
cls_brier_20_BiLSTM, cls_logloss_20_BiLSTM, cls_acc_20_BiLSTM,
cls_brier_20_GRU, cls_logloss_20_GRU, cls_acc_20_GRU,
cls_brier_20_BiGRU, cls_logloss_20_BiGRU, cls_acc_20_BiGRU

```

A.2 Implementation Details

This appendix provides selected implementation details that support the main system described in Chapters 4 and 5. The aim is not to reproduce the full codebase, but to document the key components used for feature construction, target generation, model selection, robustness controls, and investment simulation.

A.2.1 Feature Engineering Code

This subsection presents representative code fragments for the main feature-engineering stages. These snippets illustrate how technical, textual, and reliability-aware inputs were generated before being passed into the modelling pipeline.

Technical Indicators The first stage of feature engineering augments each ticker with standard technical indicators. These include moving averages, momentum indicators, volatility measures, and volume-based signals. The listing below shows the per-ticker transformation used to compute these variables.

```

def apply_ta_indicators(df_group):
    df_group = df_group.copy()
    df_group.set_index(pd.DatetimeIndex(df_group['date']), inplace=True)

    df_group.ta.ema(length=12, append=True)
    df_group.ta.ema(length=26, append=True)
    df_group.ta.ema(length=50, append=True)
    df_group.ta.macd(fast=12, slow=26, signal=9, append=True)
    df_group.ta.rsi(length=14, append=True)
    df_group.ta.stochrsi(length=14, append=True)
    df_group.ta.atr(length=14, append=True)
    df_group.ta.obv(append=True)

    bb = ta.bbands(df_group['close'], length=20, std=2)
    df_group['BB_upper'] = bb['BBU_20_2.0']
    df_group['BB_middle'] = bb['BBM_20_2.0']
    df_group['BB_lower'] = bb['BBL_20_2.0']

    return df_group.reset_index(drop=True)

```

Listing A.1: Per-ticker technical indicator generation.

These indicators were used as part of the core numerical representation of each stock and formed the basis of the `base` feature group used in benchmarking.

Natural Language Processing Text-derived signals were generated from several transformer-based components, each designed to capture a different aspect of market-related text. The examples below illustrate how sentiment, emotion, stance, and FinBERT-style financial sentiment were extracted.

The following listing shows the sentiment-scoring setup used to assign a sentiment label to each tweet.

```
tokenizer = AutoTokenizer.from_pretrained(
    "nlptown/bert-base-multilingual-uncased-sentiment"
)
bert_model = AutoModelForSequenceClassification.from_pretrained(
    "nlptown/bert-base-multilingual-uncased-sentiment"
)

def cal_sentiment_score(text):
    inputs = tokenizer.encode(
        text, return_tensors="pt", truncation=True, max_length=512
    ).to(device)
    with torch.no_grad():
        outputs = bert_model(inputs)
    return int(torch.argmax(outputs.logits)) + 1
```

Listing A.2: Per-tweet sentiment scoring.

Emotion features were derived separately in order to capture finer-grained affective structure beyond simple polarity.

```
classifier = pipeline(
    "text-classification",
    model="j-hartmann/emotion-english-distilroberta-base",
    top_k=None,
    device=device
)

def emotion_analysis(text):
    analysis = classifier(text, top_k=None)
    return [e["score"] for e in analysis]
```

Listing A.3: Tweet-level emotion scoring.

The stance component was used to estimate whether a tweet expressed a positive or negative position relative to the asset.

```
def stance_score(text):
    result = nlp(text)
    return result[0]["label"], result[0]["score"]
```

Listing A.4: Financial stance detection.

Finally, a finance-specific transformer was used to produce domain-aligned sentiment estimates.

```
classifier = pipeline(
    "text-classification",
    model="ProsusAI/finbert",
    top_k=None,
    device=0 if torch.cuda.is_available() else -1
)
```

Listing A.5: Batch FinBERT inference.

Together, these NLP pipelines supplied the sentiment, emotion, stance, and FinBERT-based feature groups compared in the main experiments.

Meta-Feature Construction Meta-features were introduced to model recent predictive reliability. Rather than relying only on raw market and text inputs, the system also incorporated lagged model outputs and rolling error statistics. The example below shows how representative regression and classification reliability features were computed.

```
df["regression_abs_err_ret_1d"] = (
    df["ret_1d_meta"] - df["regression_pred_ret_1d"]
).abs()

df["brier"] = (df["prob_up_1d"] - df["up_1d_meta"]) ** 2
df["logloss"] = -(
    df["up_1d_meta"] * np.log(df["prob_up_1d"]) +
    (1 - df["up_1d_meta"]) * np.log(1 - df["prob_up_1d"])
)
```

Listing A.6: Regression and classification reliability features.

These features were designed to capture time-varying reliability in a leak-safe way and were used in the meta-feature experiments discussed in Chapter 6.

A.2.2 Target Construction and Horizon Design

The target-generation process was designed to support both 1-day and 5-day forecasting while preserving temporal integrity. The following listing shows how forward returns and corresponding binary direction targets were constructed for a forecast horizon H .

```
def add_horizon_targets(df, H, price_col='close'):
    df = df.sort_values(['ticker', 'date']).copy()
    df[f'ret_{H}d'] = df.groupby('ticker')[price_col].shift(-H)/df[price_col] - 1
    df[f'target_{H}d'] = (df[f'ret_{H}d'] > 0).astype(int)
    return df
```

Listing A.7: Construction of H -day return and binary targets.

This construction was used throughout the project to align the features observed at time t with outcomes realised at time $t + H$.

A.2.3 Model Hyperparameter Tuning and Selection

Hyperparameter tuning was performed with Optuna in order to search over architecture choice, sequence length, hidden-layer width, dropout, learning rate, and feature-group configuration. The listing below shows the structure of the optimisation objective.

```
def objective(trial):
    params = {
        'feature_set': trial.suggest_categorical(
            'feature_set', list(group_sets)
        ),
        'model_type': trial.suggest_categorical(
            'model_type', ['LSTM', 'BiLSTM', 'GRU', 'BiGRU']
        ),
        'sequence_length': trial.suggest_int(
            'sequence_length', 5, 20, step=5
        ),
        'hidden1': trial.suggest_categorical('hidden1', [64, 128, 256]),
        'hidden2': trial.suggest_categorical('hidden2', [32, 64, 128]),
        'dropout': trial.suggest_float('dropout', 0.0, 0.8, step=0.1),
        'learning_rate': trial.suggest_float(
            'learning_rate', 1e-6, 1e-2, log=True
        ),
    }
    }
```

Listing A.8: Optuna-based hyperparameter tuning.

Once the optimisation stage was complete, the best-performing configuration was loaded from the saved results tables, as shown below.

```
optuna_df = pd.read_csv(OPTUNA_CSV)
best_cfg = optuna_df.iloc[0]
```

Listing A.9: Loading the best benchmarking configuration.

This tuning stage allowed model selection to be driven by empirical validation performance rather than fixed manual settings.

A.2.4 Early Exit Mechanism

The system also included an early-exit mechanism for 5-day trades. This was designed to allow positions to be closed before the full holding horizon when a shorter-horizon signal indicated deterioration. Figure A.1 shows the frequency with which model-based early-exit conditions were triggered across the dataset.

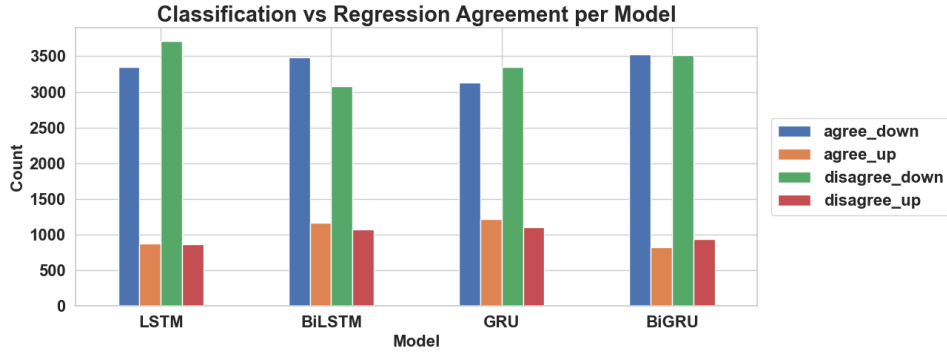


Figure A.1: Frequency of model early-exit agreements across the dataset.

This mechanism was analysed in the main text both as a risk-management tool and as a component that interacts with meta-features and horizon choice.

A.2.5 Early-Exit Example

Fig A.2 displays an example of the signals produced by a model that we use to trigger an early exit. All 4 models (LSTM, BiLSTM, GRU & BiGRU) are used to predict stock price each day in a walk-forward manner.

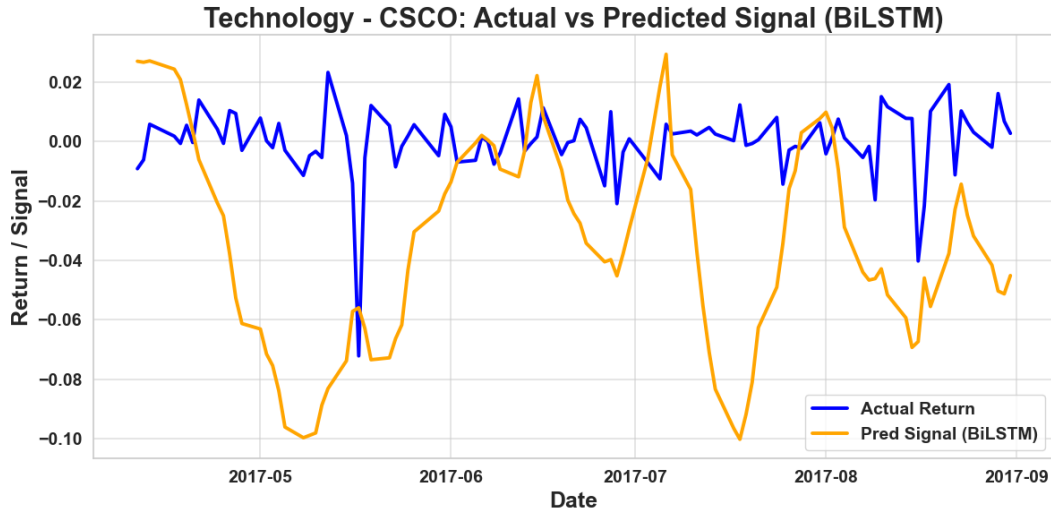


Figure A.2: An example of the regressive return signals involved in triggering early exit for CSCO produced by a BiLSTM.

A.2.6 Portfolio Construction

Portfolio construction used sector-aware ranking and a softmax-style weighting scheme to convert model scores into candidate selection probabilities. For scores $z_1, \dots, z_K$ within a sector, the selection probability of candidate j is

$$\Pr(j) = \frac{\exp(\alpha z_j)}{\sum_{m=1}^K \exp(\alpha z_m)},$$

where α controls how strongly the selection favours the highest-ranked candidate.

This formulation allows the allocator to emphasise stronger signals while still retaining diversification across multiple opportunities.

A.2.7 Portfolio Value Calculation

Portfolio value was updated by applying realised returns to the chosen set of positions. If V_t denotes portfolio value at time t , then after applying position returns,

$$V_{t+1} = V_t \left(1 + \sum_i w_i r_{i,t} \right),$$

where w_i is the capital weight allocated to asset i .

This equation defines the basic portfolio update used throughout the simulation stage.

A.2.8 Robustness Controls

Several additional controls were introduced to preserve temporal validity and avoid artificial leakage or unrealistic optimisation behaviour.

Creating Contiguous Sequences Because the price and text data were not always perfectly continuous, sequence construction was restricted to contiguous periods. The following function shows how valid training sequences were generated within such intervals.

```
def create_contiguous_sequences(data: np.ndarray, targets: np.ndarray,
                               contiguous_periods: list,
                               sequence_length: int, H: int):

    X, y = [], []

    for start_idx, end_idx in contiguous_periods:
        usable_end = end_idx - H
        period_length = usable_end - start_idx + 1

        if period_length < sequence_length + 1:
            continue

        for i in range(start_idx + sequence_length - 1, usable_end + 1):
            X.append(data[i - sequence_length + 1 : i + 1])
            y.append(targets[i + H])

    return np.array(X) if X else np.array([]), np.array(y) if y else np.array([])
```

Listing A.10: Construction of contiguous training sequences.

This helped ensure that each training sequence represented a valid chronological window rather than an artefact of missing days or discontinuities.

No-Shuffle Training To preserve the temporal order of financial data, both train/validation splitting and mini-batch loading were carried out without shuffling. The example below illustrates the relevant setup.

```

X_tr, X_val, y_tr, y_val = train_test_split(X_seq, y_seq, test_size=0.2,
                                           shuffle=False)

train_loader = DataLoader(
    train_ds,
    batch_size=train_bs,
    shuffle=False,
    drop_last=False,
    num_workers=NUM_WORKERS_TRAIN,
    pin_memory=PIN_MEMORY,
    persistent_workers=PERSISTENT_WORKERS,
)

val_loader = DataLoader(
    val_ds,
    batch_size=val_bs,
    shuffle=False,
    drop_last=False,
    num_workers=NUM_WORKERS_EVAL,
    pin_memory=PIN_MEMORY,
    persistent_workers=PERSISTENT_WORKERS,
)

```

Listing A.11: Chronological splitting and no-shuffle data loading.

This design choice was important for ensuring that validation and test results reflected genuine forward-looking evaluation.

A.2.9 Investment Simulation Parameters

Table A.1 summarises the main parameters and constraints used in the investment simulation framework. These settings were chosen to keep portfolio construction realistic by limiting leverage, concentration, and daily deployment.

Together, these controls define the trading environment used in the walk-forward simulation results reported in Chapter 6.

A.2.10 Compared Models and Feature Groups

The main benchmarking study compared four recurrent model families: LSTM, BiLSTM, GRU, and BiGRU. These models were evaluated across multiple horizon settings and a wide range of feature-group combinations spanning market, NLP, sector, and meta-feature inputs. The listing below summarises the feature-group structure used in benchmarking.

```

model_types = ["LSTM", "BiLSTM", "GRU", "BiGRU"]

horizon_steps = [1, 5]

feature_columns = [
    'open', 'high', 'low', 'close', 'volume',
    'roll_ret_1d', 'roll_ret_5d', 'roll_ret_20d',
    'ema_12', 'ema_26', 'ema_50', 'macd_12_26_9', 'macdh_12_26_9',
    'macds_12_26_9', 'rsi_14', 'stochrsik_14_14_3_3', 'stochrsid_14_14_3_3',
    'attr_14', 'bb_upper', 'bb_middle', 'bb_lower', 'obv',

```

Table A.1: Key parameters used in the investment simulation framework.

Parameter	Description	Value
Initial Capital	Starting portfolio capital	\$100,000
Transaction Cost	Proportional cost applied per trade	0.1%
Kelly Fraction	Fractional Kelly scaling factor	0.25
Kelly Trade Cap	Maximum raw Kelly allocation per trade	0.50
MPT Window	Lookback window for covariance estimation	60 days
MPT Risk Aversion (λ)	Mean-variance risk penalty parameter	1.0
MPT Ridge Regularisation	Stabilisation term for covariance matrix	10^{-4}
MPT Max Weight	Maximum asset weight under MPT allocation	30%
MPT Long Only	Restrict portfolio weights to non-negative values	True
MPT Budget Fraction	Fraction of capital allocated through MPT	30%
Retraining Interval	Walk-forward model refitting frequency	200 days
Maximum Day Gap	Maximum allowed trading-day gap in data	5 days
Max Total Utilisation	Maximum deployed portfolio capital	70%
Max New Daily Utilisation	Maximum capital deployed per day	30%
Max Ticker Utilisation	Maximum exposure to a single ticker	50%
Max Sector Utilisation	Maximum exposure to a single sector	50%
Max Position Size	Maximum capital allocated per trade	10%
Max Positions per Ticker	Maximum simultaneous trades per ticker	3
Minimum Trade Size	Minimum trade value threshold	\$100
Drawdown Throttle	Utilisation reduction during drawdowns	(8%, 0.60), (15%, 0.35)

```

]

sentiment_columns = ['sentiment']

emotion_columns = [
    'emotion_anger', 'emotion_disgust', 'emotion_fear', 'emotion_joy',
    'emotion_neutral', 'emotion_sadness', 'emotion_surprise',
    'emotion_anger_pct', 'emotion_disgust_pct', 'emotion_fear_pct',
    'emotion_joy_pct', 'emotion_neutral_pct', 'emotion_sadness_pct',
    'emotion_surprise_pct',
]

unified_emotion_columns = [
    'positive_emotion', 'negative_emotion', 'uncertainty_emotion',
    'positive_emotion_pct', 'negative_emotion_pct', 'uncertainty_emotion_pct',
]

stance_columns = ['stance_label', 'stance_score']

finbert_columns = [
    'finbert_label', 'finbert_score', 'finbert_up', 'finbert_down',
    'finbert_neutral',
]

sector_columns = [
    'sector_open_mean', 'sector_high_mean', 'sector_low_mean', 'sector_close_mean',
    'sector_volume_mean', 'sector_ret_1d', 'sector_ret_5d', 'sector_ret_20d',
    'sector_range', 'sector_vol_20d', 'ema_12_sector', 'ema_26_sector',
    'ema_50_sector', 'macd_12_26_9_sector', 'macdh_12_26_9_sector',
    'macds_12_26_9_sector', 'rsi_14_sector', 'sector_bb_upper',
    'sector_bb_middle', 'sector_bb_lower', 'market_close',
    'sector_rel_strength', 'sector_dispersion_1d'
]

```

```

]

meta_regression_columns = [
    'reg_pred_ret_1d_LSTM', 'reg_pred_ret_1d_BiLSTM',
    'reg_pred_ret_1d_GRU', 'reg_pred_ret_1d_BiGRU',
]

meta_classification_columns = [
    'cls_prob_up_1d_LSTM', 'cls_prob_up_1d_BiLSTM',
    'cls_prob_up_1d_GRU', 'cls_prob_up_1d_BiGRU',
]

meta_regression_err_columns = [
    'reg_abs_err_lag1_LSTM', 'reg_mae_20_LSTM', 'reg_rmse_20_LSTM',
    'reg_dir_acc_20_LSTM',
    'reg_abs_err_lag1_BiLSTM', 'reg_mae_20_BiLSTM', 'reg_rmse_20_BiLSTM',
    'reg_dir_acc_20_BiLSTM',
    'reg_abs_err_lag1_GRU', 'reg_mae_20_GRU', 'reg_rmse_20_GRU',
    'reg_dir_acc_20_GRU',
    'reg_abs_err_lag1_BiGRU', 'reg_mae_20_BiGRU', 'reg_rmse_20_BiGRU',
    'reg_dir_acc_20_BiGRU',
]

meta_classification_err_columns = [
    'cls_brier_20_LSTM', 'cls_logloss_20_LSTM', 'cls_acc_20_LSTM',
    'cls_brier_20_BiLSTM', 'cls_logloss_20_BiLSTM', 'cls_acc_20_BiLSTM',
    'cls_brier_20_GRU', 'cls_logloss_20_GRU', 'cls_acc_20_GRU',
    'cls_brier_20_BiGRU', 'cls_logloss_20_BiGRU', 'cls_acc_20_BiGRU',
]

feature_sets = {
    'base': feature_columns,
    'sentiment': feature_columns + sentiment_columns,
    'emotion': feature_columns + emotion_columns,
    'unified_emotion': feature_columns + unified_emotion_columns,
    'finbert': feature_columns + finbert_columns,
    'all_nlp': feature_columns + sentiment_columns + emotion_columns +
        unified_emotion_columns + stance_columns + finbert_columns,
    'sector': feature_columns + sector_columns,
    'sector_sentiment': feature_columns + sector_columns + sentiment_columns,
    'sector_emotion': feature_columns + sector_columns + emotion_columns,
    'sector_unified_emotion': feature_columns + sector_columns +
        unified_emotion_columns,
    'sector_finbert': feature_columns + sector_columns + finbert_columns,
    'sector_all_nlp': feature_columns + sector_columns + sentiment_columns +
        emotion_columns + unified_emotion_columns + stance_columns +
        finbert_columns,
    'meta': feature_columns + meta_regression_columns +
        meta_classification_columns,
    'meta_err': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns,
    'meta_sentiment': feature_columns + meta_regression_columns +
        meta_classification_columns + sentiment_columns,
    'meta_stance': feature_columns + meta_regression_columns +
        meta_classification_columns + stance_columns,
    'meta_emotion': feature_columns + meta_regression_columns +

```

```

        meta_classification_columns + emotion_columns,
'meta_unified_emotion': feature_columns + meta_regression_columns +
        meta_classification_columns + unified_emotion_columns,
'meta_finbert': feature_columns + meta_regression_columns +
        meta_classification_columns + finbert_columns,
'meta_all_nlp': feature_columns + meta_regression_columns +
        meta_classification_columns + sentiment_columns + emotion_columns +
        unified_emotion_columns + stance_columns + finbert_columns,
'meta_sector': feature_columns + meta_regression_columns +
        meta_classification_columns + sector_columns,
'meta_sector_sentiment': feature_columns + meta_regression_columns +
        meta_classification_columns + sector_columns + sentiment_columns,
'meta_sector_stance': feature_columns + meta_regression_columns +
        meta_classification_columns + sector_columns + stance_columns,
'meta_sector_emotion': feature_columns + meta_regression_columns +
        meta_classification_columns + sector_columns + emotion_columns,
'meta_sector_unified_emotion': feature_columns + meta_regression_columns +
        meta_classification_columns + sector_columns + unified_emotion_columns,
'meta_sector_finbert': feature_columns + meta_regression_columns +
        meta_classification_columns + sector_columns + finbert_columns,
'meta_sector_all_nlp': feature_columns + meta_regression_columns +
        meta_classification_columns + sector_columns + sentiment_columns +
        emotion_columns + unified_emotion_columns + stance_columns +
        finbert_columns,
'meta_err_sentiment': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + sentiment_columns,
'meta_err_stance': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + stance_columns,
'meta_err_emotion': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + emotion_columns,
'meta_err_unified_emotion': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + unified_emotion_columns,
'meta_err_finbert': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + finbert_columns,
'meta_err_all_nlp': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + sentiment_columns + emotion_columns +
        unified_emotion_columns + stance_columns + finbert_columns,
'meta_err_sector': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + sector_columns,
'meta_err_sector_sentiment': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + sector_columns + sentiment_columns,
'meta_err_sector_stance': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + sector_columns + stance_columns,
'meta_err_sector_emotion': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + sector_columns + emotion_columns,
'meta_err_sector_unified_emotion': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +

```

```

        meta_classification_err_columns + sector_columns + unified_emotion_columns,
    'meta_err_sector_finbert': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + sector_columns + finbert_columns,
    'meta_err_sector_all_nlp': feature_columns + meta_regression_columns +
        meta_classification_columns + meta_regression_err_columns +
        meta_classification_err_columns + sector_columns + sentiment_columns +
        emotion_columns + unified_emotion_columns + stance_columns +
        finbert_columns,
}

```

Listing A.12: Model families, horizons, and feature-group structure used in benchmarking.

This structure illustrates the breadth of the design space explored during benchmarking. The comparison of these feature groups was central to determining which representations were most useful under different task formulations and forecast horizons.

A.3 Training Diagnostics

This appendix presents the full set of training and validation loss curves for the main model families and feature-set combinations evaluated in the study. These plots complement the summary discussion in the main text by showing the detailed optimisation behaviour under each task formulation. In particular, they are useful for comparing convergence stability, relative training speed, and the extent to which meta-features alter validation behaviour.

A.3.1 Binary Classification Loss Curves

The binary classification loss curves are shown separately for the meta-feature and non-meta settings. This separation makes it easier to compare the effect of reliability-aware inputs on convergence and validation stability.

Figure A.3 shows the binary classification training behaviour when meta-features are included. In the main body, these curves are summarised through aggregate loss statistics; here the full visual comparison is retained for completeness.

Figure A.4 provides the corresponding binary classification curves without meta-features, allowing a direct comparison with the meta-feature case above.

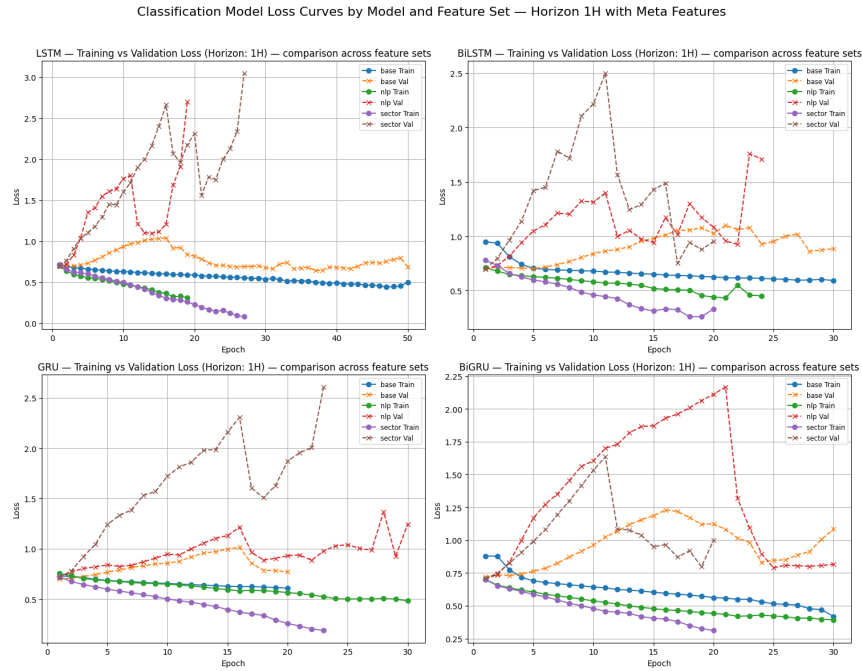


Figure A.3: Binary classification training and validation loss curves with meta-features.

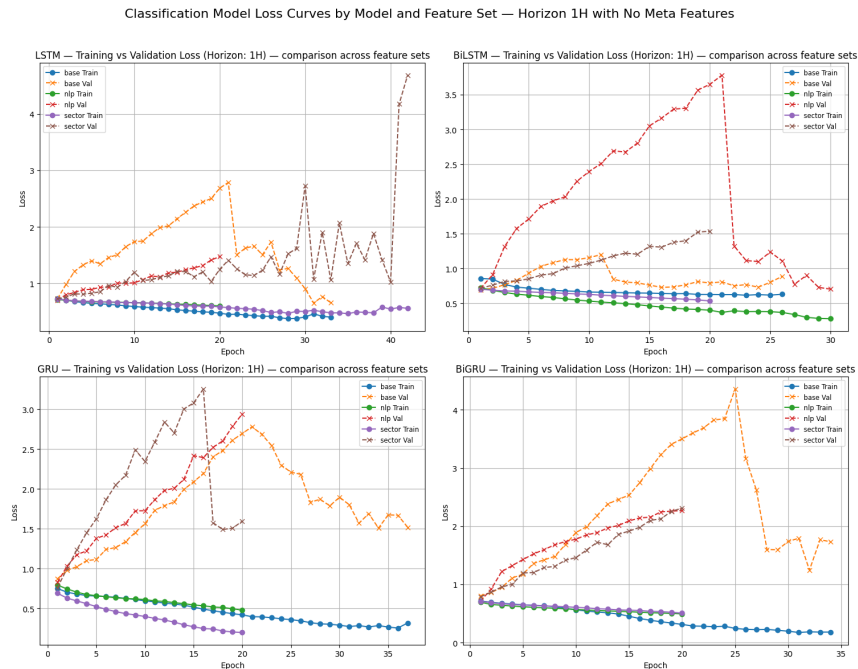


Figure A.4: Binary classification training and validation loss curves without meta-features.

A.3.2 Ordinal Multi-Class Loss Curves

The ordinal multi-class setting is more challenging than binary classification because the model must recover ordered return categories rather than simple up/down direction. The corresponding loss curves are therefore useful for assessing whether training remains stable under this harder formulation.

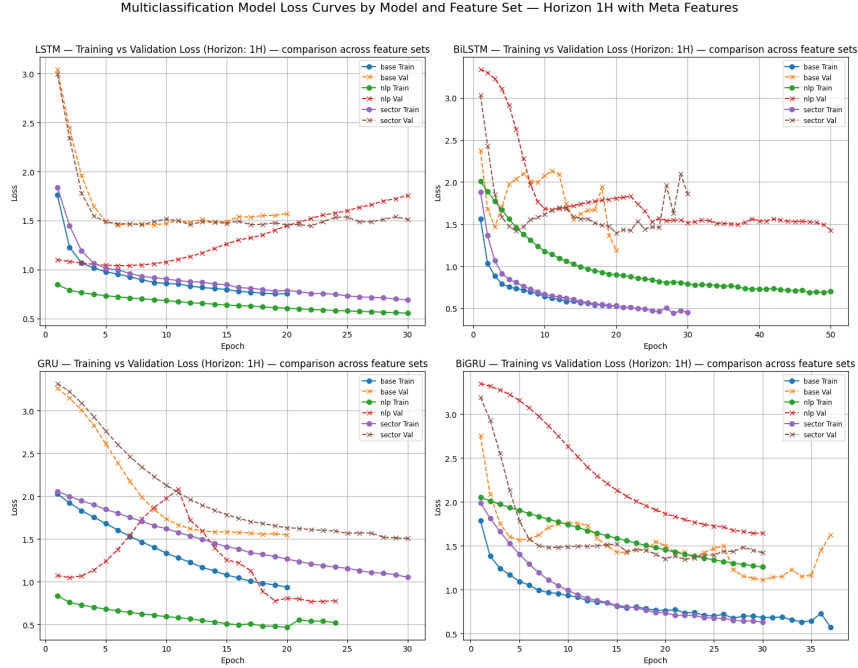


Figure A.5: Ordinal multi-class training and validation loss curves with meta-features.

Figure A.5 shows the ordinal training and validation curves when meta-features are incorporated. These curves help illustrate how reliability-aware inputs affect optimisation in the multi-class setting.

Figure A.6 reports the corresponding ordinal curves without meta-features, providing the non-meta baseline for comparison.

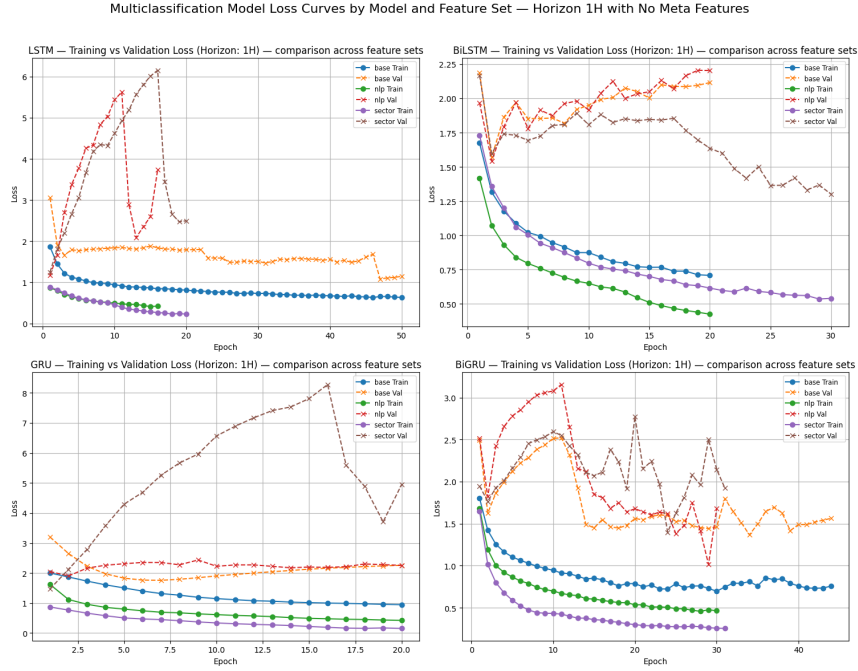


Figure A.6: Ordinal multi-class training and validation loss curves without meta-features.

A.3.3 Regression Loss Curves

The regression curves are presented separately below. As discussed in the main text, regression proved less useful than the classification-based formulations for downstream decision-making, but the training diagnostics remain informative for understanding relative optimisation behaviour.

Figure A.7 shows the regression loss curves with meta-features, highlighting the optimisation behaviour once reliability-aware inputs are introduced.

Figure A.8 provides the corresponding regression curves without meta-features.

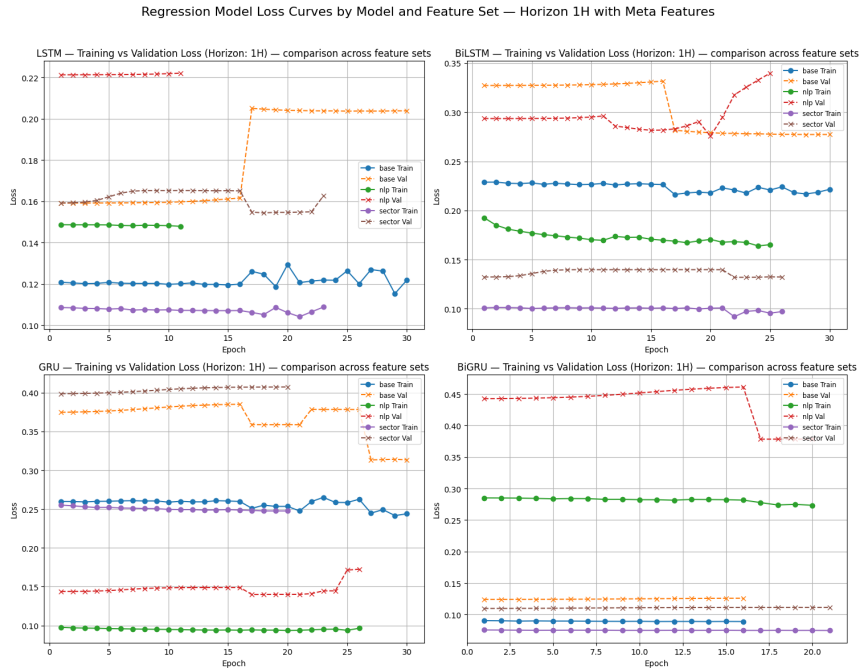


Figure A.7: Regression training and validation loss curves with meta-features.

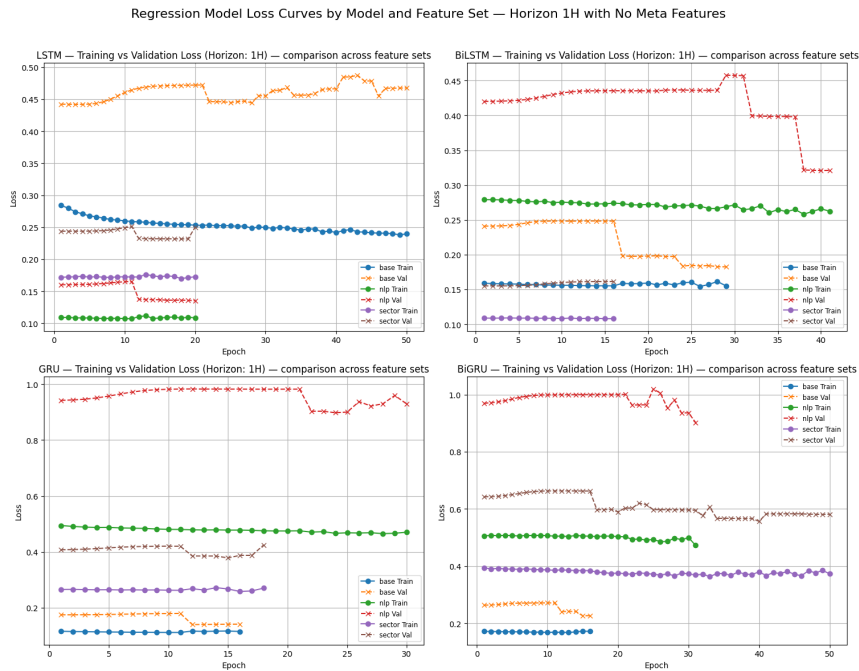


Figure A.8: Regression training and validation loss curves without meta-features.

A.4 Full Predictive Benchmark Results

This appendix reports the fuller benchmark tables that support the headline results discussed in Chapter 6. The purpose of these tables is to show the broader ranking structure behind the selected configurations used in the main analysis. For readability, the tables are grouped first by 1-day predictive results and then by 5-day variable-horizon results.

A.4.1 1-Day Binary Classification

Table A.2 reports the strongest 1-day binary classification configurations without meta-features. The ranking is ordered by overall predictive quality, with particular emphasis on Matthews Correlation Coefficient (MCC), which was used as the main selection metric for the investment stage.

Table A.2: Top binary classification results for the 1-day prediction horizon without meta-features.

Model	Feature Set	Dir Acc	MCC	F1	Precision	Recall
BiGRU	sentiment	0.525	0.086	0.331	0.476	0.360
BiLSTM	sentiment	0.525	0.077	0.363	0.488	0.404
BiGRU	sector	0.528	0.069	0.389	0.495	0.435
GRU	sentiment	0.521	0.069	0.379	0.528	0.423
LSTM	base	0.516	0.069	0.405	0.508	0.486
BiLSTM	sector_sentiment	0.519	0.064	0.307	0.505	0.325
GRU	base	0.514	0.063	0.358	0.499	0.415
LSTM	sector_unified_emotion	0.514	0.060	0.373	0.483	0.412
LSTM	sentiment	0.522	0.060	0.425	0.517	0.490
GRU	sector_sentiment	0.513	0.059	0.356	0.459	0.414
BiLSTM	base	0.518	0.057	0.357	0.490	0.408

The table shows that sentiment-enhanced variants appear frequently among the strongest binary models, although competitive non-text and sector-enhanced configurations are also present.

A.4.2 1-Day Regression

Table A.3 gives the corresponding 1-day regression benchmark results. These are ordered by MAE, since the regression task was primarily evaluated in terms of return-magnitude error.

As in the main text, the spread in MAE is relatively narrow, which reflects the difficulty of obtaining large improvements in short-horizon return-magnitude prediction.

Table A.3: Regression results for the 1-day prediction horizon, ordered by MAE (lower is better).

Model	Feature Set	Dir Acc	MAE	MSE	MCC	F1	Recall
BiLSTM	emotion	0.510	0.011571	0.000270	0.002	0.314	0.433
LSTM	sector_sentiment	0.496	0.011589	0.000272	-0.002	0.344	0.488
BiLSTM	sector_sentiment	0.496	0.011590	0.000271	-0.012	0.386	0.548
BiGRU	base	0.494	0.011598	0.000272	-0.005	0.380	0.520
GRU	sector_sentiment	0.501	0.011609	0.000272	0.002	0.425	0.566
LSTM	emotion	0.507	0.011616	0.000273	0.002	0.412	0.581
GRU	base	0.502	0.011626	0.000272	0.002	0.364	0.496
BiGRU	all_nlp	0.501	0.011627	0.000273	-0.013	0.384	0.525
BiGRU	sector_sentiment	0.499	0.011636	0.000273	0.002	0.402	0.539
GRU	all_nlp	0.504	0.011664	0.000274	0.022	0.370	0.464
LSTM	base	0.505	0.011741	0.000280	0.022	0.408	0.550

A.4.3 1-Day Ordinal Multi-Class

Table A.4 reports the 1-day ordinal multi-class benchmark results. These results are useful for understanding how the ranking changes when the task shifts from simple direction prediction to ordered return-bucket prediction.

Table A.4: Ordinal multi-class results for the 1-day prediction horizon.

Model	Feature Set	Micro Acc	Macro F1	MCC	Dir Acc	F1	Recall
BiGRU	base	0.152	0.104	0.078	0.516	0.362	0.400
BiGRU	sector_emotion	0.178	0.119	0.075	0.519	0.370	0.389
LSTM	sector_emotion	0.186	0.120	0.071	0.522	0.441	0.496
BiGRU	sentiment	0.153	0.107	0.071	0.510	0.361	0.388
GRU	sector_emotion	0.191	0.124	0.070	0.522	0.424	0.462
BiLSTM	sector_unified_emotion	0.167	0.116	0.065	0.513	0.397	0.426
GRU	base	0.156	0.106	0.063	0.517	0.374	0.397
GRU	all_nlp	0.165	0.118	0.059	0.515	0.407	0.453
LSTM	sentiment	0.181	0.116	0.055	0.510	0.411	0.465
LSTM	base	0.169	0.117	0.054	0.515	0.369	0.386
BiLSTM	emotion	0.149	0.113	0.046	0.503	0.398	0.435
BiLSTM	base	0.154	0.104	0.044	0.501	0.350	0.393

Compared with the binary case, the absolute classification metrics are lower, which is expected given the greater difficulty of exact ordered-bucket prediction.

A.5 Meta-Feature Experimental Results

This appendix reports the full set of 1-day results obtained using reliability-aware meta-features. Whereas the main text focuses on the strongest configurations, the tables below provide a broader view of model behaviour across the meta-feature design space.

A.5.1 Binary Classification with Meta-Features

Table A.5 summarises the strongest binary classification configurations with meta-features. The reported values are means with standard deviations across companies.

Table A.5: Binary classification performance with meta-features (1-day horizon).

Model	Feature Set	Dir Acc	MCC	F1	Recall
GRU	meta_sector_finbert	0.516 $\pm$ 0.063	0.081 $\pm$ 0.128	0.358 $\pm$ 0.242	0.400 $\pm$ 0.360
BiGRU	meta	0.526 $\pm$ 0.058	0.076 $\pm$ 0.116	0.403 $\pm$ 0.237	0.458 $\pm$ 0.367
LSTM	meta	0.513 $\pm$ 0.060	0.075 $\pm$ 0.119	0.341 $\pm$ 0.245	0.387 $\pm$ 0.374
GRU	meta	0.512 $\pm$ 0.062	0.072 $\pm$ 0.114	0.399 $\pm$ 0.246	0.478 $\pm$ 0.383
BiLSTM	meta_sector_finbert	0.509 $\pm$ 0.055	0.067 $\pm$ 0.103	0.360 $\pm$ 0.238	0.409 $\pm$ 0.364
BiLSTM	meta_stance	0.520 $\pm$ 0.053	0.065 $\pm$ 0.103	0.422 $\pm$ 0.218	0.480 $\pm$ 0.354
BiGRU	meta_sector_unified_emotion	0.513 $\pm$ 0.060	0.061 $\pm$ 0.112	0.294 $\pm$ 0.254	0.330 $\pm$ 0.371
BiGRU	meta_finbert	0.510 $\pm$ 0.060	0.055 $\pm$ 0.115	0.341 $\pm$ 0.257	0.398 $\pm$ 0.384
BiLSTM	meta	0.506 $\pm$ 0.053	0.054 $\pm$ 0.111	0.372 $\pm$ 0.257	0.449 $\pm$ 0.391
LSTM	meta_stance	0.510 $\pm$ 0.058	0.053 $\pm$ 0.114	0.369 $\pm$ 0.249	0.434 $\pm$ 0.385
GRU	meta_emotion	0.514 $\pm$ 0.058	0.051 $\pm$ 0.112	0.399 $\pm$ 0.233	0.454 $\pm$ 0.355
LSTM	meta_err_sector_stance	0.507 $\pm$ 0.052	0.028 $\pm$ 0.100	0.391 $\pm$ 0.253	0.464 $\pm$ 0.375

The table shows that the strongest binary meta-feature configurations remain competitive with the best non-meta baseline, while also illustrating that performance varies substantially across feature combinations.

A.5.2 Regression with Meta-Features

Table A.6 reports the corresponding regression results. Here the strongest configurations are very close in MAE, which again reflects the narrow spread observed in the main text.

Table A.6: Regression performance with meta-features (1-day horizon).

Model	Feature Set	Dir Acc	MAE	MSE
BiGRU	meta_sector	0.509 $\pm$ 0.054	0.01135	0.000242
BiGRU	meta_err	0.511 $\pm$ 0.052	0.01132	0.000239
GRU	meta_err_sector_stance	0.504 $\pm$ 0.061	0.01132	0.000240
GRU	meta_unified_emotion	0.502 $\pm$ 0.053	0.01134	0.000242
GRU	meta	0.501 $\pm$ 0.060	0.01132	0.000240
BiLSTM	meta_err_sector_sentiment	0.505 $\pm$ 0.049	0.01130	0.000239
BiLSTM	meta	0.501 $\pm$ 0.053	0.01134	0.000241
LSTM	meta_sector	0.499 $\pm$ 0.051	0.01134	0.000240
LSTM	meta_err_emotion	0.500 $\pm$ 0.051	0.01132	0.000240
BiLSTM	meta_err_unified_emotion	0.507 $\pm$ 0.050	0.01130	0.000240
BiGRU	meta_emotion	0.499 $\pm$ 0.056	0.01134	0.000241
LSTM	meta	0.501 $\pm$ 0.048	0.01130	0.000239

A.5.3 Ordinal Multi-Class with Meta-Features

Table A.7 provides the ordinal multi-class meta-feature results. These are useful for understanding the broader ranking behind the best ordinal meta-feature model discussed in the main text.

Table A.7: Ordinal multi-class performance with meta-features (1-day horizon).

Model	Feature Set	Micro Acc	Macro F1	MCC
BiLSTM	meta_sector_sentiment	0.163 ± 0.069	0.111 ± 0.053	0.083 ± 0.100
BiGRU	meta_unified_emotion	0.159 ± 0.076	0.101 ± 0.051	0.081 ± 0.108
BiGRU	meta_sector_sentiment	0.165 ± 0.069	0.110 ± 0.050	0.071 ± 0.115
GRU	meta_sector_sentiment	0.151 ± 0.075	0.097 ± 0.050	0.070 ± 0.101
LSTM	meta_sector_sentiment	0.174 ± 0.068	0.117 ± 0.047	0.069 ± 0.113
BiLSTM	meta	0.157 ± 0.069	0.106 ± 0.055	0.064 ± 0.119
LSTM	meta	0.183 ± 0.065	0.123 ± 0.051	0.062 ± 0.108
BiGRU	meta	0.156 ± 0.074	0.107 ± 0.054	0.060 ± 0.139
BiLSTM	meta_unified_emotion	0.159 ± 0.069	0.112 ± 0.055	0.056 ± 0.115
GRU	meta	0.179 ± 0.069	0.122 ± 0.053	0.034 ± 0.113
GRU	meta_stance	0.165 ± 0.083	0.089 ± 0.053	0.024 ± 0.102
LSTM	meta_stance	0.172 ± 0.072	0.098 ± 0.054	0.022 ± 0.088

A.6 Variable-Horizon Results

This appendix reports the full 5-day horizon benchmark results for the binary classification, regression, and ordinal multi-class tasks, both with and without meta-features. These tables complement the shorter horizon-comparison discussion in Chapter 6 by showing the broader model rankings at the longer forecast window.

A.6.1 Binary Classification: No Meta-Features

Table A.8 shows the 5-day binary classification results without meta-features. As in the main text, several of the strongest models combine recurrent architectures with sentiment or sector-enhanced inputs.

Table A.8: Binary classification results at the 5-day horizon without meta-features.

Model	Feature Set	Dir Acc	MCC	F1	Precision	Recall
GRU	sector_sentiment	0.529	0.152	0.388	0.608	0.427
BiLSTM	base	0.531	0.149	0.412	0.599	0.471
BiGRU	sentiment	0.538	0.144	0.406	0.591	0.440
BiLSTM	sector_finbert	0.524	0.133	0.351	0.616	0.365
BiGRU	base	0.518	0.125	0.373	0.574	0.428
BiLSTM	finbert	0.521	0.125	0.349	0.600	0.369
GRU	base	0.526	0.124	0.411	0.545	0.465
LSTM	finbert	0.515	0.124	0.359	0.552	0.419
LSTM	sector_sentiment	0.544	0.120	0.371	0.593	0.372
BiGRU	sector_finbert	0.523	0.119	0.422	0.579	0.472
GRU	emotion	0.519	0.116	0.364	0.527	0.410
LSTM	base	0.526	0.096	0.392	0.557	0.415

A.6.2 Binary Classification: With Meta-Features

Table A.9 reports the corresponding 5-day binary results with meta-features. These results are especially important because the strongest binary investment outcomes in the main body emerged from this broader family of models.

Table A.9: Binary classification results at the 5-day horizon with meta-features.

Model	Feature Set	Dir Acc	MCC	F1	Precision	Recall
BiGRU	meta	0.531	0.145	0.349	0.624	0.367
BiLSTM	meta	0.530	0.143	0.365	0.603	0.390
GRU	meta	0.534	0.132	0.398	0.595	0.456
LSTM	meta_sentiment	0.521	0.121	0.369	0.585	0.415
BiGRU	meta_sentiment	0.515	0.116	0.372	0.630	0.406
LSTM	meta	0.524	0.115	0.361	0.580	0.386
GRU	meta_err_finbert	0.527	0.115	0.357	0.585	0.374
GRU	meta_sector	0.522	0.113	0.424	0.577	0.490
LSTM	meta_sector_finbert	0.529	0.113	0.401	0.601	0.425
BiLSTM	meta_sector	0.508	0.109	0.311	0.604	0.332
BiLSTM	meta_err_unified_emotion	0.528	0.097	0.330	0.553	0.348
BiGRU	meta_sector_sentiment	0.516	0.097	0.336	0.550	0.367

A.6.3 Regression: No Meta-Features

Table A.10 shows the 5-day regression results without meta-features. The narrow spread in MAE persists at this horizon, although magnitude error is noticeably larger than in the 1-day setting.

Table A.10: Regression results at the 5-day horizon without meta-features.

Model	Feature Set	Dir Acc	MCC	MAE	F1
LSTM	sector_finbert	0.533	0.055	0.025984	0.415
BiGRU	sentiment	0.526	0.035	0.026335	0.484
BiLSTM	sector_all_nlp	0.517	0.028	0.026248	0.383
GRU	all_nlp	0.515	0.027	0.026699	0.465
LSTM	base	0.526	0.025	0.026202	0.434
BiGRU	sector_sentiment	0.521	0.023	0.026253	0.419
LSTM	finbert	0.525	0.017	0.026291	0.406
GRU	base	0.529	0.012	0.026280	0.441
BiLSTM	emotion	0.511	0.006	0.026486	0.393
BiLSTM	base	0.534	0.005	0.026103	0.465
BiGRU	base	0.522	0.002	0.026219	0.463
GRU	sector_emotion	0.507	0.001	0.026552	0.391

A.6.4 Regression: With Meta-Features

Table A.11 provides the corresponding 5-day regression results with meta-features. As in the 1-day case, meta-feature variants remain tightly grouped in MAE.

Table A.11: Regression results at the 5-day horizon with meta-features.

Model	Feature Set	Dir Acc	MCC	MAE	F1
BiLSTM	meta_sector_emotion	0.526	0.042	0.025322	0.387
BiLSTM	meta_err_unified_emotion	0.534	0.039	0.025395	0.474
LSTM	meta_err_sector_emotion	0.533	0.038	0.025301	0.474
BiGRU	meta	0.526	0.029	0.025447	0.409
BiGRU	meta_sector_emotion	0.522	0.028	0.025453	0.492
GRU	meta_all_nlp	0.532	0.026	0.025237	0.415
LSTM	meta_err	0.526	0.016	0.025225	0.429
LSTM	meta_stance	0.517	0.008	0.025442	0.424
GRU	meta	0.500	0.005	0.025604	0.388
BiGRU	meta_all_nlp	0.514	0.003	0.025291	0.417
BiLSTM	meta_err	0.521	-0.005	0.025303	0.386
GRU	meta_err_sector_unified_emotion	0.523	-0.005	0.025585	0.462

A.6.5 Ordinal Multi-Class: No Meta-Features

Table A.12 reports the 5-day ordinal multi-class results without meta-features. These results support the main finding that longer-horizon ordinal classification is materially stronger than its 1-day counterpart.

Table A.12: Ordinal multi-class results at the 5-day horizon without meta-features.

Model	Feature Set	Micro Acc	Macro F1	MCC	Dir Acc	F1
BiGRU	sector	0.208	0.127	0.145	0.533	0.400
BiLSTM	sector_finbert	0.217	0.134	0.137	0.541	0.459
BiLSTM	base	0.214	0.129	0.128	0.520	0.428
LSTM	base	0.209	0.124	0.120	0.513	0.379
GRU	base	0.197	0.116	0.119	0.508	0.423
BiGRU	emotion	0.208	0.129	0.115	0.527	0.413
LSTM	sector_finbert	0.189	0.116	0.115	0.520	0.412
BiGRU	base	0.202	0.120	0.113	0.511	0.401
GRU	sector_all_nlp	0.222	0.122	0.110	0.511	0.536
LSTM	sentiment	0.206	0.123	0.109	0.522	0.443
BiLSTM	emotion	0.212	0.122	0.107	0.513	0.436
GRU	emotion	0.214	0.125	0.097	0.512	0.480

A.6.6 Ordinal Multi-Class: With Meta-Features

Finally, Table A.13 reports the 5-day ordinal results with meta-features. These results provide the full ranking behind the strongest ordinal meta-feature configuration discussed in the main body.

Table A.13: Ordinal multi-class results at the 5-day horizon with meta-features.

Model	Feature Set	Micro Acc	Macro F1	MCC	Dir Acc	F1
GRU	meta_emotion	0.213	0.128	0.126	0.530	0.377
BiLSTM	meta	0.216	0.118	0.109	0.512	0.546
BiGRU	meta	0.213	0.112	0.108	0.517	0.536
LSTM	meta_all_nlp	0.224	0.112	0.105	0.525	0.545
BiGRU	meta_emotion	0.211	0.114	0.102	0.514	0.324
LSTM	meta	0.221	0.114	0.099	0.516	0.536
BiLSTM	meta_sector	0.216	0.117	0.097	0.521	0.577
GRU	meta	0.216	0.117	0.092	0.526	0.435
GRU	meta_sector_stance	0.224	0.115	0.089	0.516	0.565
BiGRU	meta_sector	0.216	0.111	0.085	0.517	0.570
BiLSTM	meta_emotion	0.214	0.099	0.071	0.510	0.574
LSTM	meta_sector	0.224	0.116	0.067	0.509	0.571

A.7 Investment Simulation Supporting Results

This appendix provides the additional material underlying the investment simulation results presented in Chapter 6. It first identifies the benchmark models selected for the simulation stage, then reports the full binary and ordinal portfolio results, and finally presents the cumulative return curves for each setting.

A.7.1 Selected Binary Models for Investment Simulation

The binary investment simulations were based on the strongest 1-day and 5-day configurations from the benchmarking stage, selected primarily by Matthews Correlation Coefficient (MCC). Table A.14 summarises the models that were carried forward into the portfolio simulation framework.

Table A.14: Selected binary models carried forward to the investment simulation stage.

Setting	Horizon	Model / Feature Set	Directional Accuracy	MCC
No Meta	1-day	BiGRU / sentiment	0.525	0.086
No Meta	5-day	GRU / sector_sentiment	0.529	0.152
Meta	1-day	GRU / meta_sector_finbert	0.516	0.081
Meta	5-day	BiGRU / meta	0.531	0.145

These selected models represent the forecast inputs used for the binary investment experiments discussed in the main text.

A.7.2 Selected Ordinal Models for Investment Simulation

The ordinal multi-class simulations were built from the strongest ordinal models under the same benchmark-selection logic. Table A.15 reports the corresponding 1-day and 5-day models used in the simulation stage.

These models form the basis of the ordinal investment simulations reported below.

Table A.15: Selected ordinal multi-class models carried forward to the investment simulation stage.

Setting	Horizon	Model / Feature Set	Directional Accuracy	MCC	F1
No Meta	1-day	BiGRU / base	0.516	0.078	0.152
No Meta	5-day	BiGRU / sector	0.533	0.145	0.208
Meta	1-day	BiLSTM / meta_sector_sentiment	0.525	0.083	0.163
Meta	5-day	GRU / meta_emotion	0.530	0.126	0.213

A.7.3 Full Binary Investment Simulation Results

Table A.16 reports the full binary investment simulation results, combining both the no-meta and meta-feature settings across 1-day and 5-day horizons. This table provides the full ranking behind the compact summary results presented in Chapter 6.

Table A.16: Binary investment simulation results with and without meta-features.

Setting	Mode	Horizon	Early Exit	Start Equity	End Equity	ROI (%)	Sharpe	Volatility
No Meta	Hybrid Kelly-MPT	5	OFF	100000	103939	3.94	2.21	0.044
No Meta	MPT	5	OFF	100000	103031	3.03	1.74	0.044
No Meta	Kelly	5	OFF	100000	101982	1.98	1.36	0.037
No Meta	Kelly	1	OFF	100000	100968	0.97	1.04	0.024
No Meta	MPT	1	OFF	100000	100013	0.01	0.03	0.031
No Meta	Hybrid Kelly-MPT	1	OFF	100000	99982	-0.02	-0.00	0.027
Meta	MPT	5	OFF	100000	112465	12.47	4.34	0.069
Meta	Hybrid Kelly-MPT	5	OFF	100000	108665	8.66	4.26	0.049
Meta	Kelly	5	OFF	100000	102763	2.76	1.79	0.039
Meta	Kelly	1	OFF	100000	101051	1.05	1.26	0.021
Meta	Hybrid Kelly-MPT	1	OFF	100000	100249	0.25	0.29	0.023
Meta	MPT	1	OFF	100000	99981	-0.02	0.00	0.034

The table highlights the strong separation between 1-day and 5-day performance in the binary setting, as well as the substantial improvement associated with meta-features in the strongest 5-day strategies.

A.7.4 Full Ordinal Multi-Class Investment Simulation Results

Table A.17 reports the full ordinal multi-class investment simulation results. As with the binary system, the table combines meta and non-meta settings so that horizon effects and allocation behaviour can be compared directly.

Table A.17: Ordinal multi-class investment simulation results with and without meta-features.

Setting	Mode	Horizon	Early Exit	Start Equity	End Equity	ROI (%)	Sharpe	Volatility
No Meta	MPT	5	OFF	100000	110561	10.56	4.59	0.055
No Meta	Hybrid Kelly-MPT	5	OFF	100000	105233	5.23	5.06	0.025
No Meta	Kelly	5	OFF	100000	103345	3.35	3.81	0.022
No Meta	MPT	1	OFF	100000	99730	-0.27	-0.19	0.033
No Meta	Kelly	1	OFF	100000	99054	-0.95	-0.67	0.035
No Meta	Hybrid Kelly-MPT	1	OFF	100000	99039	-0.96	-0.68	0.035
Meta	Kelly	5	OFF	100000	103609	3.61	2.22	0.041
Meta	Hybrid Kelly-MPT	5	OFF	100000	102926	2.93	1.54	0.048
Meta	MPT	5	OFF	100000	102825	2.83	1.47	0.049
Meta	Kelly	1	OFF	100000	101719	1.72	1.22	0.036
Meta	Hybrid Kelly-MPT	1	OFF	100000	101467	1.47	1.11	0.034
Meta	MPT	1	OFF	100000	100942	0.94	0.75	0.032

These results show that the ordinal system exhibits a different response to meta-features than the

binary system, particularly at the longer horizon where the strongest non-meta results remain difficult to surpass.

A.7.5 Cumulative Return Figures

The following figures provide the cumulative portfolio return curves corresponding to the full simulation tables above. These plots are useful for understanding how performance evolved over time rather than only at the final end-of-period portfolio value.

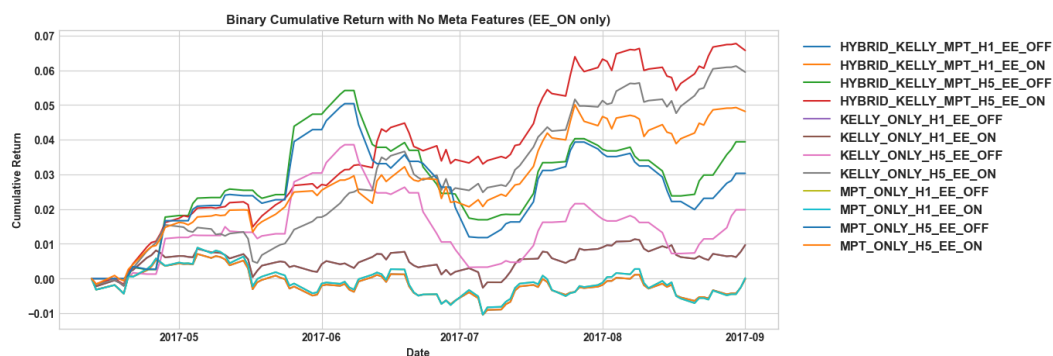


Figure A.9: Cumulative portfolio returns for binary strategies without meta-features.

Figure A.9 shows the binary no-meta trajectories, where the 5-day strategies exhibit clearer upward drift than the weaker 1-day strategies.

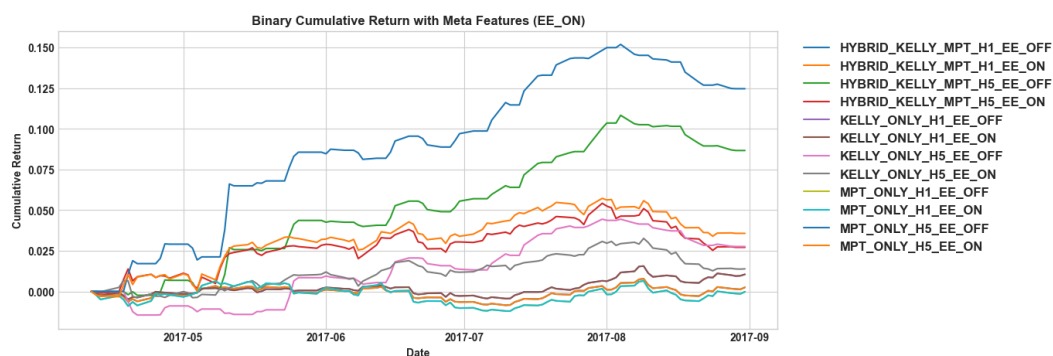


Figure A.10: Cumulative portfolio returns for binary strategies with meta-features.

Figure A.10 shows the corresponding binary meta-feature trajectories. These curves illustrate the stronger cumulative growth observed in the best-performing 5-day binary meta strategies.

Figure A.11 shows the ordinal no-meta case, where the 5-day strategies outperform the 1-day configurations by a substantial margin.

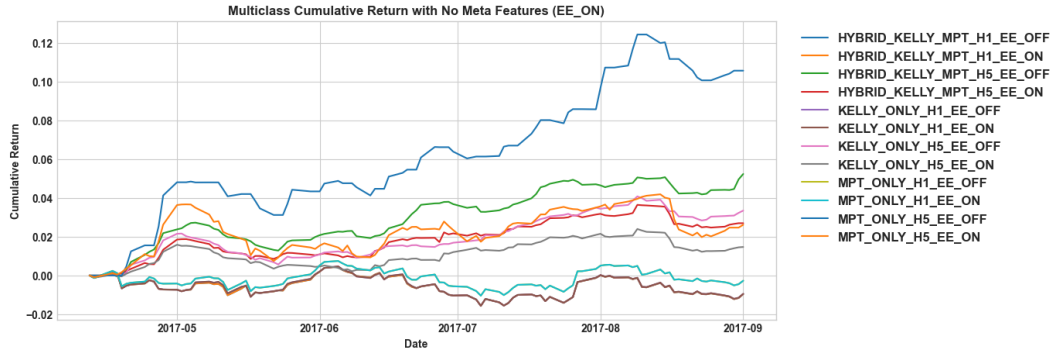


Figure A.11: Cumulative portfolio returns for ordinal multi-class strategies without meta-features.

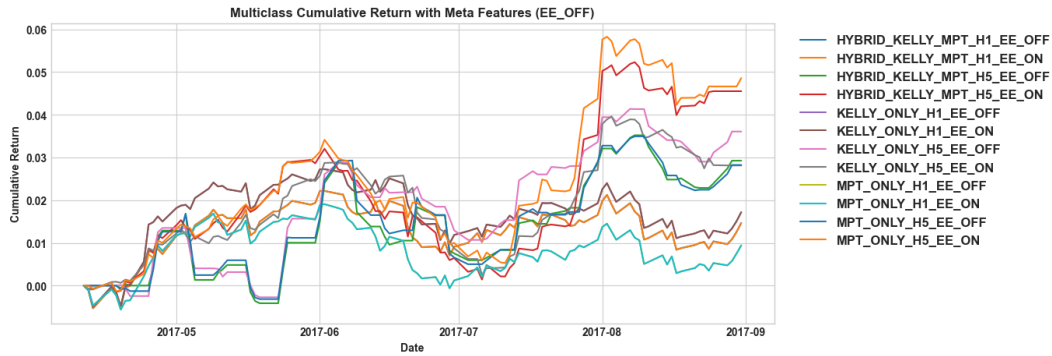


Figure A.12: Cumulative portfolio returns for ordinal multi-class strategies with meta-features.

Figure A.12 shows the ordinal meta-feature trajectories. Compared with the non-meta case, these curves illustrate a weaker long-horizon return profile but improved short-horizon stability.

A.8 Risk and Early-Exit Supplementary Results

This appendix provides the detailed risk and early-exit evidence supporting the discussion in Chapter 6. The tables and figures below report early-exit activity, drawdown behaviour, and rolling 20-day risk statistics for the binary and ordinal systems, both with and without meta-features.

Table A.18 summarises how often the early-exit mechanism was triggered across tasks, settings, and allocation strategies. These counts provide additional context for the early-exit analysis in the main text by showing how actively the liquidation rule intervened during the simulation period.

The ordinal no-meta Kelly strategy shows the highest exit frequency, indicating a comparatively unstable signal environment in that setting.

Table A.18: Summary of early-exit activity across tasks and settings.

Task	Setting	Strategy	Total Exits	Avg Exits/Day
Binary	No Meta	Kelly	341	3.41
Binary	No Meta	MPT	337	3.37
Binary	No Meta	Hybrid Kelly–MPT	269	2.69
Binary	Meta	Kelly	353	3.53
Binary	Meta	MPT	317	3.17
Binary	Meta	Hybrid Kelly–MPT	309	3.09
Ordinal	No Meta	Kelly	561	5.61
Ordinal	No Meta	MPT	313	3.13
Ordinal	No Meta	Hybrid Kelly–MPT	307	3.07
Ordinal	Meta	Kelly	342	3.42
Ordinal	Meta	MPT	324	3.24
Ordinal	Meta	Hybrid Kelly–MPT	267	2.67

A.8.1 Binary System Without Meta-Features

This subsection reports the full binary risk results without meta-features. It includes drawdown summaries, end-of-drawdown recovery values, rolling volatility, rolling Sharpe statistics, and the corresponding figures.

Full Drawdown Statistics Table A.19 reports the maximum drawdown for each binary no-meta configuration together with the associated peak and trough dates. These values indicate the severity and timing of the largest capital declines experienced during the simulation.

Table A.19: Full drawdown statistics for the binary system without meta-features.

Mode	Horizon	Early Exit	Max DD	Peak Date	Trough Date
MPT	5	OFF	-3.6713%	2017-06-06	2017-07-05
HYBRID	5	OFF	-3.5307%	2017-06-06	2017-07-05
KELLY	5	OFF	-3.4001%	2017-06-06	2017-07-03
MPT	1	OFF	-1.9150%	2017-05-05	2017-07-06
MPT	1	ON	-1.9150%	2017-05-05	2017-07-06
HYBRID	1	OFF	-1.7415%	2017-05-05	2017-07-06
HYBRID	1	ON	-1.7415%	2017-05-05	2017-07-06
KELLY	5	ON	-1.1399%	2017-06-19	2017-06-29
HYBRID	5	ON	-1.1350%	2017-08-09	2017-08-17
MPT	5	ON	-1.1174%	2017-06-19	2017-07-03
KELLY	1	OFF	-1.1143%	2017-05-05	2017-07-06
KELLY	1	ON	-1.1143%	2017-05-05	2017-07-06

The largest drawdowns occur in the 5-day no-early-exit setting, while enabling early exit substantially reduces the worst peak-to-trough losses.

Table A.20 reports the end-of-drawdown values, which provide an indication of how far each strategy had recovered from its worst drawdown by the end of the simulation period.

Table A.20: End-of-drawdown recovery values for the binary system without meta-features.

Mode	Horizon	Early Exit	End DD
MPT	5	OFF	-1.9096%
HYBRID	5	OFF	-1.4013%
KELLY	5	OFF	-1.8030%
MPT	1	OFF	-0.8726%
HYBRID	1	OFF	-0.7219%
KELLY	5	ON	-0.1640%
HYBRID	5	ON	-0.1869%
MPT	5	ON	-0.1857%
KELLY	1	OFF	-0.1652%

Rolling Risk Metrics (20-Day Window) The next two tables summarise rolling 20-day volatility and Sharpe ratio behaviour for the same binary no-meta strategies. These statistics provide a time-varying view of risk rather than a single end-of-period summary.

Table A.21: Rolling volatility statistics (20-day window) for the binary system without meta-features.

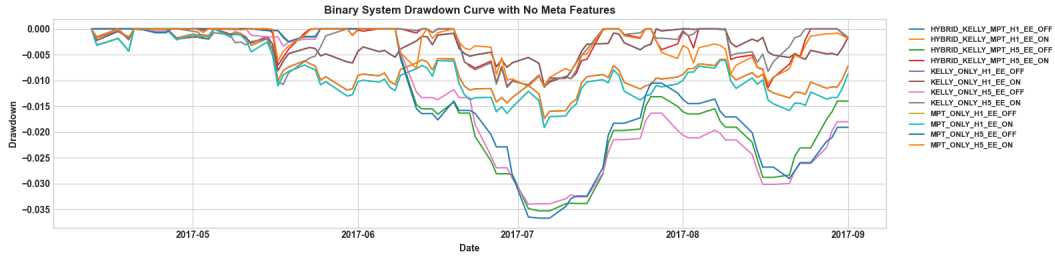
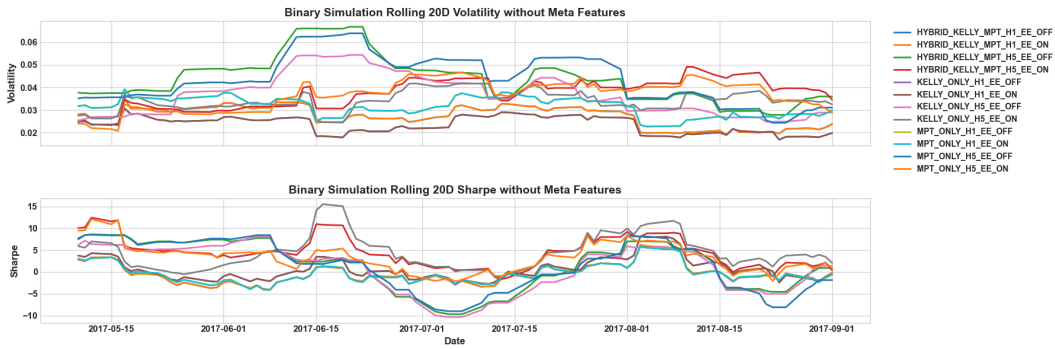
Mode	Horizon	Early Exit	Mean Vol	Max Vol
HYBRID	5	ON	0.0374	0.0491
KELLY	5	ON	0.0342	0.0418
MPT	5	ON	0.0365	0.0467
HYBRID	5	OFF	0.0434	0.0668
MPT	5	OFF	0.0433	0.0639
KELLY	5	OFF	0.0365	0.0544
KELLY	1	OFF	0.0236	0.0307
MPT	1	OFF	0.0314	0.0392
HYBRID	1	OFF	0.0272	0.0350

These rolling results reinforce the main-text finding that early exit materially improves the stronger 5-day binary no-meta strategies.

Table A.22: Rolling Sharpe statistics (20-day window) for the binary system without meta-features.

Mode	Horizon	Early Exit	Mean Sharpe	Max Sharpe	Min Sharpe
HYBRID	5	ON	4.5074	12.4359	-1.6577
KELLY	5	ON	4.4724	15.5176	-1.2235
MPT	5	ON	3.3261	12.0925	-2.1337
HYBRID	5	OFF	1.5083	8.6150	-9.6907
MPT	5	OFF	1.4112	8.5256	-9.0081
KELLY	5	OFF	0.8861	8.0608	-10.3427
KELLY	1	OFF	0.6285	8.1392	-2.9141
MPT	1	OFF	-0.3875	5.9018	-4.1504
HYBRID	1	OFF	-0.4389	6.1727	-4.0468

Figures The figures below provide the corresponding visual drawdown and rolling-risk trajectories for the binary no-meta system.

**Figure A.13:** Drawdown curves for the binary system without meta-features.**Figure A.14:** Rolling 20-day volatility and Sharpe ratio for the binary system without meta-features.

A.8.2 Binary System With Meta-Features

This subsection reports the corresponding detailed results for the binary system with meta-features. These tables allow direct comparison with the no-meta case above.

Full Drawdown Statistics Table A.23 reports the maximum drawdown statistics for the binary meta-feature system.

The binary meta-feature system exhibits smaller worst drawdowns than the strongest no-meta 5-day strategies, although the relationship with early exit differs from the no-meta case.

Table A.23: Full drawdown statistics for the binary system with meta-features.

Mode	Horizon	Early Exit	Max DD	Peak Date	Trough Date
HYBRID	5	ON	-2.7465%	2017-07-31	2017-08-24
MPT	5	OFF	-2.3648%	2017-08-04	2017-08-29
MPT	5	ON	-2.2019%	2017-07-31	2017-08-24
HYBRID	5	OFF	-1.9556%	2017-08-04	2017-08-29
KELLY	5	ON	-1.9302%	2017-08-09	2017-08-24
MPT	1	OFF	-1.8300%	2017-05-08	2017-07-11
MPT	1	ON	-1.8300%	2017-05-08	2017-07-11
KELLY	5	OFF	-1.6096%	2017-07-31	2017-08-29
HYBRID	1	OFF	-1.1828%	2017-05-08	2017-07-10
HYBRID	1	ON	-1.1828%	2017-05-08	2017-07-10
KELLY	1	OFF	-1.0215%	2017-08-09	2017-08-21
KELLY	1	ON	-1.0215%	2017-08-09	2017-08-21

Table A.24: End-of-drawdown recovery values for the binary system with meta-features.

Mode	Horizon	Early Exit	End DD
HYBRID	5	ON	-2.5691%
MPT	5	OFF	-2.3648%
MPT	5	ON	-2.0337%
HYBRID	5	OFF	-1.9556%
KELLY	5	ON	-1.8127%
MPT	1	OFF	-0.6648%
HYBRID	1	OFF	-0.5273%
KELLY	1	OFF	-0.5013%

Rolling Risk Metrics (20-Day Window) The rolling volatility and rolling Sharpe tables below provide the time-varying risk profile for the binary meta-feature system.

These results show that meta-features improve the stronger binary 5-day risk-adjusted performance, although they do not consistently reduce volatility.

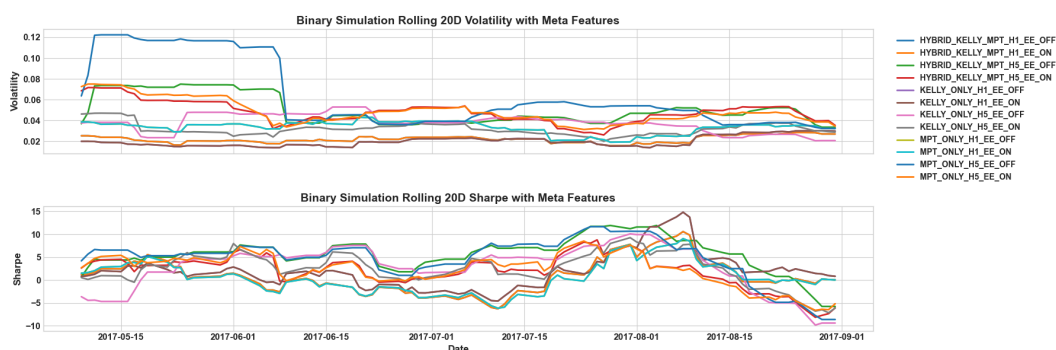
Table A.25: Rolling volatility statistics (20-day window) for the binary system with meta-features.

Mode	Horizon	Early Exit	Mean Vol	Max Vol
HYBRID	5	OFF	0.0499	0.0750
MPT	5	OFF	0.0630	0.1223
KELLY	5	OFF	0.0375	0.0530
KELLY	5	ON	0.0318	0.0471
MPT	5	ON	0.0482	0.0751
HYBRID	5	ON	0.0478	0.0717
KELLY	1	OFF	0.0201	0.0302
HYBRID	1	OFF	0.0220	0.0285
MPT	1	OFF	0.0331	0.0432

Table A.26: Rolling Sharpe statistics (20-day window) for the binary system with meta-features.

Mode	Horizon	Early Exit	Mean Sharpe	Max Sharpe	Min Sharpe
HYBRID	5	OFF	5.5423	11.9235	-5.7633
MPT	5	OFF	4.7818	11.6608	-8.6113
KELLY	5	OFF	2.7579	10.0893	-9.8231
KELLY	5	ON	2.4492	9.2632	-7.1356
MPT	5	ON	2.2816	8.5262	-6.6045
HYBRID	5	ON	2.0146	8.7736	-8.1283
KELLY	1	OFF	1.9283	14.8209	-4.5821
HYBRID	1	OFF	0.7741	10.5883	-6.2758
MPT	1	OFF	0.5262	9.0511	-6.1493

Figures The figure below gives the rolling 20-day volatility and Sharpe ratio profile for the binary meta-feature system.

**Figure A.15:** Rolling 20-day volatility and Sharpe ratio for the binary system with meta-features.

A.8.3 Ordinal Multi-Class System Without Meta-Features

This subsection reports the drawdown and rolling-risk statistics for the ordinal multi-class system without meta-features.

Full Drawdown Statistics Table A.27 reports the full drawdown summary for the ordinal no-meta system.

Table A.27: Full drawdown statistics for the ordinal multi-class system without meta-features.

Mode	Horizon	Early Exit	Max DD	Peak Date	Trough Date	End DD
MPT	5	ON	-2.8165%	2017-05-03	2017-05-22	-1.5070%
MPT	5	OFF	-2.1024%	2017-08-09	2017-08-23	-1.6585%
KELLY	1	OFF	-2.0267%	2017-06-05	2017-07-06	-1.4094%
KELLY	1	ON	-2.0267%	2017-06-05	2017-07-06	-1.4094%
HYBRID	1	OFF	-2.0061%	2017-06-05	2017-07-06	-1.4018%
HYBRID	1	ON	-2.0061%	2017-06-05	2017-07-06	-1.4018%
MPT	1	OFF	-1.7767%	2017-06-05	2017-07-06	-1.0110%
MPT	1	ON	-1.7767%	2017-06-05	2017-07-06	-1.0110%
KELLY	5	OFF	-1.5615%	2017-05-02	2017-05-23	-0.7049%
HYBRID	5	OFF	-1.4012%	2017-05-05	2017-05-23	0.0000%
KELLY	5	ON	-1.3464%	2017-05-01	2017-06-08	-0.9057%
HYBRID	5	ON	-1.1294%	2017-08-09	2017-08-25	-0.9121%

The ordinal no-meta system shows stronger 5-day performance overall, but still experiences distinct drawdown episodes, particularly under MPT.

Rolling Risk Metrics (20-Day Window) The following table reports rolling volatility and Sharpe behaviour for the ordinal no-meta strategies.

Table A.28: Rolling risk statistics for the ordinal multi-class system without meta-features.

Mode	Horizon	Early Exit	Mean Vol	Max Vol	Mean Sharpe	Min Sharpe
KELLY	5	OFF	0.0194	0.0335	5.0339	-8.4097
MPT	5	OFF	0.0529	0.0754	4.8418	-1.2871
HYBRID	5	OFF	0.0232	0.0343	4.3161	-4.3363
HYBRID	5	ON	0.0226	0.0328	3.1742	-6.0076
KELLY	5	ON	0.0195	0.0281	1.8827	-8.5642
MPT	5	ON	0.0430	0.0728	1.6803	-6.1705
MPT	1	OFF	0.0327	0.0403	-0.0050	-6.8335
MPT	1	ON	0.0327	0.0403	-0.0050	-6.8335
KELLY	1	OFF	0.0344	0.0427	-0.5186	-7.7029
KELLY	1	ON	0.0344	0.0427	-0.5186	-7.7029
HYBRID	1	OFF	0.0344	0.0427	-0.5414	-7.8300
HYBRID	1	ON	0.0344	0.0427	-0.5414	-7.8300

These results show that the strongest ordinal no-meta risk-adjusted performance occurs in the 5-day setting without early exit.

Figures The following figures show the corresponding drawdown and rolling-risk trajectories for the ordinal no-meta system.

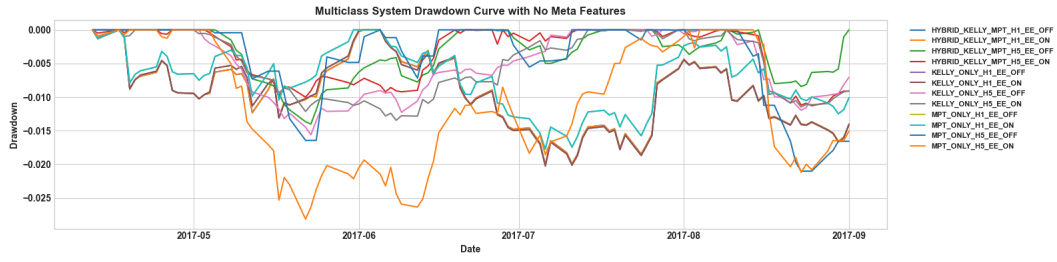


Figure A.16: Drawdown curves for the ordinal multi-class system without meta-features.

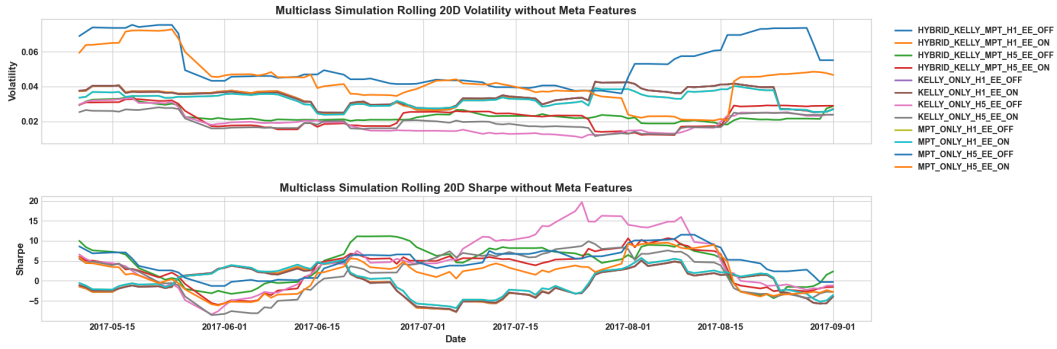


Figure A.17: Rolling 20-day volatility and Sharpe ratio for the ordinal multi-class system without meta-features.

A.8.4 Ordinal Multi-Class System With Meta-Features

This subsection reports the corresponding ordinal drawdown and rolling-risk results once meta-features are introduced.

Full Drawdown Statistics Table A.29 reports the full drawdown summary for the ordinal meta-feature system.

Relative to the no-meta ordinal case, the ranking of drawdown severity changes, particularly once early exit is enabled.

Table A.29: Full drawdown statistics for the ordinal multi-class system with meta-features.

Mode	Horizon	Early Exit	Max DD	Peak Date	Trough Date	End DD
HYBRID	5	ON	-2.9671%	2017-06-02	2017-07-06	-0.6461%
MPT	5	ON	-2.7934%	2017-06-02	2017-07-11	-0.9154%
KELLY	5	ON	-2.5586%	2017-06-05	2017-07-06	-1.1046%
MPT	5	OFF	-2.3583%	2017-06-08	2017-07-03	-0.6476%
HYBRID	5	OFF	-2.2126%	2017-06-05	2017-07-03	-0.5747%
MPT	1	OFF	-1.9330%	2017-06-02	2017-06-29	-0.9468%
MPT	1	ON	-1.9330%	2017-06-02	2017-06-29	-0.9468%
KELLY	5	OFF	-1.7505%	2017-06-05	2017-07-03	-0.5098%
KELLY	1	OFF	-1.6994%	2017-06-02	2017-06-29	-0.9801%
KELLY	1	ON	-1.6994%	2017-06-02	2017-06-29	-0.9801%
HYBRID	1	OFF	-1.4798%	2017-06-02	2017-06-29	-0.7342%
HYBRID	1	ON	-1.4798%	2017-06-02	2017-06-29	-0.7342%

Rolling Risk Metrics (20-Day Window) The table below reports rolling volatility and rolling Sharpe for the ordinal meta-feature system.

Table A.30: Rolling risk statistics for the ordinal multi-class system with meta-features.

Mode	Horizon	Early Exit	Mean Vol	Max Vol	Mean Sharpe	Min Sharpe
MPT	5	ON	0.0455	0.0766	2.5841	-8.0653
KELLY	5	ON	0.0342	0.0477	2.2652	-9.8862
HYBRID	5	ON	0.0455	0.0734	2.1259	-8.3034
KELLY	5	OFF	0.0410	0.0554	1.8737	-9.3084
HYBRID	5	OFF	0.0496	0.0699	1.8143	-5.1992
MPT	5	OFF	0.0509	0.0686	1.6046	-5.4583
HYBRID	1	OFF	0.0319	0.0424	0.7041	-6.2006
HYBRID	1	ON	0.0319	0.0424	0.7041	-6.2006
KELLY	1	OFF	0.0327	0.0488	0.4767	-6.8043
KELLY	1	ON	0.0327	0.0488	0.4767	-6.8043
MPT	1	OFF	0.0311	0.0391	0.1752	-8.9869
MPT	1	ON	0.0311	0.0391	0.1752	-8.9869

These values show that the strongest ordinal meta-feature Sharpe ratios occur in the 5-day early-exit setting, in contrast to the no-meta case.

Figures The figure below shows the rolling 20-day volatility and Sharpe profile for the ordinal meta-feature system.

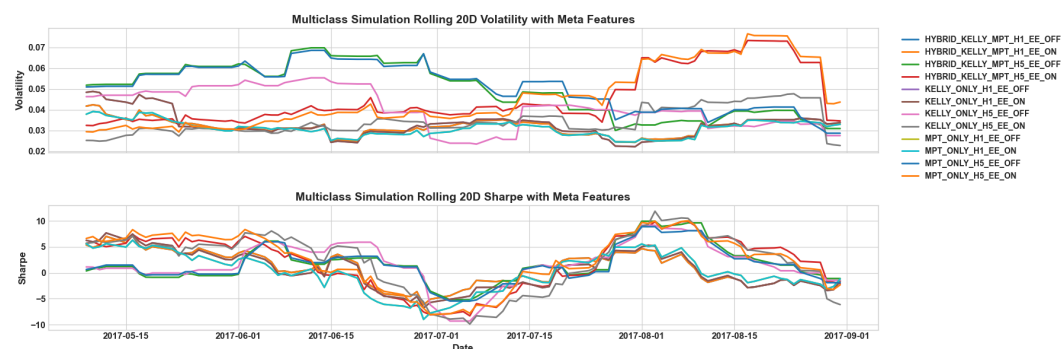


Figure A.18: Rolling 20-day volatility and Sharpe ratio for the ordinal multi-class system with meta-features.

B | Source Code

The full implementation described in this dissertation is available on GitHub:

[Source Code Repository](#)¹.

¹GitHub repository: github.com/dhdylanhuang/Stock-Trend-Prediction-and-Investment-Strategy-Optimisation-Using-Text-and-Time-Series-Data

Bibliography

- Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M. (2019), ‘Optuna: A next-generation hyperparameter optimization framework’.
URL: <https://arxiv.org/abs/1907.10902>
- Araci, D. (2019), ‘Finbert: Financial sentiment analysis with pre-trained language models’.
URL: <https://arxiv.org/abs/1908.10063>
- Arratia, A., Daou, M., Kagerhuber, J. and Smolyarova, Y. (2025), ‘Examining challenges in implied volatility forecasting: A critical review of data leakage and feature engineering combined with high-complexity models’, *Computational Economics* pp. 1–23.
- Bailey, D., Borwein, J. J., Lopez de Prado, M. and Zhu, Q. (2013), ‘The probability of back-test over-fitting’, *SSRN Electronic Journal*.
- Bequé, A., Coussemont, K., Gayler, R. and Lessmann, S. (2017), ‘Approaches for credit scorecard calibration: An empirical analysis’, *Knowledge-Based Systems* **134**, 213–227.
URL: <https://www.sciencedirect.com/science/article/pii/S0950705117303568>
- Blaskowitz, O. and Herwartz, H. (2014), ‘Testing the value of directional forecasts in the presence of serial correlation’, *International Journal of Forecasting* **30**(1), 30–42.
URL: <https://www.sciencedirect.com/science/article/pii/S0169207013000873>
- Brassington, G. (2017), Mean absolute error and root mean square error: which is the better metric for assessing model performance?, in ‘EGU General Assembly Conference Abstracts’, EGU General Assembly Conference Abstracts, p. 3574.
- Breiman, L. (1962), Optimal gambling systems for favorable games.
URL: <https://api.semanticscholar.org/CorpusID:40312799>
- Brown, R. G. (1963), *Smoothing, forecasting and prediction of discrete time series.*, number 468 p. in ‘Prentice-Hall quantitative methods series’, Prentice-Hall, Englewood Cliffs, N.J.
URL: <https://catalog.hathitrust.org/Record/001512022>
- Bustarviejo, J. and Bousoño-Calzón, C. (2026), ‘Multimodal information fusion for financial forecasting via cross-attention and calibrated uncertainty’, *Machine Learning with Applications* **23**, 100840.
URL: <https://www.sciencedirect.com/science/article/pii/S2666827026000058>
- Cahuantzi, R., Chen, X. and Güttel, S. (2023), *A Comparison of LSTM and GRU Networks for Learning Symbolic Sequences*, Springer Nature Switzerland, p. 771–785.
URL: http://dx.doi.org/10.1007/978-3-031-37963-5_3
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014), ‘Learning phrase representations using rnn encoder–decoder for statistical machine translation’.
URL: <https://arxiv.org/abs/1406.1078>

- Chung, J., Kastner, K., Dinh, L., Goel, K., Courville, A. and Bengio, Y. (2016), 'A recurrent latent variable model for sequential data'.
URL: <https://arxiv.org/abs/1506.02216>
- de Prado, M. L. (2018), 'Advances in financial machine learning', pp. 1–393.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019), 'Bert: Pre-training of deep bidirectional transformers for language understanding'.
URL: <https://arxiv.org/abs/1810.04805>
- Dimitriadis, T., Gneiting, T., Jordan, A. I. and Vogel, P. (2024), 'Evaluating probabilistic classifiers: The triptych', *International Journal of Forecasting* **40**(3), 1101–1122.
URL: <https://www.sciencedirect.com/science/article/pii/S0169207023000997>
- Elton, E., Gruber, M. and Brown, S. (2007), 'Modern portfolio theory and investment analysis / e.j. elton... [et al.]'.
- Fama, E. F. and French, K. R. (1988), 'Dividend yields and expected stock returns', *Journal of Financial Economics* **22**(1), 3–25.
URL: <https://www.sciencedirect.com/science/article/pii/0304405X88900207>
- Feng, F., Chen, H., He, X., Ding, J., Sun, M. and Chua, T.-S. (2019), 'Enhancing stock movement prediction with adversarial training'.
URL: <https://arxiv.org/abs/1810.09936>
- Fischer, T. and Krauss, C. (2018), 'Deep learning with long short-term memory networks for financial market predictions', *European Journal of Operational Research* **270**(2), 654–669.
URL: <https://www.sciencedirect.com/science/article/pii/S0377221717310652>
- Giantsidi, S. and Tarantola, C. (2025), 'Deep learning for financial forecasting: A review of recent trends', *International Review of Economics Finance* **104**, 104719.
- Gneiting, T., Balabdaoui, F. and Raftery, A. (2007), 'Probabilistic forecasts, calibration and sharpness', *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* **69**, 243 – 268.
- Graves, A. and Schmidhuber, J. (2005), 'Frame-wise phoneme classification with bidirectional lstm and other neural network architectures', *Neural Networks* **18**(5), 602–610. IJCNN 2005.
URL: <https://www.sciencedirect.com/science/article/pii/S08933608005001206>
- Guo, C., Pleiss, G., Sun, Y. and Weinberger, K. Q. (2017), 'On calibration of modern neural networks'.
URL: <https://arxiv.org/abs/1706.04599>
- Hammerschmid, R. and Lohre, H. (2018), 'Regime shifts and stock return predictability', *International Review of Economics Finance* **56**, 138–160.
URL: <https://www.sciencedirect.com/science/article/pii/S1059056017308134>
- Hartmann, J. (2022), 'Emotion english distilroberta-base', <https://huggingface.co/j-hartmann/emotion-english-distilroberta-base/>.
- He, S. and Gu, S. (2022), 'Multi-modal attention network for stock movements prediction'.
URL: <https://arxiv.org/abs/2112.13593>
- Hiruta, K. and Senoguchi, J. (2025), 'Wavelet-cnn for temporal data: Enhancing long-term stock price prediction via multi-resolution wavelet decomposition and cnn-based feature extraction', *International Journal of Information Management Data Insights* **5**(2), 100360.
URL: <https://www.sciencedirect.com/science/article/pii/S2667096825000424>

- Ho, M. K., Darman, H. and Musa, S. (2021), 'Stock price prediction using arima, neural network and lstm models', *Journal of Physics: Conference Series* **1988**(1), 012041.
URL: <https://doi.org/10.1088/1742-6596/1988/1/012041>
- Hochreiter, S. and Schmidhuber, J. (1997), 'Long short-term memory', *Neural computation* **9**(8), 1735–1780.
- Hu, Z., Liu, W., Bian, J., Liu, X. and Liu, T.-Y. (2019), 'Listening to chaotic whispers: A deep learning framework for news-oriented stock trend prediction'.
URL: <https://arxiv.org/abs/1712.02136>
- Huang, X., Fang, S., Qiu, S., Yu, C., Du, J. and Yang, C. (2026), 'Teff: Prediction-residual-guided rolling forecasting for multi-horizon time series'.
URL: <https://arxiv.org/abs/2602.22520>
- Huang, Z., Xu, W. and Yu, K. (2015), 'Bidirectional lstm-crf models for sequence tagging'.
URL: <https://arxiv.org/abs/1508.01991>
- Kelly Jr., J. L. (1956), 'A new interpretation of information rate', *Bell System Technical Journal* **35**(4), 917–926.
URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/j.1538-7305.1956.tb03809.x>
- Kook, L., Herzog, L., Hothorn, T., Dürr, O. and Sick, B. (2021), 'Deep and interpretable regression models for ordinal outcomes'.
URL: <https://arxiv.org/abs/2010.08376>
- Li, W., Bao, R., Harimoto, K., Chen, D., Xu, J. and Su, Q. (2020), Modeling the stock relation with graph network for overnight stock movement prediction, in C. Bessiere, ed., 'Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20', International Joint Conferences on Artificial Intelligence Organization, pp. 4541–4547. Special Track on AI in FinTech.
URL: <https://doi.org/10.24963/ijcai.2020/626>
- Lin, C. Y. and Lobo Marques, J. A. (2024), 'Stock market prediction using artificial intelligence: A systematic review of systematic reviews', *Social Sciences Humanities Open* **9**, 100864.
URL: <https://www.sciencedirect.com/science/article/pii/S2590291124000615>
- Liu, D., Tang, Z. and Cai, Y. (2022), 'A hybrid model for china's soybean spot price prediction by integrating ceemdan with fuzzy entropy clustering and cnn-gru-attention', *Sustainability* **14**, 15522.
- Luo, X., Kang, Y. and Luo, X. (2025), 'Bayesian forecast combination with predictive priors via particle filtering'.
URL: <https://arxiv.org/abs/2508.07136>
- MacLean, L. C., Thorp, E. O. and Ziemba, W. T. (2011), *The Kelly Capital Growth Investment Criterion*, WORLD SCIENTIFIC.
URL: <https://www.worldscientific.com/doi/abs/10.1142/7598>
- Maracani, A., Rosasco, L. and Natale, L. (2024), 'Trust and balance: Few trusted samples pseudo-labeling and temperature scaled loss for effective source-free unsupervised domain adaptation'.
- Markowitz, H. (1952), 'Portfolio selection', *The Journal of Finance* **7**(1), 77–91.
URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1952.tb01525.x>
- Markowitz, H. M. (1959), *Portfolio Selection: Efficient Diversification of Investments*, Yale University Press.
URL: <http://www.jstor.org/stable/j.ctt1bh4c8h>

- Micallef, A., Apap, M., Licari, J., Spiteri Staines, C. and Xiao, Z. (2025), 'A comparative framework for evaluating machine learning models in forecasting electricity demand for port microgrids', *Energy and AI* **20**, 100494.
URL: <https://www.sciencedirect.com/science/article/pii/S2666546825000266>
- Niculescu-Mizil, A. and Caruana, R. (2005), Predicting good probabilities with supervised learning, pp. 625–632.
- Noorian, A., Harounabadi, A. and Hazratifard, M. (2023), 'A sequential neural recommendation system exploiting bert and lstm on social media posts', *Complex Intelligent Systems* **10**.
- Pesaran, M. H. and Timmermann, A. (2002), 'Market timing and return prediction under model instability', *Journal of Empirical Finance* **9**(5), 495–510.
URL: <https://ideas.repec.org/a/eee/empfin/v9y2002i5p495-510.html>
- Peterson, Z. (2018), 'Kelly's criterion in portfolio optimization: A decoupled problem'.
URL: <https://arxiv.org/abs/1710.00431>
- Rousseau, A.-J., Becker, T., Appeltans, S., Blaschko, M. and Valkenborg, D. (2025), 'Post hoc calibration of medical segmentation models', *Discover Applied Sciences* **7**(3), 180.
URL: <https://doi.org/10.1007/s42452-025-06587-0>
- Saha, R., Lakshmanan, L. and Ng, R. (2024), 'Stance detection with explanations', *Computational Linguistics* **50**, 193–235.
- Schuster, M. and Paliwal, K. (1997), 'Bidirectional recurrent neural networks', *IEEE Transactions on Signal Processing* **45**(11), 2673–2681.
- Schwert, G. W. (1989), 'Why does stock market volatility change over time?', *The Journal of Finance* **44**(5), 1115–1153.
URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1989.tb02647.x>
- Sharpe, W. F. (1994), 'The sharpe ratio', *Journal of Portfolio Management* **21**(1), 49. Copyright - Copyright Euromoney Institutional Investor PLC Fall 1994.
URL: <http://ezproxy.lib.gla.ac.uk/login?url=https://www.proquest.com/scholarly-journals/sharpe-ratio/docview/195581284/se-2>
- Shuangao, W., Yi, L., Padmanaban, R., Shamsudeen, M. and R, S. (2020), 'Comparison of back propagation, long short-term memory (lstm), attention-based lstm neural networks application in futures market of china using r programming', *International Journal of Advanced Computer Science and Information Technology* **6**, 101–114.
- Sinha, S., Mishra, S., Mishra, V. and Ahmed, T. (2022), 'Sector influence aware stock trend prediction using 3d convolutional neural network', *Journal of King Saud University - Computer and Information Sciences* **34**(4), 1511–1522.
URL: <https://www.sciencedirect.com/science/article/pii/S1319157822000416>
- Souza, L. A. (2026), Performance-driven causal signal engineering for financial markets under non-stationarity, Papers 2603.13638, arXiv.org.
URL: <https://ideas.repec.org/p/arx/papers/2603.13638.html>
- Stan, S., Lunghi, M., Vargetto, V., Ricci, C., Repetto, R., Leo, B. and Gan, S.-H. (2026), 'Causal reconstruction of sentiment signals from sparse news data'.
URL: <https://arxiv.org/abs/2603.23568>
- Staudemeyer, R. C. and Morris, E. R. (2019), 'Understanding lstm – a tutorial into long short-term memory recurrent neural networks'.
URL: <https://arxiv.org/abs/1909.09586>

- Thorp, E. (2008), 'The kelly criterion in blackjack, sports betting, and the stock market', *Handbook of Asset and Liability Management* 1.
- Thorp, E. O. (1997), *The Kelly Criterion in Blackjack Sports Betting, and the Stock Market*, chapter 54, pp. 789–832.
URL: https://www.worldscientific.com/doi/abs/10.1142/9789814293501_0054
- Touzani, Y. and Douzi, K. (2021), 'An lstm and gru based trading strategy adapted to the moroccan market', *Journal of Big Data* 8(1), 126.
URL: <https://doi.org/10.1186/s40537-021-00512-z>
- Varshney, R. P. and Sharma, D. K. (2024), 'Optimizing time-series forecasting using stacked deep learning framework with enhanced adaptive moment estimation and error correction', *Expert Systems with Applications* 249, 123487.
URL: <https://www.sciencedirect.com/science/article/pii/S095741742400352X>
- von Plato, J. (2005), Chapter 75 – a.n. kolmogorov, grundbegriffe der wahrscheinlichkeitsrechnung (1933), in I. Grattan-Guinness, R. Cooke, L. Corry, P. Crépel and N. Guicciardini, eds, 'Landmark Writings in Western Mathematics 1640–1940', Elsevier Science, Amsterdam, pp. 960–969.
URL: <https://www.sciencedirect.com/science/article/pii/B978044450871350156X>
- Wang, J., Chen, J., Liu, J., Tang, D., Chen, D. Z. and Wu, J. (2025), 'A survey on ordinal regression: Applications, advances and prospects'.
URL: <https://arxiv.org/abs/2503.00952>
- Wei, H., Xie, R., Cheng, H., Feng, L., An, B. and Li, Y. (2022), 'Mitigating neural network overconfidence with logit normalization'.
URL: <https://arxiv.org/abs/2205.09310>
- Wenqiu, Z., Li, Y., Liu, Q. and Zeng, Z. (2023), 'Micro-expression recognition based on spatio-temporal feature extraction of key regions', *Computers, Materials Continua* 77, 1373–1392.
- Wright, C. S. (2026), 'Utility-weighted forecasting and calibration for risk-adjusted decisions under trading frictions'.
URL: <https://arxiv.org/abs/2601.07852>
- Wu, Y. (2020), Weather transformer for stock return classification with model uncertainty estimation, Technical report, Bloomberg.
URL: <https://assets.bbhub.io/professional/sites/10/1026302weatherTransformerWhitePaper.pdf>
- Xiao, J., Deng, T. and Bi, S. (2024), 'Comparative analysis of lstm, gru, and transformer models for stock price prediction'.
URL: <https://arxiv.org/abs/2411.05790>
- Xu, Y. and Cohen, S. B. (2018), Stock movement prediction from tweets and historical prices, in I. Gurevych and Y. Miyao, eds, 'Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)', Association for Computational Linguistics, Melbourne, Australia, pp. 1970–1979.
URL: <https://aclanthology.org/P18-1183/>
- Yi, W. and Hasuike, T. (2026), 'Smart predict–then–optimize paradigm for portfolio optimization in real markets'.
URL: <https://arxiv.org/abs/2601.04062>

Zadrozny, B. and Elkan, C. (2001), Obtaining calibrated probability estimates from decision trees and naive bayesian classifiers, in 'Proceedings of the Eighteenth International Conference on Machine Learning', ICML '01, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, p. 609–616.

Zadrozny, B. and Elkan, C. (2002), Transforming classifier scores into accurate multiclass probability estimates, in 'Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining', KDD '02, Association for Computing Machinery, New York, NY, USA, p. 694–699.

URL: <https://doi.org/10.1145/775047.775151>

Zhang, X. and Zhang, X. (2023), 'Optimal model averaging based on forward-validation', *Journal of Econometrics* **237**(2, Part C), 105295.

URL: <https://www.sciencedirect.com/science/article/pii/S030440762200094X>

Ziemba, W. T. and MacLean, L. C. (2011), 'Using the kelly criterion for investing'.

URL: https://ideas.repec.org/h/spr/isochp/978-1-4419-9586-5_1.html